

Оглавление

Две формы представления информации. Способы представления дискретной информации. Системы счисления, используемые в вычислительной технике: двоичная, 8-я, 10-я, 16-я, двоично-десятичная	3
Представление чисел с фиксированной точкой. Прямой, обратный и дополнительный код. Формирование битовых признаков переноса, переполнения, отрицательного результата, нуля.....	3
Представление символьных и строковых данных. Принципы построения кодовых таблиц ASCII, KOI-8, ISO8859-5, Windows-1251, UTF-8, UTF-16.	4
Базовые элементы вычислительной техники: ячейки, регистры, шины, вентили, тактовые генераторы, логические схемы, триггеры, регистры, счетчики, сумматоры.....	5
Структура и принцип функционирования ЭВМ. Порядок функционирования простого процессора на примере калькулятора.	8
Операционная система Unix — ядро ОС и файловая система.....	9
Операционная система Unix — интерпретаторы, стандартные потоки ввода вывода, фильтры.	10
Операционная система Unix — основные команды, права файлов и способы их задания.	11
Состав и структура БЭВМ. Адресные пространства БЭВМ. Система команд БЭВМ, форматы команд. Машинные циклы.....	13
Организация вычислений в БЭВМ. Сдвиги, арифметические и логические операции. Цикл выборки команды.....	16
Организация массивов данных. Режимы адресации. Цикл выборки адреса и операнда БЭВМ.....	16
Управление вычислительным процессом в БЭВМ. Команды ветвлений, цикл исполнения команды LOOP.	18
Подпрограммы в БЭВМ. Цикл исполнения команд перехода и возврата из подпрограммы. Стек, передача параметров. Позиционно-независимый код. Загрузчик и библиотеки.	19
Организация ввода-вывода в вычислительных системах. Инициация обмена, передача информации и завершение обмена. Драйверы.	20
Организация ввода-вывода в БЭВМ. Устройства ввода-вывода, команды	22
Организация асинхронного обмена в БЭВМ. Пример программы. Временные издержки асинхронного обмена.	25
Организация прерываний в БЭВМ. Вектора прерываний, контроллер прерывания.	26
Организация обмена по прерыванию программы в БЭВМ. Пример программы. Цикл прерывания.....	28
Микропрограммный уровень БЭВМ. Структура МПУ. Форматы микрокоманд.	31

Структура и принципы работы арифметико-логического устройства и коммутатора.	
Регистр состояния БЭВМ	34
Микропрограммное управление вентильными схемами. Схема управления.	
Интерпретатор БЭВМ.	37
Архитектура ЭВМ. Гарвардская и фон-Неймановская архитектура. Организация обмена архитектуры ЭВМ с использованием шин.	40
Архитектура многопроцессорных ЭВМ. Системный коммутатор. Архитектуры UMA и NUMA	41
Структура современных процессоров. Окружение процессора. CISC, RISC, VLIW.....	43
Адресуемая память, организация и временные диаграммы. Конструктивные особенности современной памяти.	46
Память, ориентированная на записи (блочная память). Организация дисковой памяти и памяти на магнитных лентах.	48
Характеристики запоминающих устройств. Пирамида памяти.....	50
Ассоциативная память, Кэш-память. Влияние промахов кэш-памяти на производительность.	52
Предназначение и организация виртуальной памяти. Сегментно-страничная организация. Устройство управления памятью (MMU), буфер трансляции (TLB).	54
Сетевые технологии, Понятие сети ЭВМ, классификация компьютерных сетей.	
Сообщение и пакет. Модель взаимодействия открытых систем.	57
Модель TCP/IP: передающая среда, каналный и сетевой уровень. Адресация, передача и маршрутизация пакетов.	59
Модель TCP/IP: выделение адресов (DHCP), сервисы имен, транспортный и прикладной уровни.	62
Интерфейсы ввода-вывода. Контроллеры внешних устройств. Уровни стандартизации, сопряжения с системной шиной, циклы обмена. Регистры контроллера.	64
Параллельная передача данных. Контроллеры параллельной передачи и приема.	66
Синхронные последовательные интерфейсы. Контроллеры последовательной передачи и приема.....	67
Асинхронный обмен. Принцип деления частоты, формат кадра	71
Контроллер передачи асинхронного последовательного интерфейса.....	73
Контроллера приёма асинхронного последовательного интерфейса.	74
Организация прямого доступа к памяти. Контроллер ПДП (DMA).....	75

Две формы представления информации. Способы представления дискретной информации. Системы счисления, используемые в вычислительной технике: двоичная, 8-я, 10-я, 16-я, двоично-десятичная

- 1) Форма 1 – аналоговая. У нас есть реальная величина, а есть аналог в ЭВМ – напряжение, они изменяются по одному закону. Сигнал непрерывен. Аналоговые ЭВМ обладают высоким быстродействием. Погрешность вычисления мала и при этом неизменная, то есть не накапливается, она появляется ввиду тех. ограничений.
- 2) Форма 2 – цифровая. Вместо всевозможного числа уровней сигнала, как в аналоговой форме, используем только 2: 1 и 0. У сигнала есть 3 уровня: нижний – слабый сигнал трактуется как 0; неопределенный; верхний – сильный сигнал трактуется как 1.

Способы представления дискретной инфы

Каждое значение из набора исходных данных может быть представлено в ЭВМ в виде нескольких цифровых сигналов, один из которых соответствует числу единиц, десятков и тд. Десятичная сс не является лучшим способом представления такой информации, тк устройству считывания приходится распознавать 10 разных состояний. Значительно проще было бы использовать устройство только с 2 состояниями. Так появилась 2СС, которая также является позиционной, но использует только цифры 0 и 1. Хотя 2СС удобна для использования в ЭВМ, но она совсем неудобна для чтения и записи человеком. Так появились 8СС и 16СС (по степеням 2), значительно ускорившие ввод и чтение информации человеком

Двоично-десятичная система используется в устройствах, связанных не с обработкой информации, а с выводом ее в десятичном формате, например, в калькуляторах, такой формат позволяет использовать удобства десятичной сс, совмещая с простотой обработки 2сс. Например, дробь 0,1 не имеет точное представления в 2СС, в десятичном формате такой проблемы нет. В такой сс каждая отдельная десятичная цифра кодируется отдельной группой из 4 бит.

Представление чисел с фиксированной точкой. Прямой, обратный и дополнительный код. Формирование битовых признаков переноса, переполнения, отрицательного результата, нуля.

В БЭВМ используются числа с фиксированной точкой, разделяющей целую и дробную части, причем в БЭВМ точка находится строго после младшего бита, то есть дробные числа не поддерживаются.

В БЭВМ используется 16-разрядный регистр, в котором значения могут меняться от $(0000)_{16}$ до $(FFFF)_{16}$. БЭВМ поддерживает + и – числа. Для этого 15-й бит (если считать с 0) отводится под знак, а под число остается 15 бит. Для отображения положительных

чисел и 0 используется прямой код, т. е. такой код, который переводится в 10СС сразу без доп действий. Макс значение положительного числа в БЭВМ = $2^{15}-1$, а 15 бит = 0. Для отображения отрицательных чисел используют дополнительный код, такой код, что при сложении с прямым выдаст $(10000)_{16}$. Его получают так: сначала получают обратный код числа (инвертируют все биты), затем прибавляют 1. Макс значение отрицательного числа = -2^{15} . Доп код позволяет избежать двойного нуля в + и – части.

Одной той же схемой сумматора можно воспользоваться для выполнения операций над знаковыми и беззнаковыми числами. Признаком выхода за разрядную сетку для беззнаковых чисел является перенос в старший разряд (Carry flag). Например:

$$\begin{array}{r} 1000\ 0000\ 0000\ 0000 \\ + 1000\ 0000\ 0000\ 0000 \\ \hline 1\ 0000\ 0000\ 0000\ 0000 \end{array}$$

В ответе должно получиться $32768+32768=65536$, но тк разрядность слова всего 16 бит, то в результате сохраниться 0, единица оказалась в 16 разряде, которого нет.

Признаком переполнения разрядной сетки для знакового числа является бит переполнения (oVerflow flag). Он выставляется, если знаки слагаемых не совпадают со знаком суммы. Программно он выставляется как: перенос за разрядную сетку XOR перенос в старший разряд

Признак отрицательного результата (Negative flag) выставляется если в старшем бите результата стоит 1

Признак нуля (Zero flag) выставляется, когда все биты результата = 0.

Представление символьных и строковых данных. Принципы построения кодовых таблиц ASCII, КОИ-8, ISO8859-5, Windows-1251, UTF-8, UTF-16.

Символьные данные представляются в виде двоичных чисел для БЭВМ. То есть буквам в соответствие ставится позиция в кодовой таблице.

Существуют разные кодировки:

ASCII – американская таблица кодировки с 7 значащими битами и 8 битом четности. Включал спец символы, цифры и только латинские буквы.

КОИ-8 – советский формат для кодирования информации, сделан на основе расширенной ASCII, то есть использует 8 битов для хранения инфы, включает как русские, так и англ буквы. Причем русские буквы располагаются не по алфавиту, а так, чтобы они совпадали с фонетическим аналогом из англ (транслитом) алфавит. Так что если вдруг программа считывала только 7 битов, русский текст все равно бы читался, но только транслитом – англ буквами

ISO8859-5 – стандартная кодировка, включающая латинские и русские буквы, причем русские буквы теперь располагались по алфавиту

Windows-1251 – набор символов и кодировка, являющаяся стандартной 8-битной кодировкой для русских версий Microsoft Windows до 10-й версии

UTF-8 - одна из общепринятых 8-битных и стандартизированных кодировок текста, которая позволяет хранить символы Юникода, используя переменное количество байт (от 1 до 6). Позволяет хранить всевозможные языки мира в одной таблице, так русские буквы занимают 2 байта.

Совместимость с ASCII — любые их 7-битные символы отображаются как есть, а остальные выдают пользователю мусор (шум). Поэтому в случае, если латинские буквы и простейшие знаки препинания (включая пробел) занимают существенный объём текста, UTF-8 даёт выигрыш по объёму по сравнению с UTF-16.

Существуют разные принципы сохранения символов в памяти: с младших битов (little-endian) или со старших (big-endian).

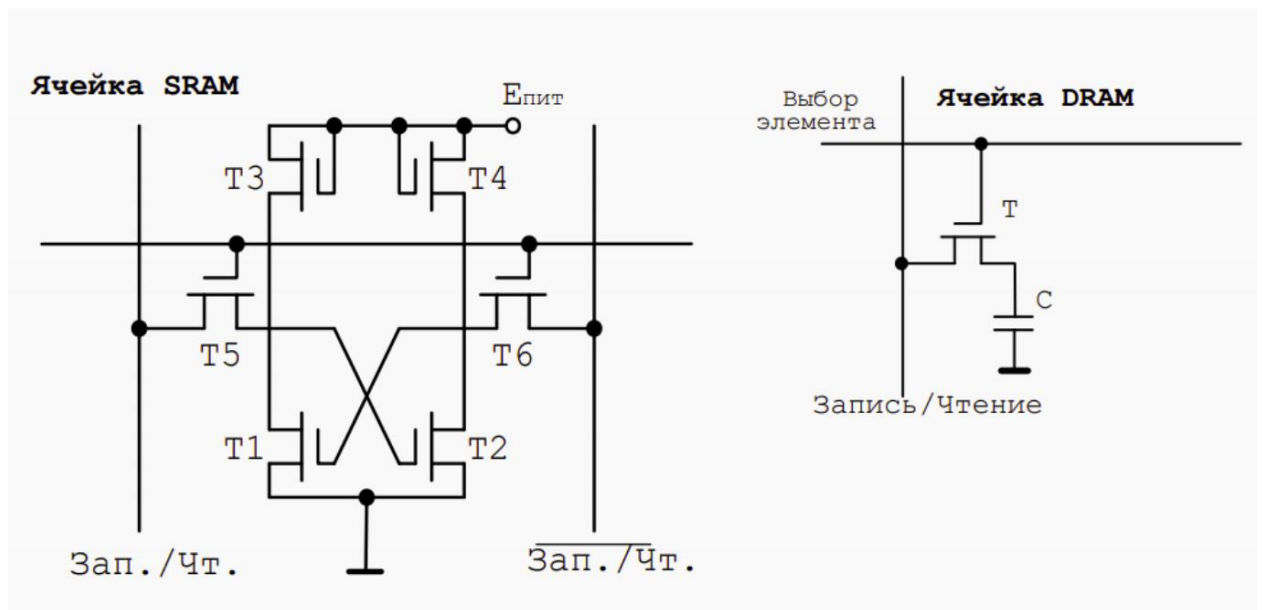
Также существуют разные принципы представления строк: либо строка заканчивается терминальным символом, либо вместе со строкой сначала идет ее длина.

Базовые элементы вычислительной техники: ячейки, регистры, шины, вентили, тактовые генераторы, логические схемы, триггеры, регистры, счетчики, сумматоры.

Ячейка памяти – минимальный адресуемый элемент запоминающего устройства ЭВМ. Ячейки имеют адрес, по которому к ним обращаются команды процессора. Ячейка может хранить либо высокий уровень напряжения, либо низкий. Обычно высокий трактуется как 1, а низкий как ноль, но может быть и наоборот. Память бывает SRAM (static) и DRAM (dynamic)

SRAM – память, в которой двоичный разряд хранится в схеме с положительной обратной связью (транзисторы постоянно подпитывают друг друга), позволяющей поддерживать состояние без регенерации. Тем не менее, сохранять данные без перезаписи SRAM может, только пока есть питание. Используется в кэшах

DRAM – значительно проще в структуре, чем SRAM состоит из транзистора и конденсатора, который постоянно разряжается, потому такая память требует постоянной подзарядки



Регистр процессора – память внутри процессора, предназначенная для хранения адресов и промежуточных результатов вычислений или данных, необходимых для работы самого процессора. Регистр характеризуется единственным числом: количеством битов, которые могут в нем храниться. Операция чтения информации, хранимой в регистре, сводится к созданию копии его содержимого, оригинал же сохраняется в регистре без изменений.

Шина - электрическая цепь, соединяющая регистр с другим регистром или иным устройством ЭВМ. Шина состоит из параллельных проводов, каждый из которых предназначен для передачи соответствующего регистра. Также шина содержит несколько дополнительных проводов, используемых для передачи сигналов синхронизации и управления. Шины служат для передачи информации лишь в направлении, обозначенном стрелкой на шине. Специальные схемы позволяют в одни моменты времени передавать информацию по шине в одну сторону, а в другие – в обратном направлении, т.е. организовать двунаправленную шину.

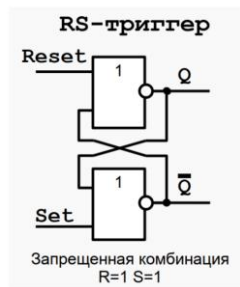
Вентильные схемы – это электронные ключевые схемы, предназначенные для управления потоком информации из регистров в шины и обратно. Такая схема содержит два входа и один выход. На один вход подается информационный сигнал (данные с регистра), а на другой (являющийся вентилем) – управляющий. Если управляющий сигнал равен 1, то данные проходят схему без препятствий, если 0 – никакая информация не пройдет через схему.

Тактовый генератор – устройство, генерирующее электрические импульсы заданной частоты. Используется для синхронизации процессов передачи информации между устройствами ЭВМ

Функциональная логическая схема - совокупность логических элементов (простейшее устройство ЭВМ, выполняющее одну определённую логическую операцию над входными сигналами согласно правилам алгебры логики) и связей между ними.

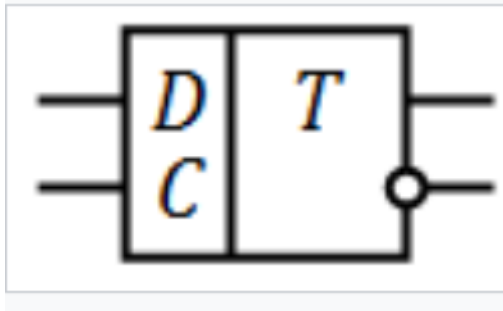
Триггер — класс электронных устройств, обладающих способностью длительно находиться в одном из двух устойчивых состояний и чередовать их под воздействием внешних сигналов.

RS-триггер получил название по названию своих входов. Вход S (Set — установить англ.) позволяет устанавливать выход Q в единичное состояние. (Устанавливать означает записывать логическую единицу). Вход R (Reset — сбросить англ.) позволяет сбрасывать выход Q (Quit — выход англ.) в нулевое



Синхронный RS триггер – подвид RS-триггера, который помимо входов SR имеет и вход C как сигнал синхронизации. В отличие от RS-триггера, синхронный RS-триггер позволяет работать с памятью в определённый такт времени.

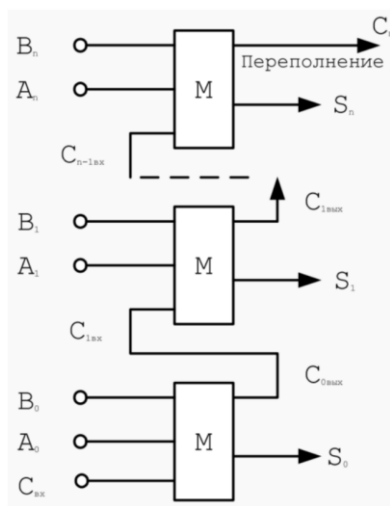
D-триггер запоминает состояние входа и выдаёт его на выход.



T-триггер – это счётный триггер, который имеет всего один вход. При поступлении сигнала на вход, состояние триггера меняется на противоположный.

Счётчик числа импульсов – устройство, которое подразумевает из себя последовательно соединённые T триггеры с дополнительным входом для сброса состояния. Позволяет хранить количество поступивших импульсов, на выходе получается двоичный или двоично-десятичный код.

Сумматор – устройство, которое преобразует исходные информационные сигналы и на выходе выдаёт сумму данных сигналов:



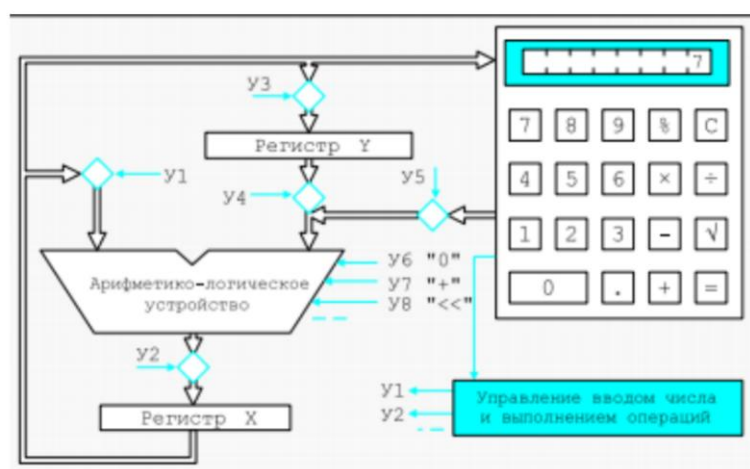
Структура и принцип функционирования ЭВМ. Порядок функционирования простого процессора на примере калькулятора.

Любая ЭВМ состоит из 3-х основных частей: устройство ввода-вывода, процессор и память.

Все вычисления и обработка данных происходит внутри процессора. В свою очередь, процессор состоит из регистров, о которых было сказано в билете выше, АЛУ, которое воспроизводит все логические и арифметические операции, МПУ, которое выполняет микрокод для реализации основных команд ЭВМ.

Устройства ввода-вывода позволяют в удобном для человека формате вводить данные в ЭВМ, а также получать результаты выполнения программ в читаемом виде

Память же нужна для хранения самих программ и входных для неё данных. Каждая ячейка памяти представляет из себя совокупность единиц и нулей. Процессор при помощи адресных команд может обращаться к конкретной ячейке памяти, зная её точный адрес.



Рассмотрим принцип функционирования ЭВМ на примере калькулятора.

Калькулятор состоит из двух регистров: X и Y, который используются для хранения промежуточных вычислений. Есть устройство ввода-вывода –кнопки и экран калькулятора соответственно. Экран калькулятора дублирует содержимое регистра X.

Есть Устройство управление, которое отправляет последовательность сигналов для реализации конкретной функции. Подобную последовательность сигналов называют циклом импульсов, которые управляют вентилями. Каждая группа импульсов выдаётся в определённый такт тактового генератора.

Есть АЛУ с специальными управляющими входами, определяющими выбранную операцию.

Рассмотрим команду ввода операнда в калькулятор. Для начала нужно сохранить предыдущее состояние памяти калькулятора, то есть перебросить значение регистра X в регистр Y. Для ввода нового числа нужно воспроизвести операция “0 + вводимая цифра”. Сначала обнулим регистр X, записав в него 0 со стороны АЛУ. Позже значение обновлённого регистра X подаётся на левый вход, а вводимая цифра на правый вход. Числа суммируются и результат записывается в регистр X.

Операционная система Unix — ядро ОС и файловая система

Unix – семейство переносимых, многозадачных и многопользовательских операционных систем. Изначально была написана на ассемблере как однопользовательская ОС для PDP-7. Позже ОС была портирована на PDP-11, переписанная на язык C. В конце 70-х стали появляться первые ответвления данного семейства операционных систем.

Основное отличие Unix-подобных систем от остальных ОС в том, что это изначально многопользовательская и многозадачная система, то есть в один и тот же момент времени к одной системе могут подключиться сразу несколько пользователей, которые могут выполнять большее количество операций. Даже сама популярная ОС – Microsoft Windows, не считая некоторых серверных версий, является однопользовательской ОС.

Ядро *NIX

Второй плюс системы – это мультиплатформенность. Ядро системы написано таким образом, чтобы его можно было легко адаптировать практически под любое железо. Поговорим конкретно про ядро ОС

Каждая операционная система выполняет служебные операции внутри ядра ОС. Ядро представляет из себя набор подсистем для адекватного функционирования программ на компьютере. Например, в ядре располагаются подсистемы управления оперативной памятью, сетевая подсистема, подсистема управления процессами и потоками, драйверы для взаимодействия с внешними устройствами, подсистема виртуальной памяти (В Unix всё есть файл) и платформо-независимый код. Любое взаимодействия пользователя или программы с ядром происходит через интерфейс системных вызовов. Любое адресное пространство исполняемой программы логически делится на отдельные сегменты, например, сегмент стека или сегмент входных данных программы.

Файловая система

ФС представляется в виде дерева. Верхним элементом иерархии такого дерева называют директорию root, Логические разделы дискового накопителя могут подключаться в файловую систему в специальных точках монтирования, которые создаёт сам администратор. Каждая директория содержит две обязательные ссылки: “.” – ссылка на саму себя, “..” – ссылка на директорию выше по иерархии дерева. В файловой системе есть каталоги, которыми пользуется сама операционная система, например, /usr/bin – директория, в которой располагаются исполняемые файлы и утилиты.

В файловой системе можно адресоваться при помощи абсолютной адресации, указывая полный путь от корня ФС, либо при помощи относительной, указывая путь от текущей директории.

Операционная система Unix — интерпретаторы, стандартные потоки ввода вывода, фильтры.

Всеми командами управляет интерпретатор команд shell. Эта программная оболочка позволяет организовывать выполнение команд, задавать им переменные окружения, анализировать коды возврата команд, организовывать ввод-вывод последовательно между несколькими командами использовать простейший язык для написания исполняемых программ – скриптов.

SH, BASH и KSH – разные типы оболочек shell, которые предоставляют интерфейс для взаимодействия пользователя с командной строкой ОС.

SH – одна из самых первых оболочек, появившихся в UNIX. Имеет базовый синтаксис и простейшие операторы, например, циклы и операторы ветвления.

BASH – это расширенная версия SH, которая является стандартной во многих дистрибутивах Linux. Из нововведений можно отметить появление автодополнения команд, расширение функционала работы с циклами и ветвлениями, история команд и поддержка работы массивов и функций. BASH обратно совместим с SH, поэтому большинство скриптов, написанных на SH, будут работать в BASH.

Потоки ввода-вывода

В каждом процессе UNIX автоматически создаёт три стандартных потока:

- 0 – Стандартный ввод (stdin);
- 1 – Стандартный вывод (stdout);
- 2 – Стандартный вывод ошибок (stderr);

Перенаправление потоков ввода-вывода

“>” – перенаправление выходного потока в файл, причём содержимое файла, если он непустой, будет перезаписан;

“>>” – перенаправление выходного потока в файл, который добавляет данные в конец файла;

“<” – перенаправление входного потока из файла;

“<<” – here-document, позволяющий передать текст напрямую, используя маркер EOF, который пишется в начале и в конце;

“|” – пайп или конвейер, позволяющий вывод одной команды передать на ввод следующей команды;

“> /dev/null” – перенаправление вывода в “чёрную дыру”;

Фильтры

Это команды, которые на вход получают данные, как-то их обрабатывают или агрегируют и результат операции выводится в стандартный поток вывода.

Пример команд: `grep`, `sort`, `tail`, `head`, `wc`.

Операционная система Unix — основные команды, права файлов и способы их задания.

- “`mkdir`” – создание пустой директории;
- “`echo`” – вывод символов в стандартный поток вывода;
- “`cat`” – конкатенация и вывод содержимого файла в стандартный поток вывода;
- “`touch`” – создание пустого файла или обновление последней даты модификации;
- “`ls`” – вывод списка файла в стандартный поток вывода;
- “`pwd`” – вывод текущей директории в стандартный поток вывода;
- “`cd`” – смена текущей директории на указанную;
- “`rm`” – удаление файлов или директории. Флаг “-r” рекурсивно удаляет содержимое директории;
- “`rmdir`” – удаление пустой директории;
- “`mv`” – перемещение файлов;
- “`head`” – вывод первых указанных строк;
- “`tail`” – вывод последних указанных строк;
- “`sort`” – сортировка строк файлов или стандартного ввода;
- “`grep`” – выборка строк по шаблону;
- “`wc`” – подсчёт количества строк, слов и байт в файле;
- “`ln`” – создаёт жёсткую ссылку. Флаг “-s” позволяет создать символическую ссылку;
- “`man`” – выводит справку по командам;

Способ задачи прав пользователей

Команда для выдачи прав: “chmod”. Для каждого файла существуют следующие категории пользователей:

- U (user) – владелец файла;
- G (group) – группа, владеющая файлом;
- O (other) – остальные пользователи;

Каждая из категории может иметь любую комбинацию из следующих файлов:

- R (read) – право на чтение файла, просмотр каталога;
- W (write) – право на запись в файл, добавление или удаление каталога;
- X (execute) – право на исполнение файла, поиск и переход в каталог;

Права принято обозначать в виде трёх восьмиричных чисел:

- 100 = 4 – чтение
- 010 = 2 – запись
- 001 = 1 – исполнение

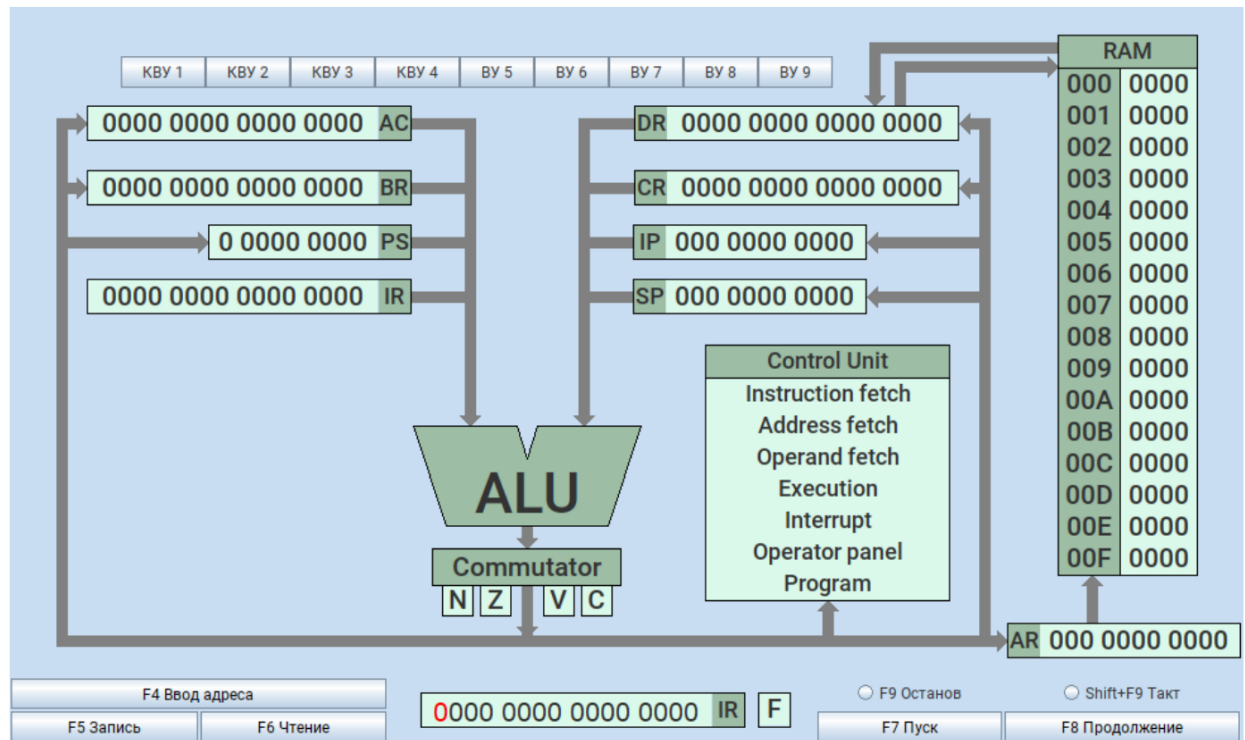
Пример использования: “chmod 644 file.txt”

Другой вариант: использовать формат ugo с помощью =,+,-:

Пример: “chmod ugo=rwx file.txt”, “chmod u+w,g-r,o=rwx file.txt”;

“chmod u=g file.txt” – копирование прав одной категории другой категории пользователей.

Состав и структура БЭВМ. Адресные пространства БЭВМ. Система команд БЭВМ, форматы команд. Машинные ЦИКЛЫ.



Базовая ЭВМ – это простая гипотетическая ЭВМ аккумулятора типа, обладающая характерными чертами, присущими многим ЭВМ. Память состоит из 2048 16-ти разрядных ячеек. Ячейки памяти с 000 – 00F выделяются под вектора прерывания. Процессор состоит из ряда регистров, АЛУ и МПУ:

- AC – 16-ти разрядный аккумулятор, используется как регистр свободного назначения;
- BR – 16-ти разрядный буферный регистр, используется для промежуточных вычислений;
- PS – 9-ти разрядный регистр состояния, нужный для корректной работы БЭВМ.

Содержит следующие значения:

1. C – Carry flag;
2. V – Overflow flag;
3. Z – Zero flag;
4. N – Negative flag;
5. 0 – для организации безусловного перехода;
6. EI – разрешение на прерывание;
7. IRQ – разрешение на прерывание. Значение бита определяется как логическое “И” между EI и сигнала с шины запроса на прерывание;
8. W – состояние тумблера “останов/работа” (1 – работа);

9. Р – программа;

- IR – 16-ти разрядный клавишный регистр.
- DR – если адресный регистр, то будет хранить один из операндов операции, если безадресная команда, то будет храниться сама команда.

Является 16-ти разрядным

- CR – 16-ти разрядный регистр, хранящий полный код команды.
- IP – 11-ти разрядный счётчик команд, указывает на адрес следующей исполняемой команды;
- SP – 11-разрядный счётчик, указывающий на вершину стека
- АЛУ – арифметическо-логическое устройство, производящее все вычислительные операции внутри процессора.
- МПУ – микропрограммное устройство. Выполняет машинные коды конкретной команды, используя интерпретатор БЭВМ.

ЭВМ способна выполнять только определённый перечень команд, что должен учитывать программист. В зависимости от того, к каким блокам памяти БЭВМ обращается, выделяют четыре группы команд: адресные команды, безадресные команды, команды ветвления и команды ввода-вывода.

Адресная команда подразумевает взаимодействие процессора с памятью БЭВМ. Адресные команды имеют следующий вид:

... с прямой абсолютной адресацией															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
КОП				0	Адрес										

... с относительной адресацией															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
КОП				1	Режим			Смещение							

... с непосредственной загрузкой операнда															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
КОП				1	1	1	1	Число							

Безадресные команды подразумевают взаимодействие только с регистрами внутри процессора без обращения к ОЗУ. Например, инкрементирование аккумулятора, расширение знака аккумулятора и так далее. Команды ветвления нужны для организации условных операторов внутри БЭВМ. Команды ввода-вывода позволяют работать с внешними устройствами:

Безадресная команда

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	Расширение КОП											

Команда ввода-вывода

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	Приказ				Устройство							

Команда ветвления

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	Расш. КОП				Смещение							

Машинные циклы

Любая команда состоит из микрокоманд, которые, в частности, открывают определённые вентили для выполнения определённых операций, например, чтение и запись определённого регистра. Все микрокоманды хранятся в МПУ, набор из которых могут выполнять определённые машинные циклы.

Можно выделить следующие машинные циклы



Организация вычислений в БЭВМ. Сдвиги, арифметические и логические операции. Цикл выборки команды.

На нативном уровне АЛУ в БЭВМ поддерживает всего несколько операций: арифметическое сложение (команда ADD, код 4XXX), логическое побитовое “И” (команда AND, код 2XXX) и инверсия (команда NOT, код 0280).

Вычитание SUB (код 6XXX) реализуется через сложение первого операнда и второго операнда, записанного в дополнительном коде со знаком. INC (код 0700) и DEC (код 0740) позволяют инкрементировать и декрементировать содержимое аккумулятора. Остальные команды уже являются производными: команда NEG (0780) позволяет поменять знак у числа, логическое “ИЛИ” (код 3XXX) производит побитовое “или” между двумя операндами.

Операции умножения и деления на железном уровне не реализованы, поэтому их можно получить только программным путём. Однако можно воспользоваться операциями арифметического сдвига ASL (код 0500) и ASR (код 0580), где биты смещаются на разряд старше (значение умножается на 2), либо биты смещаются на разряд младше (значение делится на 2).

Команды ROL (код 0400) и ROR (код 0480) организуют циклический сдвиг влево или вправо соответственно, образуя некое кольцо, по которому перемещаются биты. Перенос из младшего разряда в старший и наоборот реализовано при помощи Carry флага.

Цикл выборки команды.

- 1) IP -> BR, AR
- 2) BR + 1 -> IP; MEM(AR) -> DR
- 3) DR -> CR
- 4) Определение типа команды, адресации (возможно операнда)
- 5) Выполнение самой команды

Организация массивов данных. Режимы адресации. Цикл выборки адреса и операнда БЭВМ

Массив данных представляет из себя множество элементов, идущие последовательно в ячейках памяти. Идеальная ситуация, когда элемент по длине совпадает со словом ЭВМ.

Но есть ситуации, когда элемент не совпадает по длине со словом. Здесь есть два варианта: либо мы в каждое слово (или в кратное число слов) помещаем по элементу, либо располагаем их по порядку в плотной упаковке.

Первый вариант удобен для визуального анализа массива и для работы с ним, однако из-за него появляется разреженность памяти, да и нужно делать расширение знака, если массив состоит из чисел.

Второй вариант решает вопрос с появившимися пустотами, но здесь нужно наверняка знать длину исходных данных, да и плюсом ко всему с плотной упаковкой работать сложнее

В БЭВМ в каждом слове можно хранить максимум коды 2-х символов, записанных либо в Little-endian, либо в Big-endian (см. 3 билет). Для реализации массива в БЭВМ есть два конкретных пути, но в обоих из них мы задаём отдельную переменную индексирования, которая хранит адрес текущего рассматриваемого элемента:

- 1) Помимо самого массива в самом его конце располагается стоп-символ(Ну или же NUL-terminator), после которого программа, обрабатывающая массив, поймёт, что массив (строк) закончился.
- 2) Помимо самого массива в отдельной ячейке хранится длина массива. При помощи команды LOOP, программа обработает конечное количество символов, располагающихся в массиве. Взаимодействие с массивом прекратится в тот момент, когда значение ячейки длины массива обнулится

Режимы адресации

Код в 16СС	Название	Потактово AF and OF
0	Прямая абсолютная	AF – пропуск OF: DR -> AR MEM(AR)->DR
E	Прямая относительная	AF: SXTB(0...7)->BR BR+IP->DR OF: DR -> AR MEM(AR)->DR
8	Косвенная относительная	AF: SXTB(0...7)->BR BR+IP->AR MEM(AR)-> DR OF: DR -> AR MEM(AR)->DR
A	Косвенная автоинкрементная	AF: SXTB(0...7)->BR BR+IP->AR MEM(AR)-> DR DR+1->DR DR->MEM(AR) DR-1->DR OF: DR -> AR MEM(AR)->DR
B	Косвенная автодекрементная	AF: SXTB(0...7)->BR BR+IP->AR MEM(AR)-> DR

		DR-1->DR DR->MEM(AR) OF: DR -> AR MEM(AR)->DR
C	Косвенная со смещением относительно стек	AF: SXTB(0...7)->BR SP+BR->DR OF: DR -> AR MEM(AR)->DR
F	Прямая загрузка	OF: SXTB(0...7)->BR BR->DR

Управление вычислительным процессом в БЭВМ. Команды ветвлений, цикл исполнения команды LOOP.

Управление вычислительными процессами в БЭВМ организовано при помощи операторов ветвления, циклов и стоп-командой HLT.

Операторы ветвления в БЭВМ реализованы при помощи так называемых команд перехода, которые, основываясь на флагах NZVC, делают переход в памяти машины. Это достигается тем, что сумма смещения из команды и самого IP записывается в сам IP. Обычно команды перехода используют вместе с командой сравнения CMP (код 7XXX), которая выставляет флаги NZVC на основе операции вычитания из аккумулятора значения сравниваемого объекта.

Мнемоника	Описание
BEQ D B ranch if E qual	If Z==0 THEN IP+1+D->IP
BNE D B if Not E qual	If Z==1 THEN IP+1+D->IP
BMI B if M inus	If N==1 THEN IP+1+D->IP
BPL B if P lus	If N==0
BHIS B if H igher or S ame	If C==1
BLO B if L ower	If C==0
BVS B if o Verflow is S et	If V==1
BVC B if o Verflow is C lear	If V==0
BLT B if L ess T han	If N XOR V = 1
BGE B if G reater or E quals	If N XOR V = 0

Циклы в БЭВМ реализованы при помощи комбинации команд: LOOP(8XXX) и JUMP (CXXX). LOOP обращается к ячейке, где хранится число итераций, декрементирует его и, если значение больше 0, то выполняется следующая команда, иначе делается переход через команду. Обычно после LOOP ставят команду безусловного перехода JUMP, чтобы вернуться вновь в ту часть кода, которую мы хотим зациклить.

Цикл исполнения LOOP:

DR содержит значение операнда после OF.

- 1) DR-1->DR
- 2) DR-1->BR , DR->MEM(AR); записываем значение по адресу, проверяя стало ли оно ноль доп вычитанием
- 3) If BR(15)=0 then goto int: если BR положительный, то завершаем цикл
- 4) IP+1->IP; если BR отрицательный, то перепрыгиваем через IP
- 5) GOTO INT

Подпрограммы в БЭВМ. Цикл исполнения команд перехода и возврата из подпрограммы. Стек, передача параметров. Позиционно-независимый код. Загрузчик и библиотеки.

При написании кода часто возникает ситуация, когда отдельные части программы выполняются практически одну и ту же функцию. В целях рефакторинга эти фрагменты кода выделяются в отдельную область – подпрограмму. Обратный процесс вклинивания подпрограммы в саму программу называется инлайнинг, чем, кстати, занимаются многие современные компиляторы.

CALL

В БЭВМ подпрограммы реализованы при помощи команды CALL(DXXX), которая записывает адрес возврата на вершину стека и делает переход на тот адрес, который мы указали в коде команды

RET

В конце каждой подпрограммы в БЭВМ располагается команда RET(0A00), которая осуществляет возврат из подпрограммы. По реализации напоминает команду CALL, но только в обратном порядке

Стек – структура данных типа LIFO (Last In, First Out). Стек полезен для работы с временными данными, обработки подпрограмм и прерываний. В полноценном виде стека в БЭВМ так такого нет, есть лишь SP – регистр, который указывает на вершину стека. Сам квази-стек располагается в конце пространства памяти и растёт от конца к началу ОЗУ.

При работе со стеком подразумеваются две операции: записать данные на вершину стека командой PUSH (код 0C00) и считать данные с вершины стека командой POP (код 0800).

Первая команда сначала декрементирует SP и по полученному адресу записывает данные. Вторая команда сначала читает данные по адресу из SP, потом увеличивает счётчик на единицу.

Position Independent Code

Это такой код, который остаётся работоспособным при перемещении внутри памяти. Такой код можно получить, если адресация к входным данным (то есть к тем ячейкам, которые лежат вне программы) является прямой абсолютной, при этом адресация внутри самой программы относительна.

Обычно используются в драйверах и модулей ядра, чтобы ОС смогла самостоятельно их располагать внутри ОЗУ.

Загрузчик и динамический линковщик программ. Библиотеки

Множество подпрограмм обычно формируют библиотеки. При обращении к какой-либо библиотеке за нужной подпрограммой, ОС использует загрузчик для того, чтобы загрузить нужную библиотеку и линковщик, чтобы установить адреса переходов внутри программы.

Функции динамического линковщика:

- Загрузка по выбранному ОС адресу.
- Изменение константных частей адресов в программе.
- Загрузка базовых значений регистров.
- Динамическая загрузка разделяемых библиотек.
- Связывание адресов основной программы с вызываемыми библиотеками.

Библиотеки можно разделить на два типа: динамически линкуемые, которые подтягиваются по требованию в процессе исполнения кода, и статические линкуемые, которые одновременно линкуются с программой в явном виде на этапе компиляции.

Организация ввода-вывода в вычислительных системах. Инициация обмена, передача информации и завершение обмена. Драйверы.

В вычислительных системах подключение ВУ осуществляется при помощи специальных шин ВУ, к которым уже подключаются контроллеры ВУ.

Сами по себе ВУ гораздо медленнее процессора, поэтому, если мы подключимся напрямую, процессор будет простаивать в ожидании готовности ВУ. Как раз по этой причине мы и используем контроллеры ВУ.

Шина ВУ имеет следующие сегменты:

1. Шина данных. По ней поступают данные для обмена.
2. Шина адреса. По ней передаются адреса регистров контроллера, к которым мы будем обращаться.
3. Шина управления. По ней передаются управляющие сигналы или приказы, как обрабатывать ту или иную информацию внутри контроллера.

Обычно шину данных и шину управления не используют в одно и то же время, поэтому их можно мультиплексировать, то есть объединить в единую шину, где в один момент времени передаются управляющие сигналы, в другой – данные.

Драйверы

Драйвер – это специальная программа в памяти ЭВМ, которая организует взаимодействие ВУ, процессором ЭВМ и ОЗУ соответственно.

Драйвер знает режимы работы ВУ, его регистры. Эта программа содержит алгоритм обращения к ВУ, обработки данных со стороны ВУ и со стороны процессора к ВУ. Все драйверы управляются единым программным интерфейсом, который задаётся ОС. Драйверы реализуют три стадии обмена:

1) Инициализация обмена

Этап, который характеризуется началом обмена информацией и выводом приказа на управляющую шину для ВУ. Есть три вида инициации обмена:

1. Синхронный – драйвер отдаёт приказ на шину ВУ в определённый момент времени.
2. Асинхронный – драйвер отдаёт приказ ВУ только в случае, если само ВУ явно дало понять о своей готовности. Готовность проверяется процессором (как было в 5 лабе).
3. Управляемый прерываниями – ВУ сигнализирует процессору о своей готовности. В отличие от асинхронного обмена, процессор сразу же приступает к обработке прерывания, то есть к обмену данных между ЭВМ и ВУ

2) Передача данных

На данном этапе данные поступают с регистров ВУ в регистры процессора через шину данных. Есть два вида передачи данных:

1. Асинхронный – процессор должен подтверждать получение каждого сегмента данных.
2. Синхронный – процессор получает все данные в определённый момент времени без каких-либо подтверждений.

3) Завершение обмена

На этом этапе на процессор приходит сигнал о том, что передача данных завершена. Есть два вида завершения обмена.

1. Синхронный – оповещение о завершении обмена происходит сразу же после завершения работы драйвера.
2. Асинхронный – оповещение о завершении обмена происходит в любой момент времени после завершения работы драйвера.

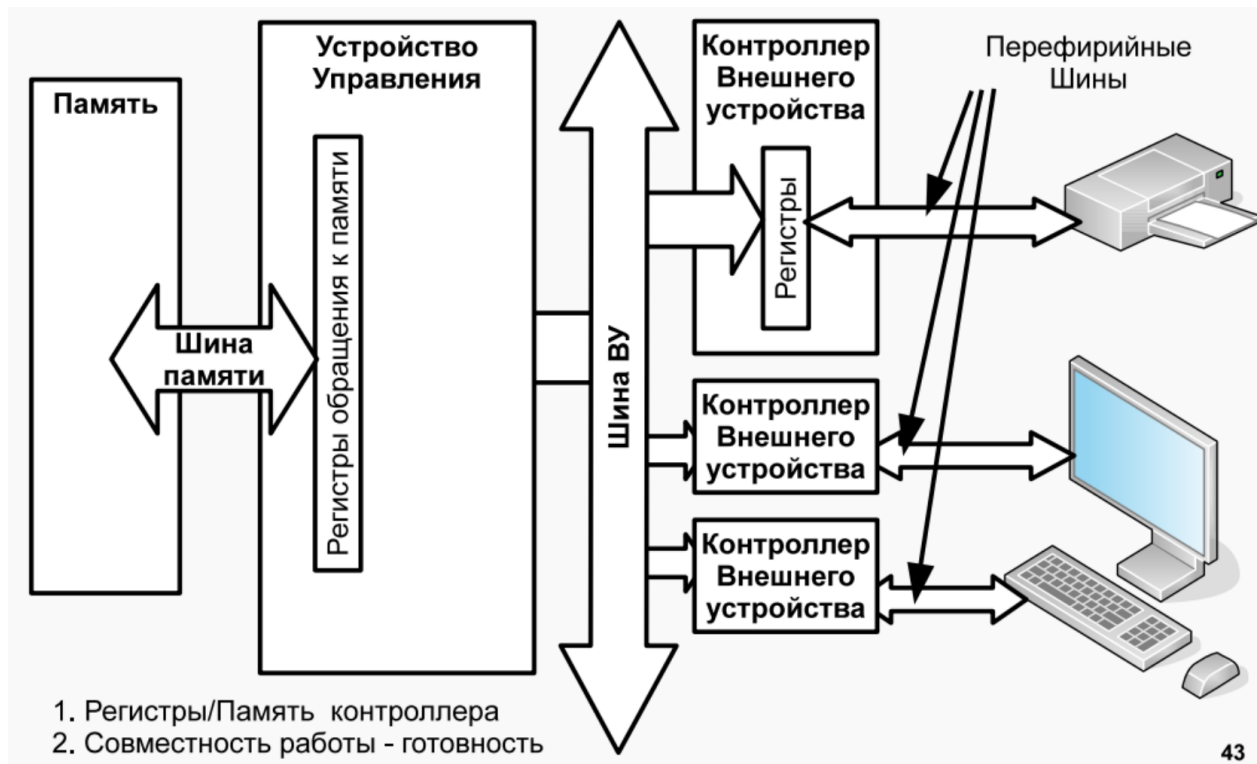
Программно-управляемый ввод-вывод

Организуется программой, которую обрабатывает процессор. С одной стороны, является самой простой в плане реализации обмен, но его можно использовать лишь для обмена небольших кусков данных. Для обмена с ВУ используются драйверы.

Управляемый аппаратурой ввод-вывод

Используется для обмена больших объёмов информации. Драйверы здесь уже не играют такой огромной роли, они нужны лишь для управления режимами работы контроллера. Вся работу выполняют контроллеры при помощи контроллера прямого доступа в память (DMA), который самостоятельно инициирует обмен и транспортирует информацию с ВУ напрямую в память в специально подготовленные ячейки. Процессор в данном вводе-выводе принимает минимальное участие.

Организация ввода-вывода в БЭВМ. Устройства ввода-вывода, команды



К ЭВМ можно подключать огромное количество различных устройств ввода-вывода или ВУ. Эти устройства принимают и передают большой объём данных, которые, по понятным причинам, не могут быть размещены в регистрах ЭВМ. Поэтому информация с ВУ передаётся как в память ЭВМ, так и с неё. При этом обмен может идти под руководством процессора (программно-управляемый обмен), так и под управлением специального внешнего устройства (управляемый аппаратурой обмен), минуя процессор при помощи контроллера прямого доступа к памяти.

Программно-управляемый ввод-вывод

Все действия с вводом-выводом регулируются процессором, то есть все данные, идущие от ВУ к памяти и наоборот, обязательно проходят через процессор. Минусом такого ввода-вывода заключается в том, что процессор пребывает в ожидании готовности ВУ. С другой стороны, реализация такого ввода-вывода довольно простая.

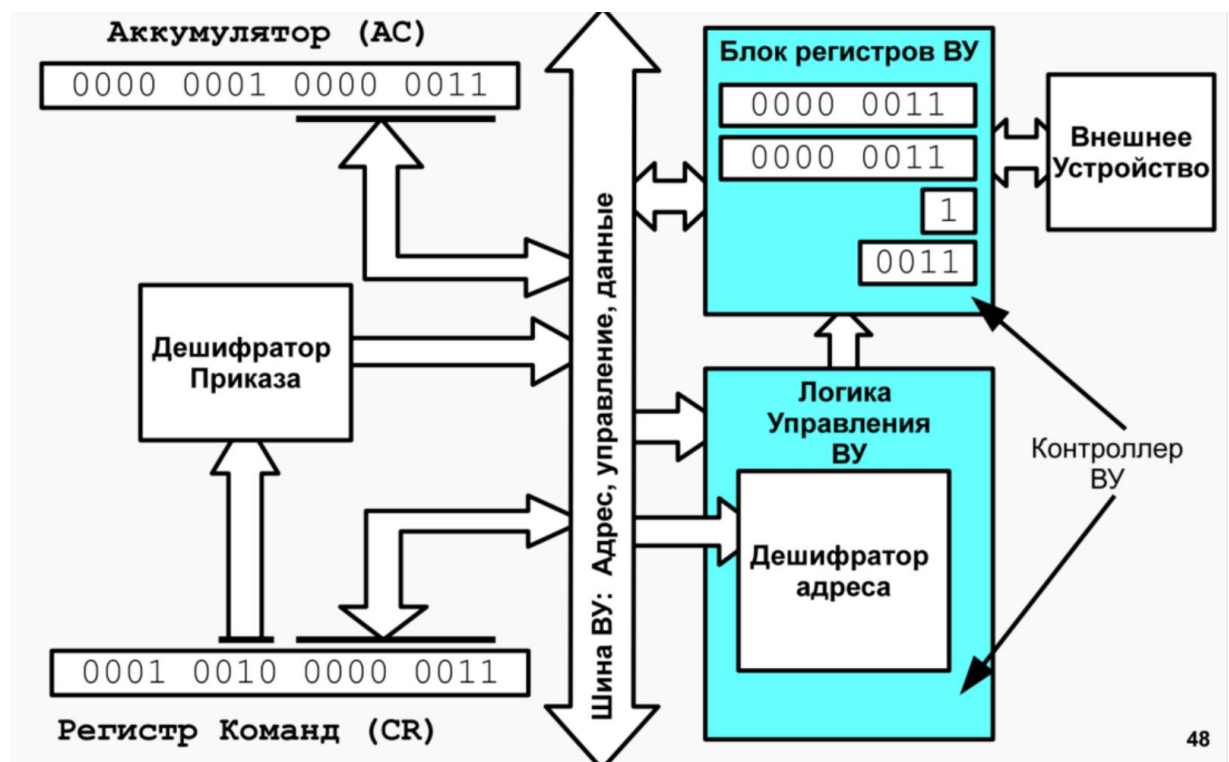
Управляемый прерываниями ввод-вывод

Работа ввода-вывода также завязана на процессоре. Однако во время ожидания готовности ВУ, процессор выполняет какую-то полезную работу. Как только ВУ отправит сигнал о своей готовности, происходит прерывание процессора. Такой подход более производительный, но при этом требует больших усилий на обработку прерываний.

Прямой доступ к памяти (Direct Memory Access)

ВУ работает напрямую с памятью, избегая процессор. Такой способ практически не нагружает процессор, следовательно, сам ввод-вывод будет работать быстрее. Но для

реализации данного подхода нужно проектировать более сложные контроллеры ВУ, способных в прямой доступ к памяти.



48

Регистры контроллеров

SR – однобитовый регистр, в котором хранится бит готовности ВУ к вводу-выводу.

Содержит единицу, если устройство готово, и ноль, если не готово. При извлечении его в аккумулятор, ставится в 6 бит слова. При этом старший байт затрагиваться не будет. То есть IN SR вернёт либо 0000.0000, либо 0100.0000

MR – 4-битовый регистр управления. Содержит в себе номер вектора прерывания и бит на запрет/разрешение прерывания.

DR – 8-битовый регистр данных, через который и происходит обмен данных. При его извлечении занимает лишь младший байт аккумулятора.

Команда ввода-вывода

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	Приказ				Регистр устройства							

Наименование	Мнемон.	Код	Описание
Запрет прерываний	DI	1000	
Разрешение прерываний	EI	1100	
Ввод	IN REG	12XX	REG → мл. байт AC
Вывод	OUT REG	13XX	мл. байт AC → REG
Прерывание	INT NUM	18XX	Програмное прерывание с вектором NUM
Возврат из прерывания	IRET	0B00	(SP)+ → PS, (SP)+ → IP

Внешние устройства БЭВМ

ВУ-0 – таймер. Устанавливает готовность и вызывает прерывание раз в DR*100 миллисекунд

ВУ-1 – устройство вывода

ВУ-2 – устройство ввода

ВУ-3 устройство ввода-вывода

ВУ-4 – устройство ввода-вывода:

По функционалу похож на ВУ-3.

- Адресуется 4-мя регистрами.
- Отдельные регистры для входных (0x8) и выходных данных (0x9) данных.

- Регистр состояния по адресу 0xA.
- Регистр управления по адресу 0xB.
- Все регистры доступны для чтения/записи.
- Позволяет реализовать сложные конфигурации подключения.

ВУ-5 (Текстовый принтер). Регистры 0xC – 0xF:

- Печатает символы из РДВУ в заданной кодировке.
- Регулируемая задержка (время печати)

ВУ-6 (Бегущая строка). Регистры 0x10 – 0x13:

- Размер матрицы: 32x8
- Сдвиг при записи нового значения в РДВУ

- Новое значение – справа
- Младший бит – нижний

ВУ-7 – 8-разрядный 7-сегментный индикатор

7 6 5 4 3 2 1 0

0

Поз

Симв .

• Формат РДВУ:

• Симв==(A₁₆) — установка в разряде знака «-»

• Симв==(B₁₆-F₁₆) — сброс разряда

Вкл

-

1

2

3

4

5

6

7

ВУ-8 (Клавиатура). Регистры 0x18-0x1C:

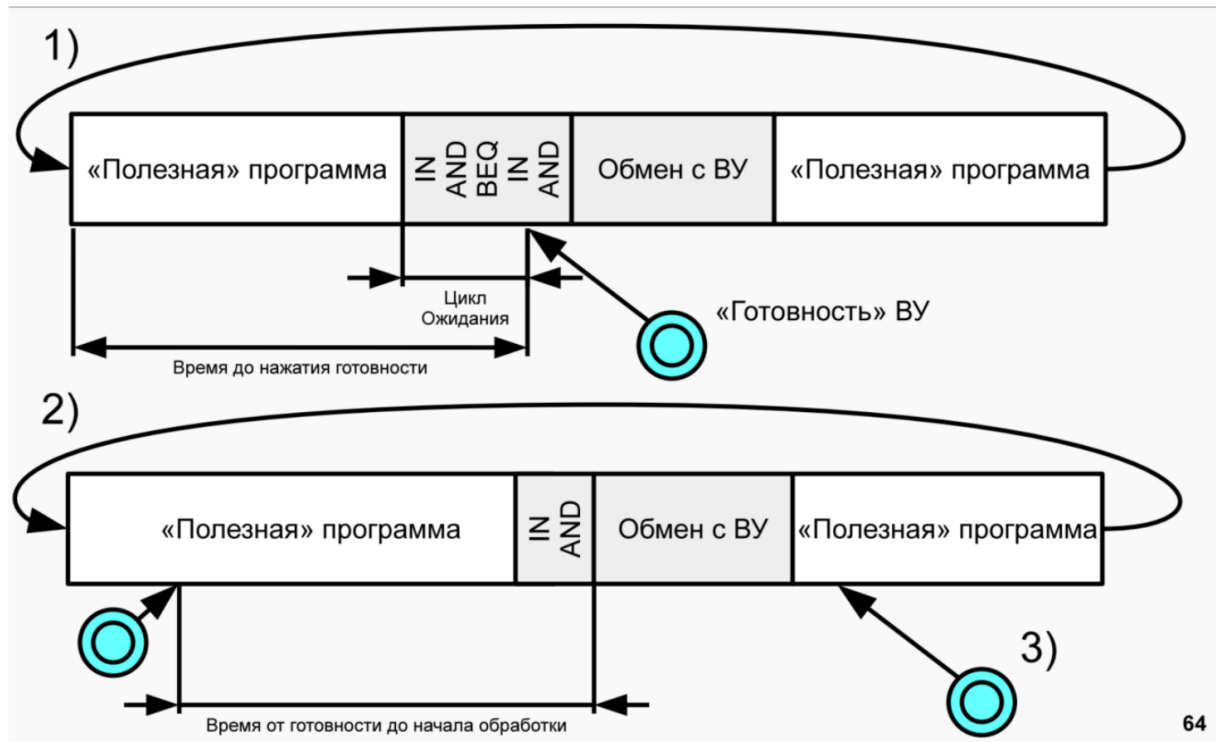
- Код нажатой клавиши в выбранной кодировке устанавливается в РДВУ
- Автоматически устанавливается готовность.

ВУ-9 (Цифровая клавиатура). Регистры 0x1C-0x1F. Клавиатура, но с цифрами

Организация асинхронного обмена в БЭВМ. Пример программы. Временные издержки асинхронного обмена.

Кратко: асинхронный обмен подразумевает ситуацию, когда БЭВМ запрашивает готовность у КВУ до того момента, пока КВУ не даст сигнал о своей готовности

Временные издержки асинхронного обмена.



64

Первый вариант подразумевает наличие спин-лупа внутри всей программы. То есть программа входит в часть кода, где происходит проверка готовности. Программа не пойдёт дальше до тех пор, пока ВУ не даст сигнал о своей готовности.

Второй вариант подразумевает отсутствие спин-лупа. То есть если на моменте проверки готовности ВУ не готово, то программа пропускает фрагмент кода с обменом ВУ и продолжает выполнять полезную программу. Из плюсов такого подхода можно отметить полезность работы самого процессора. С другой стороны, невозможно быстро начать стадию обмена с ВУ (всё зависит от того, когда была нажата кнопка готовности).

Организация прерываний в БЭВМ. Вектора прерываний, контроллер прерывания.

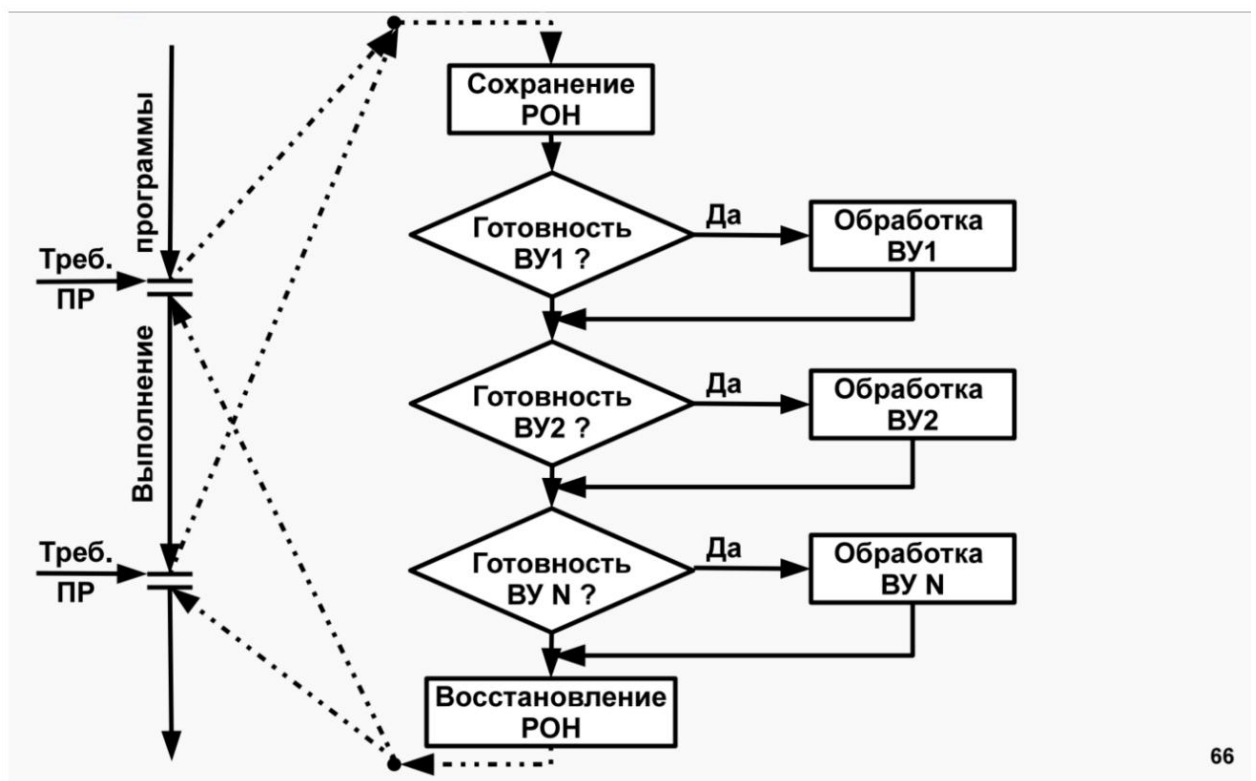
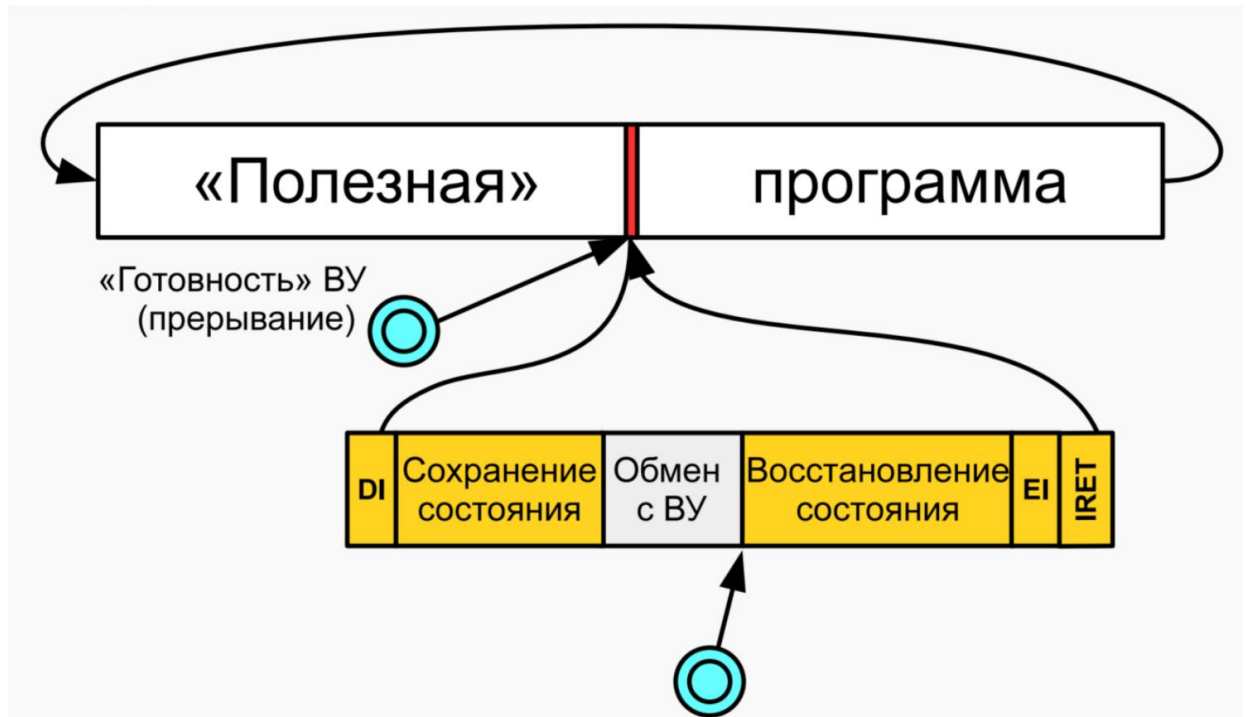
Прерывание – сигнал от программного или аппаратного обеспечения, сообщающий процессору о наступлении какого-либо события, требующего немедленной обработки. В рамках БЭВМ это сигнал о готовности ВУ к обмену данных.

Вектор прерывания – в контексте БЭВМ это совокупность адреса обработчика прерывания (то есть какой-то подпрограммы) и регистра состояния PS, с которым он стартует.

Перед началом обработки прерывания необходимо их инициализировать, то есть хотя бы установить на ничего не делающую подпрограмму. В БЭВМ доступно 8 векторов прерываний, расположенных по адресу с 0x0 по 0xF включительно. На один вектор прерывания может приходиться несколько прерываний.

Если говорить об ЭВМ в общем, то количество ячеек в векторах прерывания варьируется, да и за инициализацию векторов прерывания отвечает ОС и БИОС.

Инициация обмена по прерыванию:



Команды для работы с прерываниями

- IRET (0B00) – возврат из прерывания:

$SP \rightarrow AR$ $MEM(AR) \rightarrow DR$ $DR \rightarrow PS$ } Восстанавливает старый PS

$SP + 1 \rightarrow AR$, $AR \rightarrow MEM(AR)$ $MEM(AR) \rightarrow DR$ $DR \rightarrow IP$ } Восстанавливает старый IP

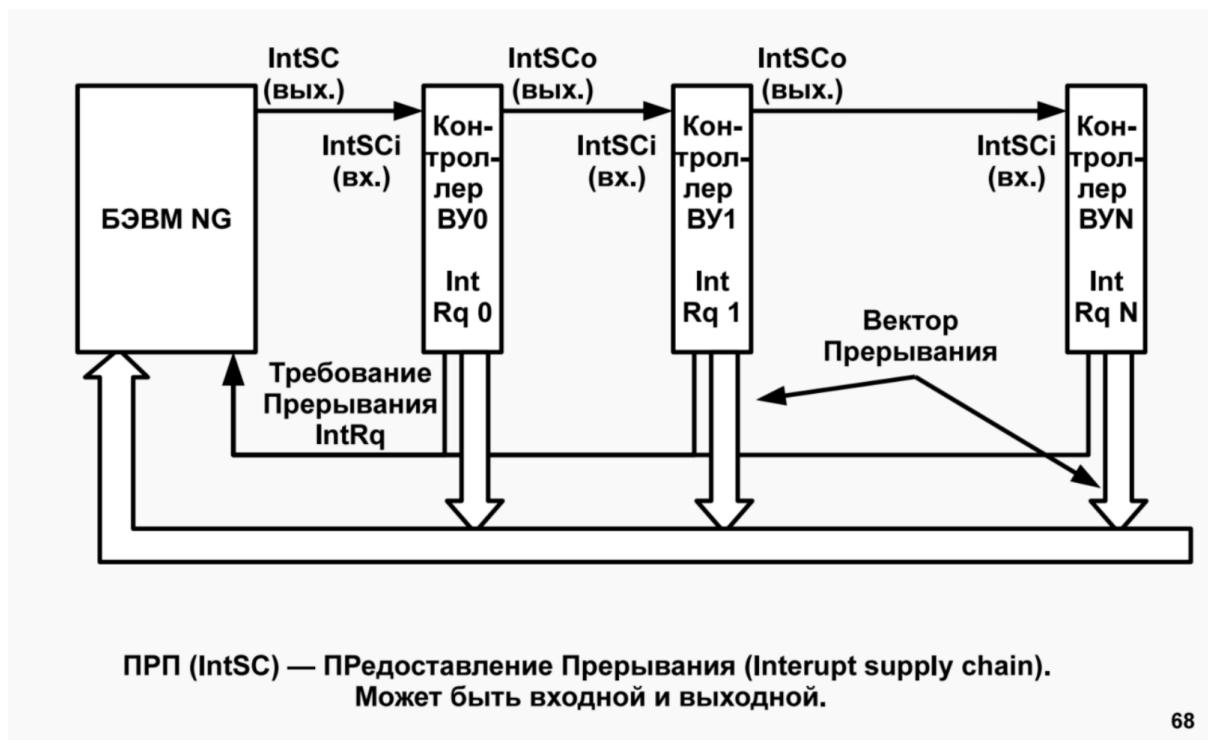
$SP + 1 \rightarrow SP$

Дальше идёт цикл обработки прерывания.

- EI (1100) – разрешение прерывания. Устанавливает в 5 бит регистра PS единицу.
- DI (1000) – запрет прерывания. Устанавливает в 5 бит регистра PS ноль.

Важно, что прерывания не обрабатываются в пошаговом режиме, так как в самом начале цикла прерывания БЭВМ проверяют наличия 7 бита W в регистре состояния, который отвечает за “РАБОТА/ОСТАНОВ” режим.

Сигнал предоставления прерывания проходят через все КВУ, пока не дойдёт до того, где был запрос на прерывание. Как КВУ понимает, что именно его ВУ вызвало прерывание – спрятано в логике управления данного КВУ. Приоритет получения КВУ сигнала предоставления прерывания организуется аппаратно. Цикл прерывания идёт после всех команд, кроме HLT, DI и EI.



Организация обмена по прерыванию программы в БЭВМ. Пример программы. Цикл прерывания.

Подобный обмен используется при работе с низкоскоростными ВУ или когда момент передачи данных между ЭВМ и ВУ неизвестен. Обмен данным между ЭВМ и ВУ инициируется сигналом готовности с ВУ. Для реализации такого типа обмена используется аппаратная проверка наличия внешнего прерывания.

По картинке выше из прошлом билета видно, что при поступлении сигнала IntRQ в регистр PS, БЭВМ проходит по всем КВУ (если конечно прерывания разрешены, тут нужно смотреть в PS на 5 бит) в поисках того самого, что сделал запрос на прерывание.

Как только БЭВМ нашла КВУ, его вектор прерывания помещается в CR. Далее происходит цикл обработки прерывания, который помещает IP и PS в стек. Сохранение AC нужно организовывать самостоятельно. То есть минимальная информация о прерванной программе хранится на вершине стека в виде 3-х слов.

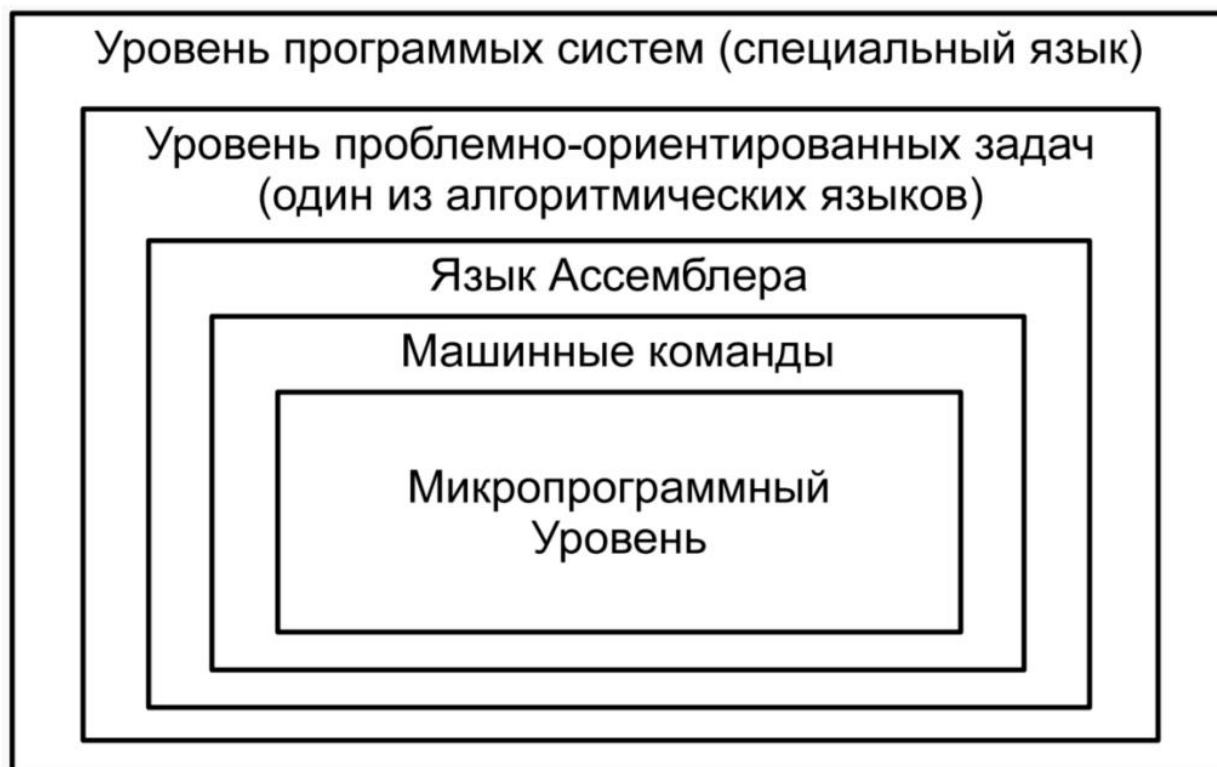
Цикл прерывания

- 1) IF PS(6) == 0 THEN GOTO INFETCH; если 6 бит состояния (запрос прерывания) = 0, то переход к след команде
- 2) Иначе БЭВМ формирует сигнал предоставления прерывания через выполнение мк INTS
- 3) Сохраняется IP и SP на вершину стека
- 4) LTOL(CR) -> BR; младшие 8 разрядов, номер вектора прерывания, записываются в BR
- 5) Вычисляем адрес с переходом на подпрограмму обработки, как номер вектора * 2, после чего пишем в DR, а потом в IP
- 6) Далее к мл разрядам BR прибавляем 1, тем самым, вычисляем адрес след ячейки вектора прерывания нового регистра состояния. Затем значения ячейки памяти по этому адресу пишем в DR -> PS.

Пример программы обработчика

Готовность ВУ1: 2*А-РДВУ1, Готовность ВУ3: РДВУ3- яч.3F	
V0:	ORG 0x0 ; Инициализация векторов прерывания
V1:	WORD \$DEFAULT,0x180 ; Вектор прерывания #0
V2:	WORD \$INT1,0x180 ; Вектор прерывания #1
V3:	WORD \$DEFAULT,0x180 ; Вектор прерывания #2
V7:	WORD \$INT3,0x180 ; Вектор прерывания #3
...	
V7:	WORD \$DEFAULT,0x180 ; Вектор прерывания #7
DEFAULT:	IRET ; Просто возврат
START:	ORG 0x020 ; Загрузка начальных векторов прерывания
	DI
	CLA
	OUT 1 ; MR КВУ-0 на вектор 0
	OUT 5 ; MR КВУ-2 на вектор 0
	LD #9 ; разрешить прерывания и вектор №1
	OUT 3 ; (1000 0001=1001) в MR КВУ-1
	LD #0xB ; разрешить прерывания и вектор №3
	OUT 7 ; (1000 0011=1011) в MR КВУ-3
	...
	JUMP \$PROG
PROG:	EI ; Установка состояния разр. прерывания
	CLA ; Первоначальная очистка аккумулятора
INCLP:	INC ; Цикл для наращивания
	BR INCLP ; содержимого аккумулятора
	ORG 0x03F
	; Ячейка для хранения кодов, поступающих с ВУ-3
IO3:	WORD ?
INT1:	ORG 0x040
	; Прерывание сохранило содержимое PS
	NOP ; отладочная точка останова (NOP/HLT)
	PUSH ; Сохранили AC
	ASL ; Умножили AC на 2
	OUT 2 ; Записали в РДВУ-1 (DR#2)
	POP ; Вернули AC назад
	NOP ; отладочная точка останова (NOP/HLT)
	IRET ; Возврат из обработки прерывания
INT3:	ORG 0x050
	; Прерывание сохранило содержимое PS
	NOP ; отладочная точка останова (NOP/HLT)
	PUSH ; AC кладем в стек
	CLA
	IN 6 ; РДВУ-3
	ST \$IO3 ; сохранение
	NOP ; отладочная точка останова (NOP/HLT)
	POP ; AC вынимаем из стека
	IRET ; Возврат из обработки прерывания

Понятие многоуровневой ЭВМ. Понятие и пример программы на разных уровнях.



Возможность исполнения на ЭВМ программы, написанной на алгоритмическом языке, обеспечивается с помощью специальных системных программ: компиляторов и интерпретаторов.

Компиляция заключается в том, что процесс выполнения алгоритма осуществляется лишь после завершения процесса перевода исходной программы в машинный код.

В интерпретации же каждый оператор исходной программы заменяется программой-интерпретатором на эквивалентную последовательность машинных команд непосредственно перед исполнением программы. В отличие от компиляции, в интерпретации во время решения задачи машине нужны не только исходная программа, но и сама программа-интерпретатор.

Существует определённая иерархия языков, где более высокоуровневый язык базируется на более низкоуровневом.

Многоуровневая ЭВМ – это вычислительная машина, имеющая средства для работы с n различными уровнями языков программирования. Нижний язык, как правило, является самым простым в реализации, но наиболее сложным в написании кода. Верхний язык наоборот: самым сложным в плане аппаратурной реализации, но самым простым в написании.

Такую машину можно рассматривать как n различных виртуальных машин, каждая из которых имеет свой машинный язык.

1. Уровень системных программ

В качестве примера можно привести в пример сложение в Excel как прикладную программу. Для этого в ячейку C1 запишем следующую формулу

“=A1 + B1” и получим свой результат:

2. Уровень проблемно-ориентированных задач.

Java – один из алгоритмических языков. Программа сложения чисел на этом языке выглядит следующим образом:

```
public class Test {  
    public static void main(String[] args) {  
        int A = 10;  
        int B = 20;  
        System.out.println(A + B);  
    }  
}
```

3. Язык ассемблера

```
ORG 0x20  
X: WORD 10  
Y: WORD 20  
Z: WORD ?  
START:  
    LD X  
    ADD Y  
    ST Z  
    HLT
```

4. Машинные коды

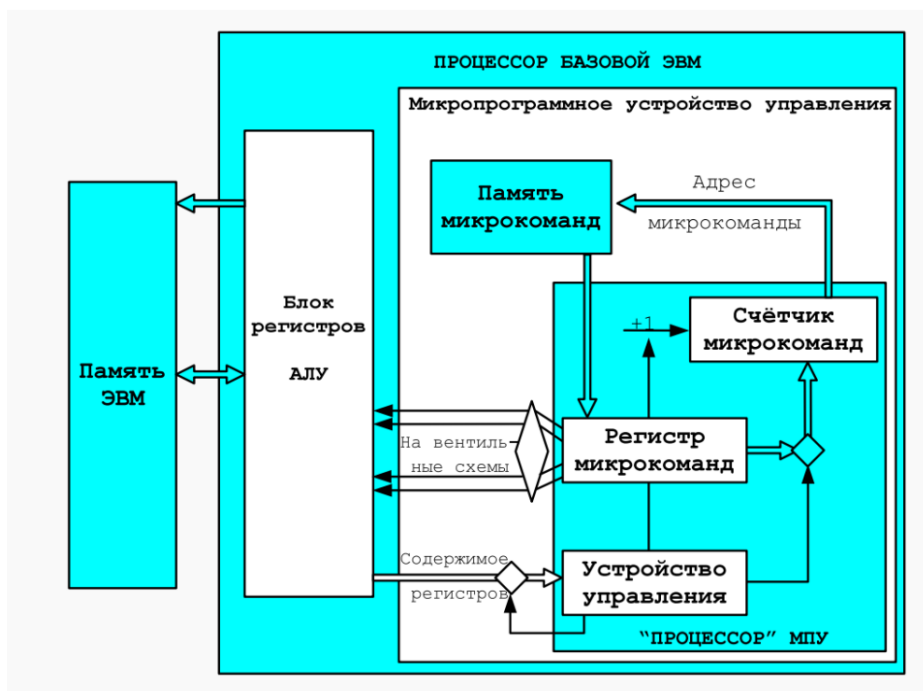
```
020 000A (X)  
021 000B (Y)  
022 0000 (Z)  
023 A020 (LD X)  
024 4021 (ADD Y)  
025 E022 (ST Z)  
026 HLT
```

5. Микропрограммный уровень

Тут рассмотрим только одну микропрограмму, так как писать много придётся

```
32 0010E09011 ADD AC + DR → AC, N, Z, V, C
```

Микропрограммный уровень БЭВМ. Структура МПУ.
Форматы микрокоманд.

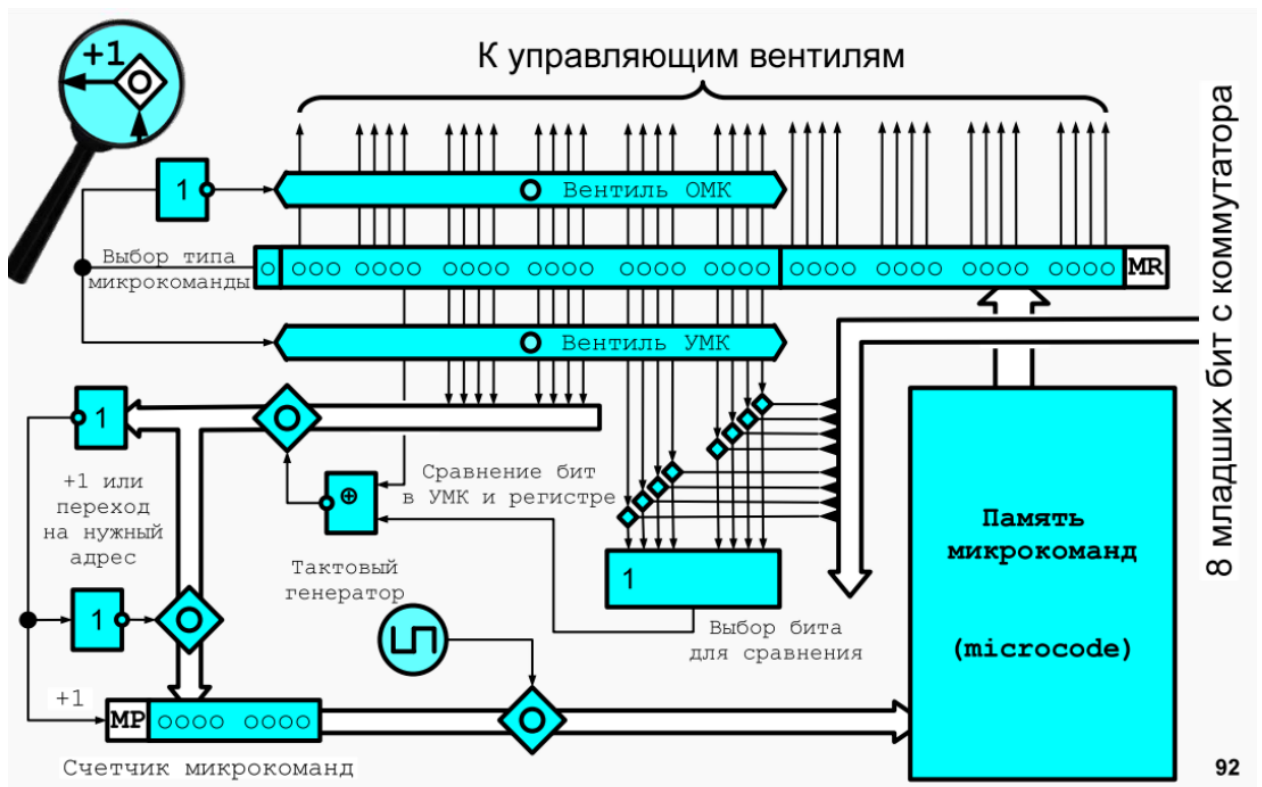


Если рассматривать БЭВМ как многоуровневую вычислительную машину, то на неё самом нижнем уровне выполняются элементарные действия по отношению словам с информацией. Такие действия называют микрооперациями.

Порядок следования микроопераций задаётся устройством управления БЭВМ, которая представляет из себя простейшую ЭВМ. Для такой ЭВМ регистры и вентильные схемы БЭВМ являются как бы внешними устройствами, с которыми производится ввод-вывод.

Программа такой ЭВМ (быть точнее МПУ) называется микропрограммой, а её команды, содержащие информацию об элементарных действиях и выполняющиеся в течение одного такта ТГ, называют микрокомандами. Микропрограмма хранится в ПЗУ – в памяти микрокоманд.

В каждом такте работы в регистр микрокоманд (РМК) пересылается очередная микрокоманда, на которую указывает счётчик микрокоманд (СЧМК), выполняющий одновременно функцию регистра счётчика микрокоманд



Внутри МПУ помимо вышеописанных памяти микрокоманд, СчМК и РМК есть устройство управление. Конкретно оно отвечает за логику обработки микрокоманды.

При поступлении микрокоманды в РМК первые 16 бит сразу отправляются на вентили. Дальше выбор вентилях зависит от типа команды.

В БЭВМ существует два вида микрокоманд:

- Операционная микрокоманда (39 бит равен нулю) – все закодированные биты поступают на вентили, то есть происходит какая-то элементарная операция, будто пересылка данных между регистрами, арифметическая или логическая операция и так далее
- Управляющая микрокоманда (39 бит равен единице) – используется как команда ветвления внутри интерпретатора БЭВМ. Проверяет определённый бит с каким-то эталонным значением и, в случае совпадения, совершает переход по адресу, закодированной в команде.

Логика изменения значения в СчМК проста. Если это операционная команда, то вентиль со стороны схемы сравнения не откроется, ибо сравнения так такого даже не было. Значит на шину к СчМК подаётся ноль, который в силу реализации инкрементируем СчМК.

Если же микрокоманда является управляющей, то всё зависит от успеха сравнения битов. Если биты совпали, то схема сравнения, основанная на NOTXOR, откроет вентиль и адрес перехода запишется в СчМК.

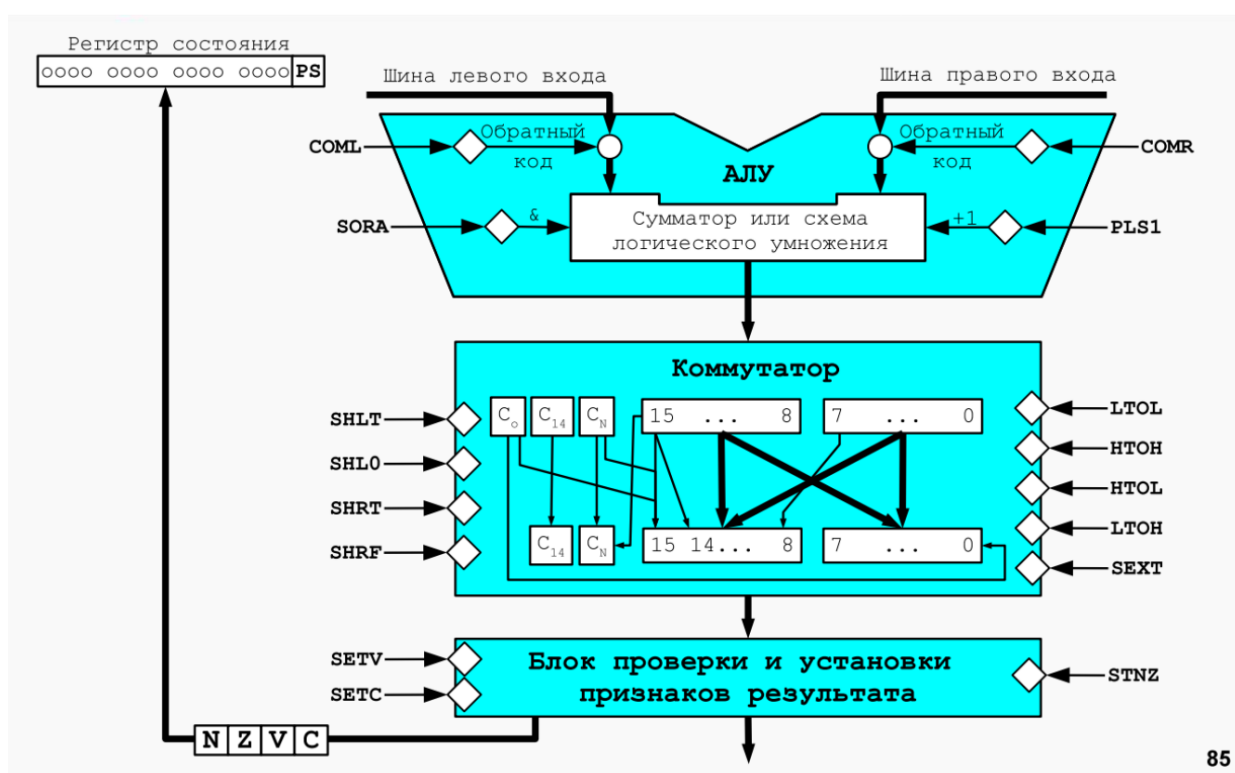
Горизонтальные и вертикальные микрокоманды.

Внутри БЭВМ используются горизонтальные микрокоманды, где каждый бит отвечает за определённый вентиль или хранит служебный бит.

Однако в больших ЭВМ, где количество вентиляй сильно больше, горизонтальные микрокоманды становятся слишком длинными. Поэтому в таких ситуациях используют кодируемые поля, которые в совокупности представляют из себя вертикальные микрокоманды.

Сократить количество битов можно за счёт кодирования вентиляй, которые не могут одновременно открыться или же кодирование полей регистра проверяемого слова. Перевод из горизонтальной в вертикальную микрокоманду производится при помощи дешифратора. Поэтому полезность вертикальной микрокоманды зависит от выгоды установки дешифратора перед количеством памяти, занимаемом горизонтальными микрокомандами.

Структура и принципы работы арифметико-логического устройства и коммутатора. Регистр состояния БЭВМ



85

Схема обратного кода реализована таким образом, то либо открывается вентиль прямого кода, либо же обратного. Реализуется это при помощи подключения к цепи управляющего сигнала инвертора к одному из вентиляй.

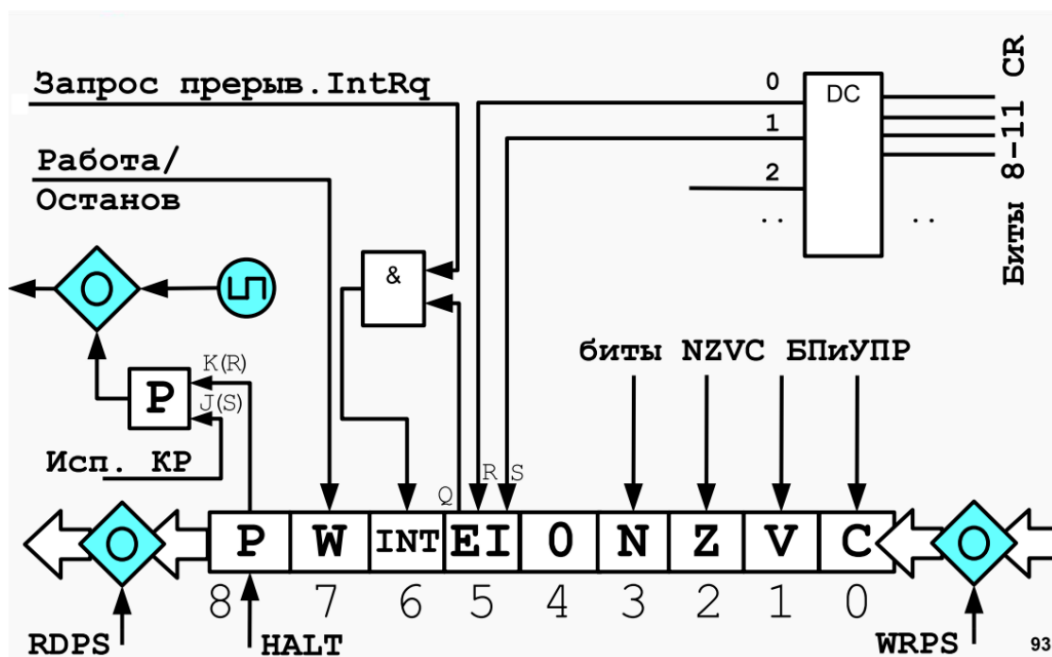
Обратный же код получается при помощи побитового инвертирования битов операнда.

Схема сложения реализована при помощи типичной схемы сумматора. Причём вентиль **PLS1** добавляет единицу как сигнал переноса в сумматор 0-го бита.

Логическое “И” реализовано как логическое “И” между каждым разрядом двух операндов. Бит переноса за разрядную сетку (Позже это будет Керри флагом) реализуется как выход переноса последнего сумматора. Таким же образом сохраняется бит переноса между 14 и 15 разрядом, которые будут нужен для определения Overflow флага. Не забываем ещё про

старый перенос Carry. Итого в коммутатор у нас поступит аж 19 битов. Выбор конкретной операции реализовано аналогичным образом, как и в схеме обратного кода.

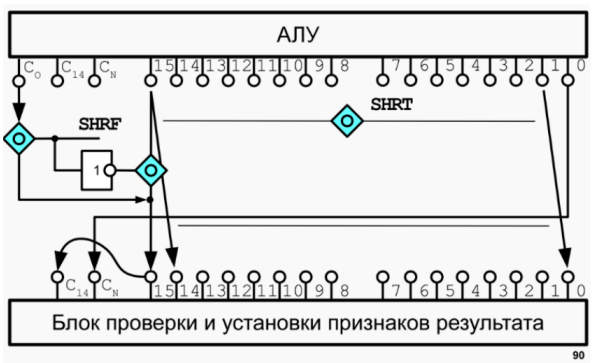
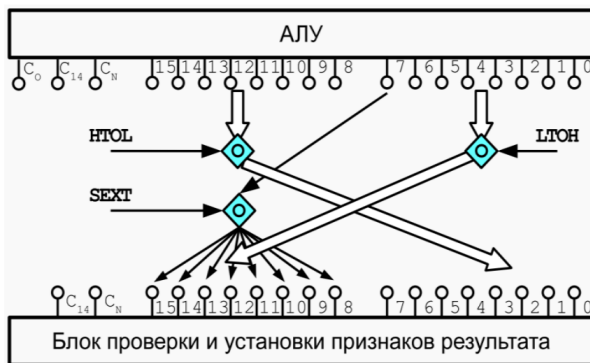
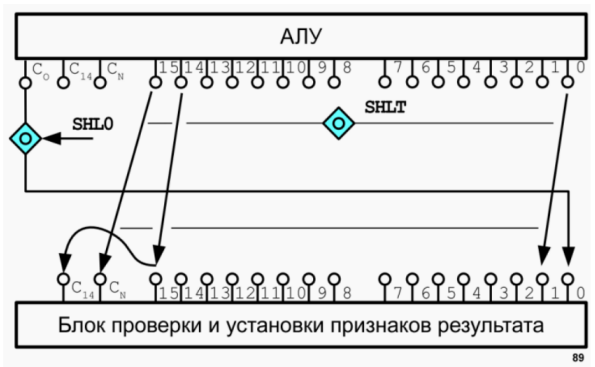
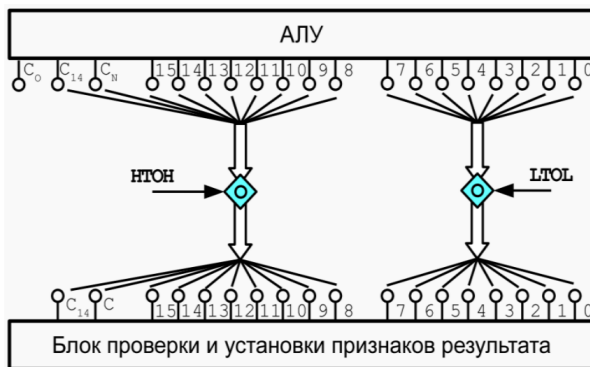
Регистр состояния



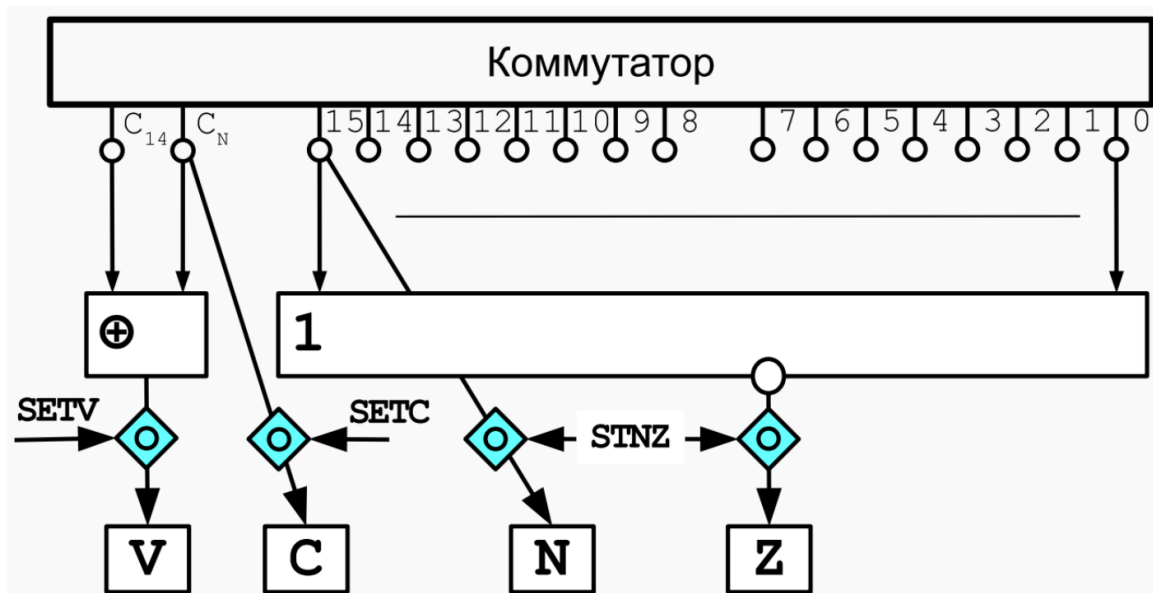
Коммутатор

Коммутатор коммутирует (капитан очевидность, знаю). Он позволяет переставлять биты нужным нам образом: менять старший и младший бит, делать расширение знака, сдвигать как арифметически, так и логически биты влево или вправо.

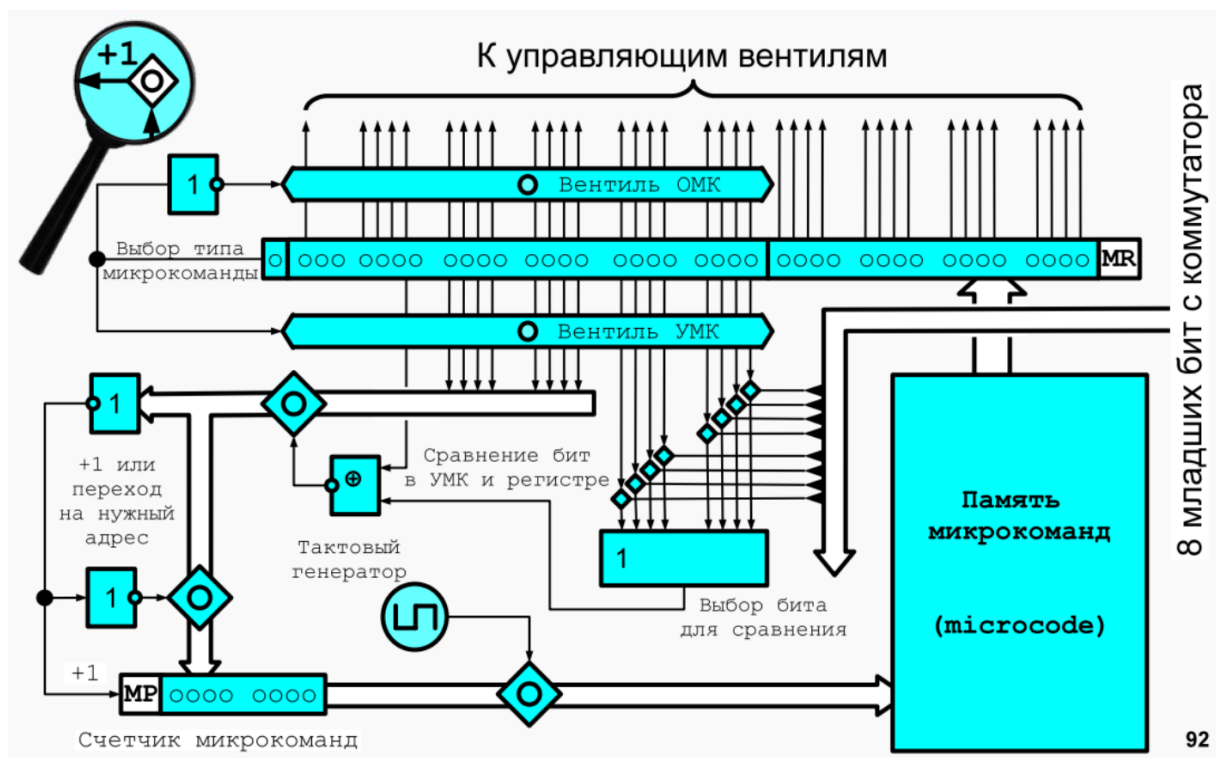
Важно уточнить моменты, связанные с битами переноса. При прямой передаче байтов биты так и остаются неизменными. При расширении знаков или обмена байтами эти биты не передадутся. При смещении битов переносы заполняться таким образом, что если из 14 в 15 мы сделали перенос в сдвиге право, то мы ставим бит переноса C14 и так далее.



Блок проверки и установки признаков результата:



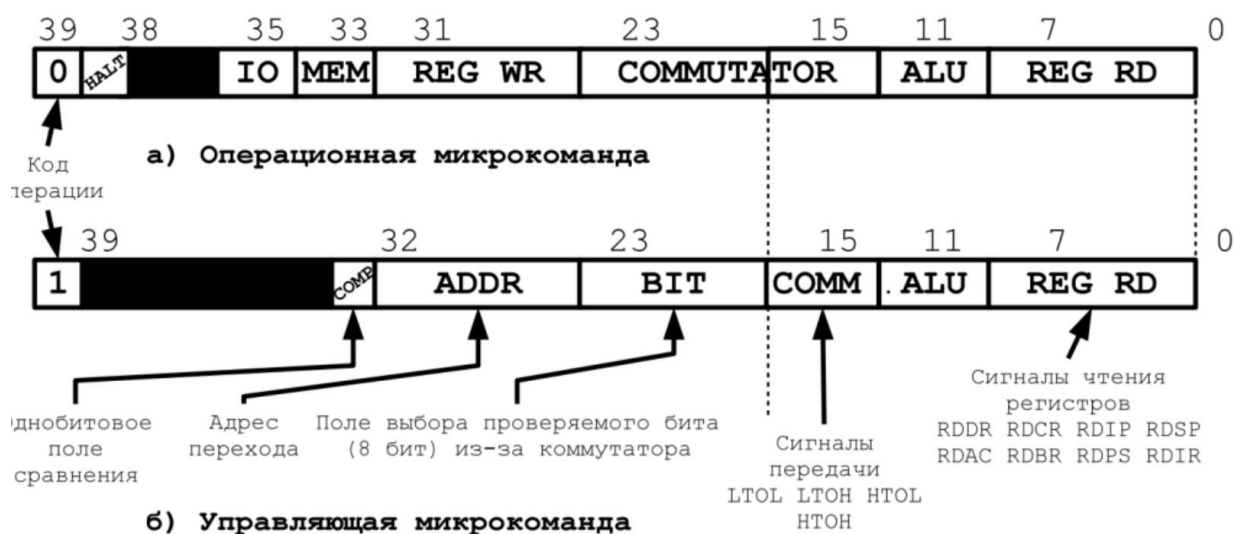
Микропрограммное управление вентиляльными схемами. Схема управления. Интерпретатор БЭВМ.



Чтобы разобраться в том, как работают вентиляльные схемы, нужно рассмотреть работу УУ. Как работает УУ было в общих чертах описано в 20 билете. Опишем принцип работы УУ в трёх случаях:

1. 39 бит равен 0, следовательно, в РМК лежит ОМК. Схема сравнения не сработает, поэтому вентиль адреса перехода не откроется. Из этого следует, что после этого вентиля будет сигнал, эквивалентный нулю. Верхний инвертор сработает и подаст сигнал об инкрементировании СчМК, при этом второй инвертор в этой цепи на выход даст нолик. Благодаря такой схеме СчМК не обнуляется при поступлении, а всего лишь инкрементируется.
2. 39 бит равен 1, что свидетельствует об УМК в РМК. Допустим, что схема сравнения выдала положительный результат. Значит, что вентиль открылся и на шину перед СчМК подался какой-то адрес перехода. Первый инвертор выдаст ноль на выход, второй выдаст уже единицу, которая откроет вентиль. Как следствие, в СчМК запишется адрес перехода.
3. 39 бит равен 1, что свидетельствует об УМК в РМК. Допустим, что схема сравнения отрицательный результат сравнения. Вентиль не откроется, следовательно, дальше логика рассуждения аналогична пункту 1.

Микрооперации, разновидности



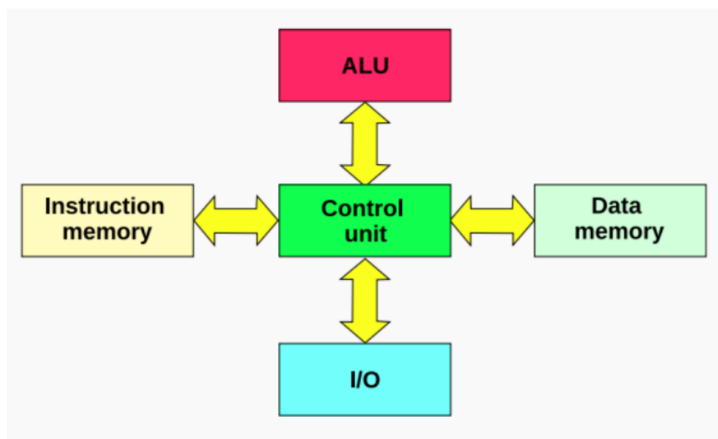
Бит	Мнемоника	Назначение
Чтение регистров		
00	RDDR	DR (РД) → правый вход АЛУ
01	RDCR	CR (РК) → правый вход АЛУ
02	RDIP	IP (СК) → правый вход АЛУ
03	RDSP	SP (УС) → правый вход АЛУ
04	RDAC	AC (А) → левый вход АЛУ
05	RDBR	BR (БР) → левый вход АЛУ
06	RDPS	PS (РС) → левый вход АЛУ
07	RDIR	KR (КР) → левый вход АЛУ
АЛУ		
08	COMR	Обратный код правого входа АЛУ
09	COML	Обратный код левого входа АЛУ
10	PLS1	Операция A + B + 1 (PLuS 1)
11	SORA	Операция И (Sum OR And) (да, мы наркоманы)
Управление коммутатором		
12	LTOL	Прямая передача младшего байта
13	LTON	Передача младшего байта в старший
14	HTOL	Передача старшего байта в младший
15	HTON	Прямая передача старшего байта
16	SEXT	Расширение знака младшего байта (sign extend)
17	SHLT	Сдвиг влево (арифметический) (SHift Left)
18	SHL0	Передача старого значения С в младший бит (для циклического сдвига влево)
19	SHRT	Сдвиг вправо (арифметический) (SHift Right)
20	SHRF	Переключатель сдвига вправо (для циклического сдвига 15 разряд)
21	SETC	Установка С
22	SETV	Установка V
23	STNZ	Установка N и Z
Запись в регистры		
24	WRDR	АЛУ → РД
25	WRCR	АЛУ → РК
26	WRIP	АЛУ → СК
27	WRSP	АЛУ → УС
28	WRAC	АЛУ → А
29	WRBR	АЛУ → БР
30	WRPS	АЛУ → РС
31	WRAR	АЛУ → РА
Работа с памятью		
32	LOAD	ОП → РД
33	STOR	РД → ОП
Организация ввода-вывода		
34	IO	Передача адреса и приказа на ВУ
35	INTS	Предоставление прерывания
36		Зарезервирован
37		Зарезервирован
Останов БЭВМ		
38	HALT	Останов
Тип команды		
39	TYPE	Бит выбора ОМК/УМК

Интерпретатор хранится в ПЗУ МПУ и представляет из себя огромную последовательность микрокоманд, по которым проходит МПУ при выполнении машинной команды БЭВМ. В интерпретаторе есть 256 ячеек для хранения горизонтальных команд. Сам по себе интерпретатор состоит из следующих циклов:

- Цикл выборки команд
- Цикл выборки адреса операнда и обработки режимов адресации
- Цикл выборки операнда
- Цикл исполнения
 - Декодирование и исполнение адресных команд
 - Декодирование и исполнение ветвлений
 - Декодирование и исполнение безадресных команд
- Декодирование и исполнение команд ввода-вывода
- Цикл прерывания
- Пультовые операции
- Свободные ячейки для:
 - Арифметической команды
 - Команды перехода
 - Безадресной команды

Архитектура ЭВМ. Гарвардская и фон-Неймановская архитектура. Организация обмена архитектуры ЭВМ с использованием шин.

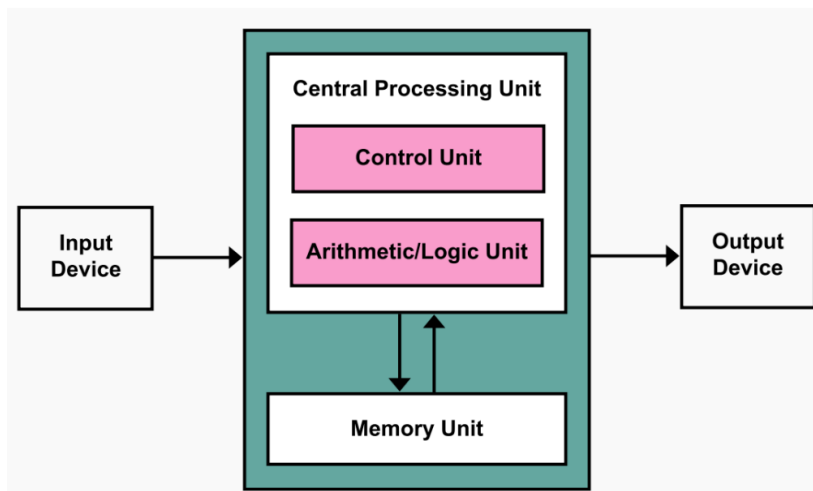
Гарвардская архитектура



Гарвардская архитектура характеризуется физическим разделением памяти команд и памяти данных. Каждая память соединяется отдельной шиной, что позволяет одновременной с чтением-записью данных при выполнении текущей команды производить выборку и декодирование следующей команды, что значительно повышает производительность всей системы, в отличие от архитектуры фон-Неймана.

Недостатком такой архитектуры заключается в необходимости проведения большого количества шин, а также в фиксированном объеме данных, выделенной для команд и данных соответственно. Вследствие этого перераспределение такой памяти происходит крайне неоперативно. Однако с развитием микроэлектронной техники в значительном степени удалось преодолеть указанные недостатки.

Архитектура фон-Неймана



Принстонская архитектура, которая часто называется архитектурой фонНеймана, характеризуется общей оперативной памятью для хранения программ, данных, а также для организации стека. Для обращения к данной памяти используется общая системная шина, по которой передаются команды и данные.

Наличие общей памяти позволяет оперативно перераспределять её объём для хранения отдельных массивов команд, данных и реализации стека в зависимости от решаемых задач. Таким образом, обеспечивается более эффективное использование оперативной памяти в каждом конкретном случае применения МП.

Использование общей шины упрощает отладку, тестирование и текущий контроль функционирования системы, повышает её надёжность. Поэтому данная архитектура очень долго доминировала в вычислительной технике.

Из недостатков можно отметить последовательность выполнения выборки команды и обрабатываемых данных по общей системной шине. При этом общая шина становится узким горлышком, которое ограничивает производительность всей вычислительной машины.

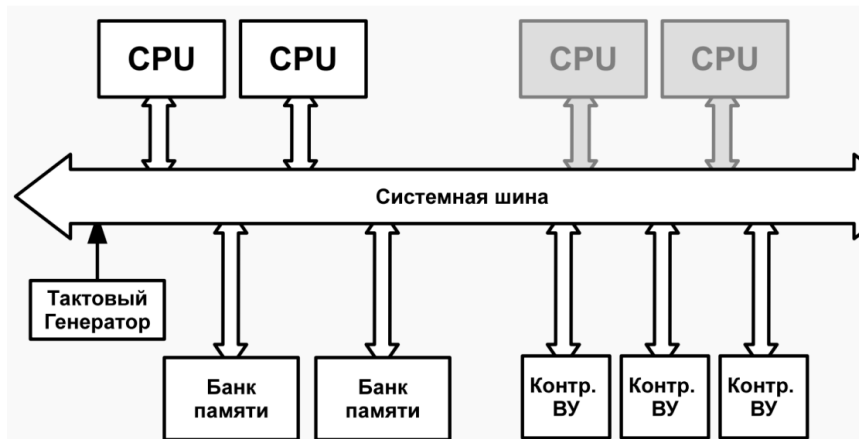
Архитектура многопроцессорных ЭВМ. Системный коммутатор. Архитектуры UMA и NUMA

UMA (Uniform Memory Access) – самая быстрая составляющая – это сам процессор. Где-то в 100 раз медленнее память, где-то в 1.000.000 раз – КВУ.

Стояла задача повысить эффективность работы процессора. Конечно, можно повышать частоту ТГ, но мы упираемся в ограничения из-за скорости света, максимально можно добиться частоты в 5 ГГц.

Тогда решили использовать сразу несколько процессоров внутри вычислительной машины. Но возникает проблема с системной шиной в связи с потерей времени и отсутствием синхронизации.

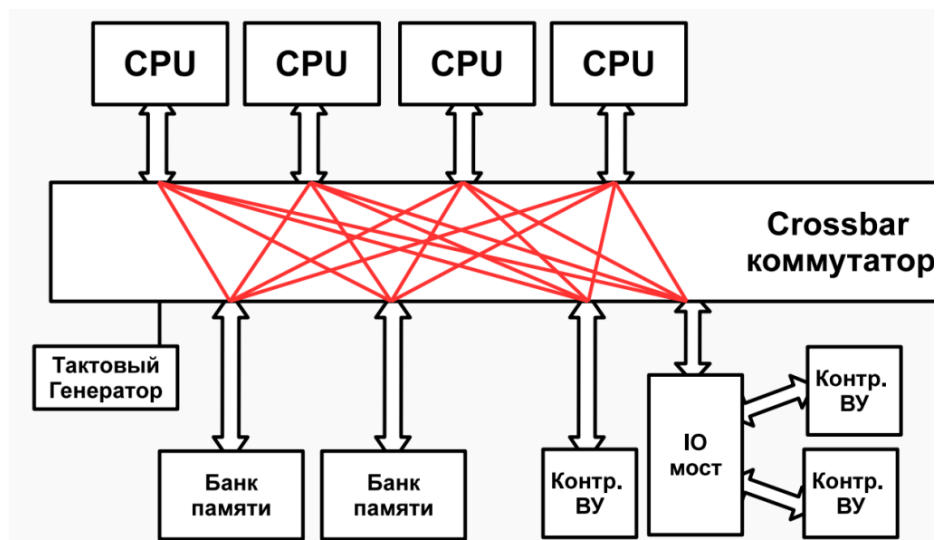
При 2-х процессорах эффективность возрастает, так как пока один процессор выполняет вычисления внутри себя, второй же взаимодействует с КВУ или памятью. При 3-х и более процессорах возникает “узкое горлышко”, коим является системная шина.



UMA – архитектура многопроцессорных компьютеров с общей памятью. Все микропроцессоры в UMA-архитектуре используют физическую память одновременно. При этом время запроса к данным не зависит от того, какой именно процессор обращается к данной памяти, и не от того, какой именно чип памяти содержит искомые данные.

Коммутатор – для решения проблемы с узким горлышком было решено создать коммутатор, где каждый порт соединён с каждым процессором, шину убрали. То есть каждый процессор в один момент времени может обращаться к одному из устройств. В результате производительность многопроцессорных машин резко возросла.

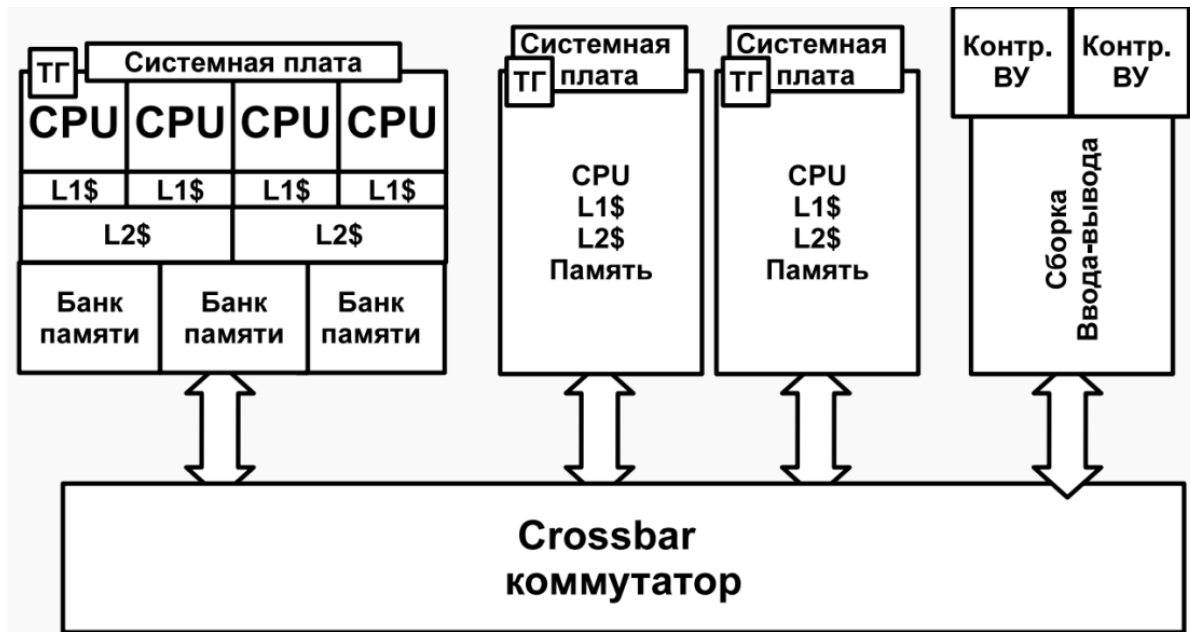
Также в явном виде разделили устройства на быстродействующие и медленные. Например, внешние устройства теперь подключаются через специальные мосты.



NUMA (Non-uniform memory access) – такая архитектура предназначена для того, чтобы ещё больше увеличивать количество процессоров.

У каждого процессора есть свой локальный кэш, локальная память и свой ТГ. Вся эта каша реализована как одна единая плата, соединённая при помощи коммутатора. Есть общая память для процессоров, но время обращения к ней зависит от того, кто к ней обращается.

Каждый процессор, теоретически, может работать на своей тактовой частоте, ну и плюсом ко всему можно на горячую менять элементы системы. Все процессоры такой архитектуры находятся под управлением ОС. ОС старается, чтобы вычисления не выходили за рамки системной платы, иначе производительность тупо упадёт.



Структура современных процессоров. Окружение процессора. CISC, RISC, VLIW.

Характеристика современных коммерческих процессоров

- Разрядность адреса и данных: 16/32/64 бита.
- Тактовые частоты: 500МГц – 5ГГц.
- Многопроцессорные с CPU от 1 до 100+ штук;
- Многоядерные с 1-16 ядер.
- От 1гб то терабайтов ОЗУ.
- Используют кэш-память разных уровней.
- Суперскалярные (процессор может выполнять более 1 команды за 1 такт)
- CISC, RISK, VLIW

CISC

CISC (Complete Instruction Set Computer) – полный набор команд

микропроцессора, который характеризуется:

- Сравнительно небольшое количество регистров общего назначения
- Большое количество машинных команды, которые семантически нагружены как операторы в высокоуровневых языках. Выполняется за большое число тактов

- Большое количество видов адресаций
- Большое количество команд различной разрядности

К подобным процессорам можно отнести семейство процессоров x86 от Intel, которые были отягощены обратной совместимостью со старыми процессорами.

Из недостатков CISC процессоров можно отметить:

- Высокая стоимость аппаратной части
- Сложность с распараллеливанием вычислений
- Состав и назначение их регистров существенно неоднородны
- Широкий набор команд усложняет декодирование инструкций, что расходует аппаратные ресурсы. Вследствие чего, возрастает число тактов, необходимых для выполнения инструкций.

RISC (Reduced Instruction Set Computer) – архитектура компьютера с сокращённым набором команд. Предполагает реализацию в ЭВМ сокращённого набора простейших, но наиболее часто используемых команд, что позволяет упростить аппаратную часть и, благодаря этому, повысить эффективность машины. Современные процессоры с RISC-архитектурой характеризуются следующим:

- Упрощённый набор команд, имеющих фиксированную длину
- Отсутствуют макрокоманды, усложняющие структуру процессора и замедляющие его работу.
- Взаимодействие с ОЗУ ограничивается лишь пересылкой данных.
- Уменьшено количество видов адресаций.

Новый подход к архитектуре позволил значительно сократить площадь, требуемую для покрытия на кристалле интегральной схемы. Это позволило увеличить количество регистров в процессоре, из чего последовало заметное уменьшение количества обращения к ОЗУ.

В пример можно привести новомодные процессоры Intel, где сам процессор является CISC, при этом ядро – RISC.

VLIW

VLIW (Very Long Instruction Word) – архитектура, которая появилась в России. Она не попадает под принципы фон Неймана, так как нарушается принцип последовательного выполнения команд.

Архитектура ЭВМ с VLIW позволяет сократить объём оборудования, нужного для реализации параллельной выдачи нескольких команд. Вместо того, чтобы последовательно выдавать команды, операции упаковываются в одну очень длинную команду. Ответственность за упаковку команд кладётся на компиляторы.

Примеры процессоров: Intel Itanium и микропроцессоры “Эльбрус”.

Структура современного процессора

Ядро процессора – это его основная часть, содержащая все функциональные блоки и осуществляющая выполнение всех логических и арифметических операций.

Принцип работы ядра процессора основан на цикле, описанном ещё Джоном фон Нейманом. В упрощённом виде этапы цикла работы ядра процессора можно представить следующим образом.

1. Блок инструкций проверяет наличие прерываний. Если оно было, то происходит сохранение текущего состояния ВУ. То есть данные регистров, счётчика команд заносятся на вершину стека. В счётчик адреса помещается адрес обработчика прерываний. По окончании прерывания, исходное состояние ВУ восстанавливается.
2. Блок выборки инструкций из счётчика команд считывает адрес команды, предназначенной для выполнения. По этому адресу находится сама команда в кэше, либо в ОЗУ. Полученные данные передаются в блок декодирования.
3. Блок декодирования расшифровывает команду, при необходимости используя для интерпретации команд микрокод из ПЗУ МПУ. Если это команда перехода, то в счётчик команд заносится адрес перехода, иначе счётчик команд инкрементируется (для процессора с длиной команды 32 бита – на 4 шага) и передаёт управление в блок выборки данных.
4. Блок выборки данных считывает из Кэш-памяти или ОЗУ нужные для выполнения команды данные. Управление передаётся планировщику.
5. Управляющий блок определяет, какому блоку выполнения инструкций обработать текущую задачу, и передаёт управление этому блоку.
6. Блоки выполнения инструкций делают своё дело и передают управление блоку сохранения результатов.
7. При необходимости, результат заносится в ОЗУ. После всех действий, управление передаётся блоку выборки команд.

Описанный выше цикл называется процессом. Отсюда и название самого процессора.

Скорость перехода от одного этапа цикла к другому определяется тактовой частотой ТГ процессора, а время работы каждого этапа и цикла зависит от внутреннего устройства ядра процессора.

Если говорить в общем, то у современных процессоров есть микросхемы поддержки и чипсеты. В основном преобладают высокоскоростные интерфейсы, например, слоты PCI и порты HDMI. Микросхема поддержки нужна для управления КБУ, которая связана с процессором каналом прямого доступа к памяти.

2. Память процессора (Кэш-память)

Находится непосредственно на кристалле и служит для временного хранения данных. Позволяет не тратить время на долгое обращение к оперативной памяти (RAM). [1]

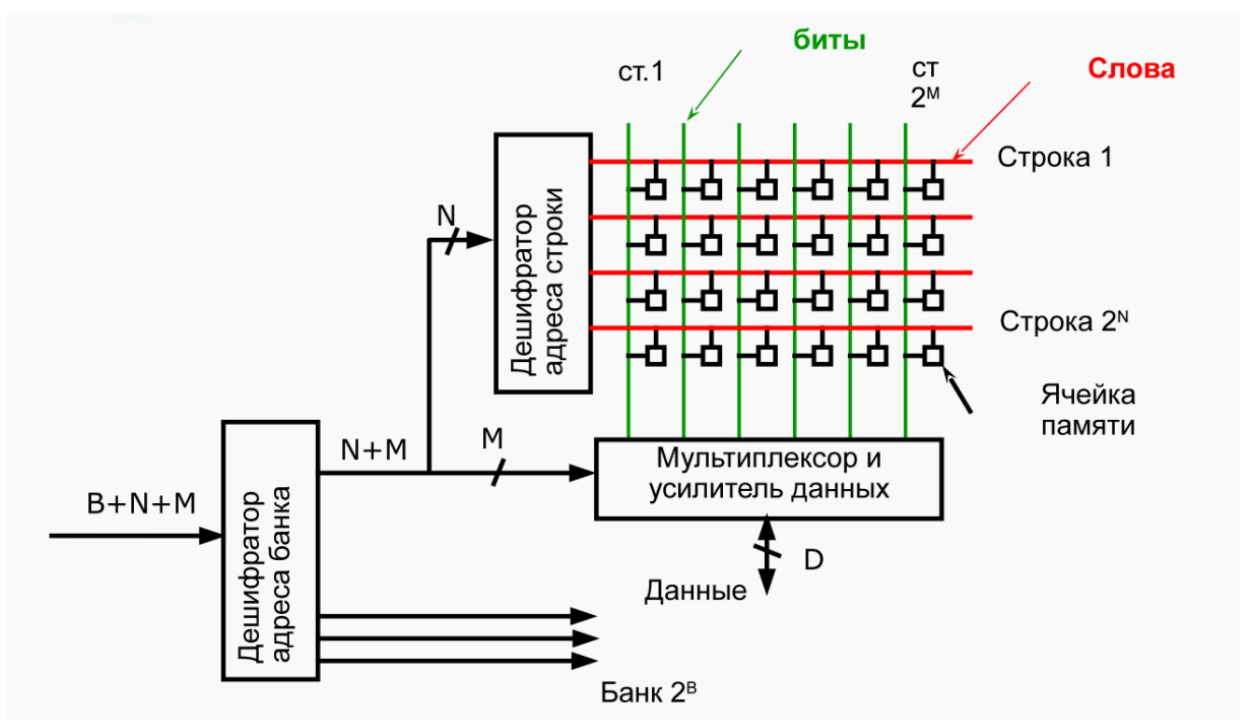
3. Специализированные блоки и сопроцессоры

Современные процессоры стали гетерогенными — они совмещают вычислительные задачи со специализированными аппаратными ускорителями на одном кристалле: [1, 2]

- **Графическое ядро (iGPU):** Обеспечивает базовый вывод картинки на монитор без необходимости покупки отдельной видеокарты.
- **Модуль нейросетевых вычислений (NPU):** Выделен для обработки алгоритмов искусственного интеллекта и машинного обучения.
- **Контроллеры:** Контроллер памяти (управляет обменом данными с ОЗУ) и контроллер шины PCI-Express (отвечает за обмен данными с видеокартой, SSD-накопителями и другими платами).

Адресуемая память, организация и временные диаграммы. Конструктивные особенности современной памяти.

Адресуемая память



Заместо того, чтобы использовать огромнейший дешифратор как в БЭВМ, решили разбить память на банки. Картинка из презы не совсем точная, так как сигнал N и M сразу подаётся к дешифратору и мультиплексору соответственно. Адресация к такой памяти выглядит следующим образом:

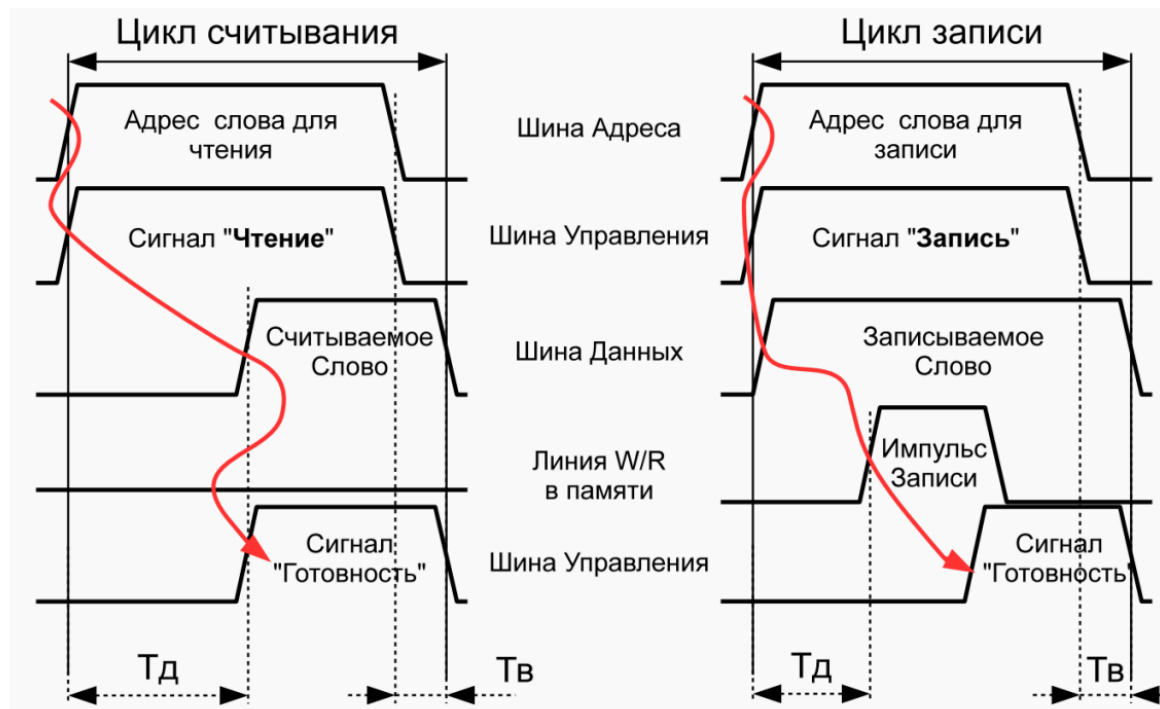
BANK	N	M
------	---	---

Соответственно каждый банк представляет из себя вот такую матрицу. Подобная конструкция памяти позволяет значительно упростить её схему. Мультиплексор работает таким образом, что на вход подаётся сигнал, который указывает, какой конкретно вход нужно пустить на выход мультиплексора и так далее.

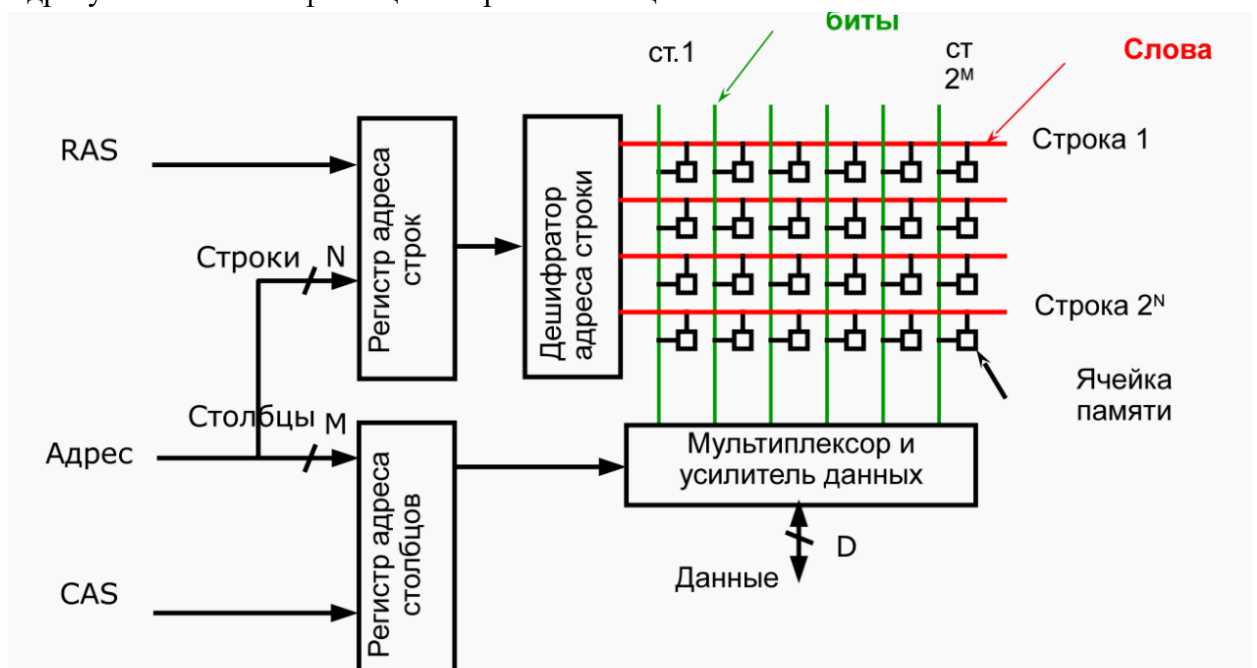
Временные диаграммы

Важно отметить, что линия W/R не на шине, а внутри памяти. По сигналу чтения срабатывают дешифраторы.

- T_d – время доступа к памяти. То есть это промежуток времени от начала запроса данных до получения первых данных.
- T_v – время восстановления памяти. То есть это промежуток времени, за которое память возвращается в исходное состояние.



Адресуемая память с фиксацией строк и столбцов



Люди решили передавать адрес не полностью, а разбить его на две части: адрес строки и адрес столбца. Половина адреса передаётся по шине памяти в один такт времени, другая половина – в следующий такт времени. Для этого используются новые управляющие сигналы RAS (Row Address Select) и CAS (Column Address Select). Теперь строка и столбец хранятся в регистрах, мы можем инкрементировать и декрементировать их, чтобы обращаться к последующим ячейкам в банке. Память поначалу будет работать в 2 раза медленнее, чем просто адресуемая память, но обращаться к ячейкам мы будем гораздо быстрее.

SDRAM (Synchronous Dynamic Random Access Memory) – память, которая уже используется в современных ЭВМ. Появился собственный тактовый генератор, сигналы инвертированы, так как это было удобнее производителям. Output enable – подключение к выходу памяти. Теперь память синхронизируется с частотой процессора.

DDR SDRAM (Double Data Rate SDRAM) – отличается от обычной SDRAM тем, что имеет вдвое больше пропускную способность. Она может принимать и передавать данные и по фронту, и по спаду тактовых импульсов.

Конструктивные особенности современной памяти

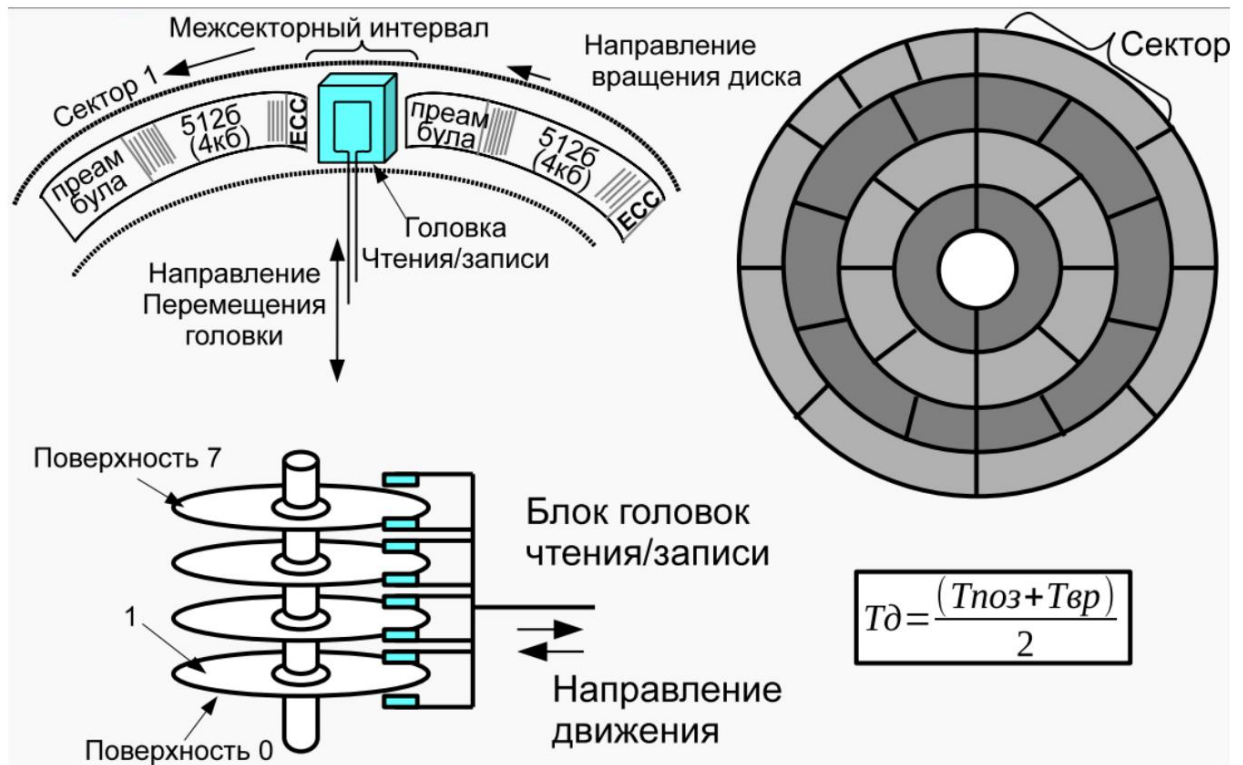
- Burst mode – пакетный режим, при котором запрос по конкретному адресу возвращаются не только данные, хранящиеся по этому адресу, но и пакет данных по нескольким последующим адресам.
- DDR – передача данных и по фронту и по спаду.
- SPD – чип, содержащий идентификационную информацию. В ней прописано, на какой частоте должна работать память. Чип читается процессором.
- Interleaving – расслоение памяти, при котором соседние по адресам ячейки размещаются в различных модулях памяти, так что появляется возможность производить несколько обращений одновременно. Такое расположение кусков памяти позволяет параллельно читать больше данных.

Память, ориентированная на записи (блочная память).

Организация дисковой памяти и памяти на магнитных лентах.

Блочная архитектура – адресное пространство разбивается на группы последовательных адресов, и каждая такая группа обеспечивается отдельным банком памяти.

Память, ориентированная на запись

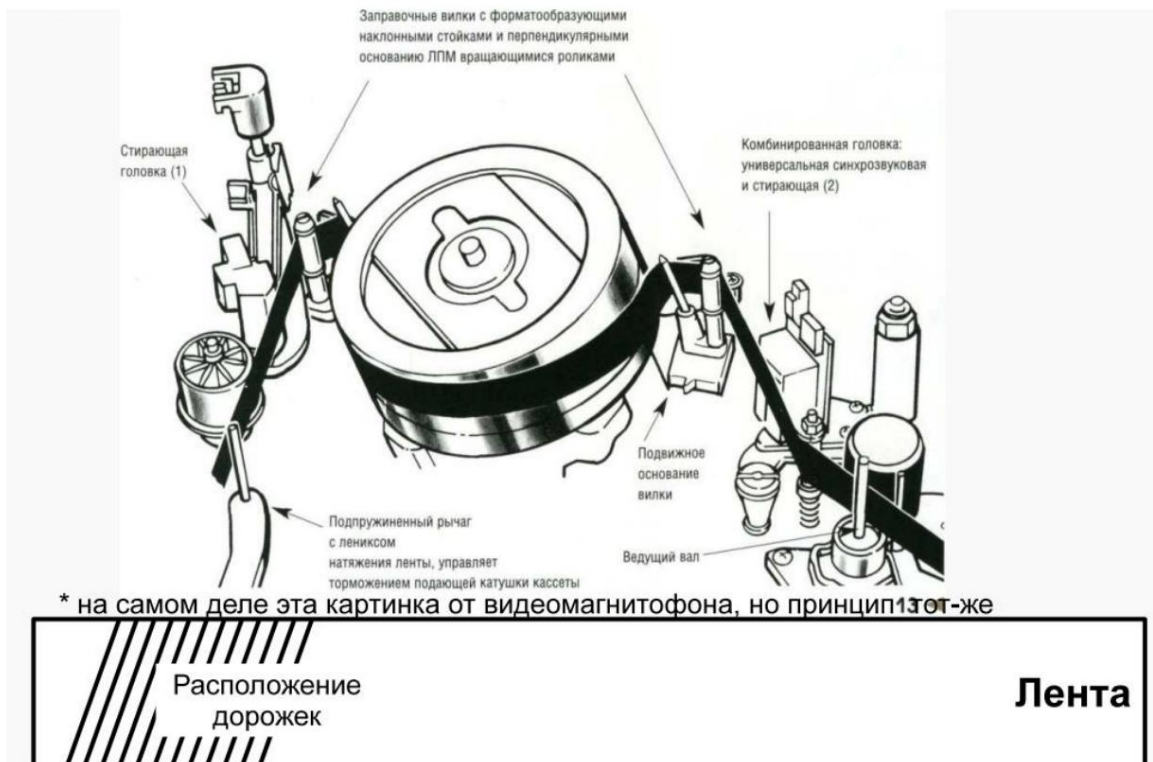


Диски (здесь пойдёт речь о жёстких) – память с блочным доступом.

Каждый раз передаётся блок диска. Блин – это поверхность, на которой расположена информация. Головки читают информацию с намагниченной поверхности. Головка идёт по диску, диск же разбит на секторы, у сектора есть преамбула (чтобы понять, что начался новый сектор), а также есть поле контроля чётности диска (ECC – ErrorCorrection Code). Дорожки имеют разный радиус, следовательно, их угловая скорость разная. Чем дальше от центра вращения, тем больше скорость чтения/записи. Поэтому обычно файлы подкачки располагают в начале диска, чтобы ЭВМ работала быстрее.

Время доступа к памяти считается как средняя сумма время позиционирования (время смещения головки до нужного блока) и время вращения (до нужного сегмента на диске).

Память с последовательным доступом



Чтобы получить доступ к ячейке памяти n , нужно пролистать $n-1$ штук ячеек до неё. Это дорого и медленно с точки зрения времени доступа, однако такие носители позволяют хранить огромные объёмы данных при сравнительной дешевизне.

Примером является ленточный накопитель. Ленту быстро разогнать нельзя, поэтому вращается головка и дорожки записываются под наклоном.

Плёнка энергоэффективна – плёнке не требуется питание для хранения данных. Она также является очень надёжной, так как вероятность возникновения ошибки при чтении и записи на 4-5 порядков ниже, чем у жёстких дисков. Плёнка безопасна, так как её необязательно подключать к компьютеру, что обрывает возможность несанкционированного доступа к данным.

Чаще всего ленты используют для хранения каких-то бэкапов, которые целиком пишутся на неё.

Характеристики запоминающих устройств. Пирамида памяти.

Характеристика памяти

1. Месторасположение: процессорные, внутренние, внешние.
2. Ёмкость: в метрических (Кило-) и двоичных (Киби-) множителях.
3. Единица пересылки: слово, строка кэша, блок на диске.

4. Метод доступа: произвольный (адресный), ориентированный на записи (прямой), последовательный, ассоциативный

5. Быстродействие и временные отношения:

a. Время доступа, T_d – промежуток времени от начала обращения до передачи информации

b. Длительность цикла памяти (время обращения), $T_{ц}$ – обратились, получили, произошло восстановление в исходное состояние.

c. Время чтения и время записи

d. Время восстановления памяти, T_v – промежуток времени, за которое память возвращается в исходное состояние.

e. Скорость передачи информации. Используется единица измерения БОД – бит в секунду.

6. Физический тип и особенности

7. Стоимость (одного байта)

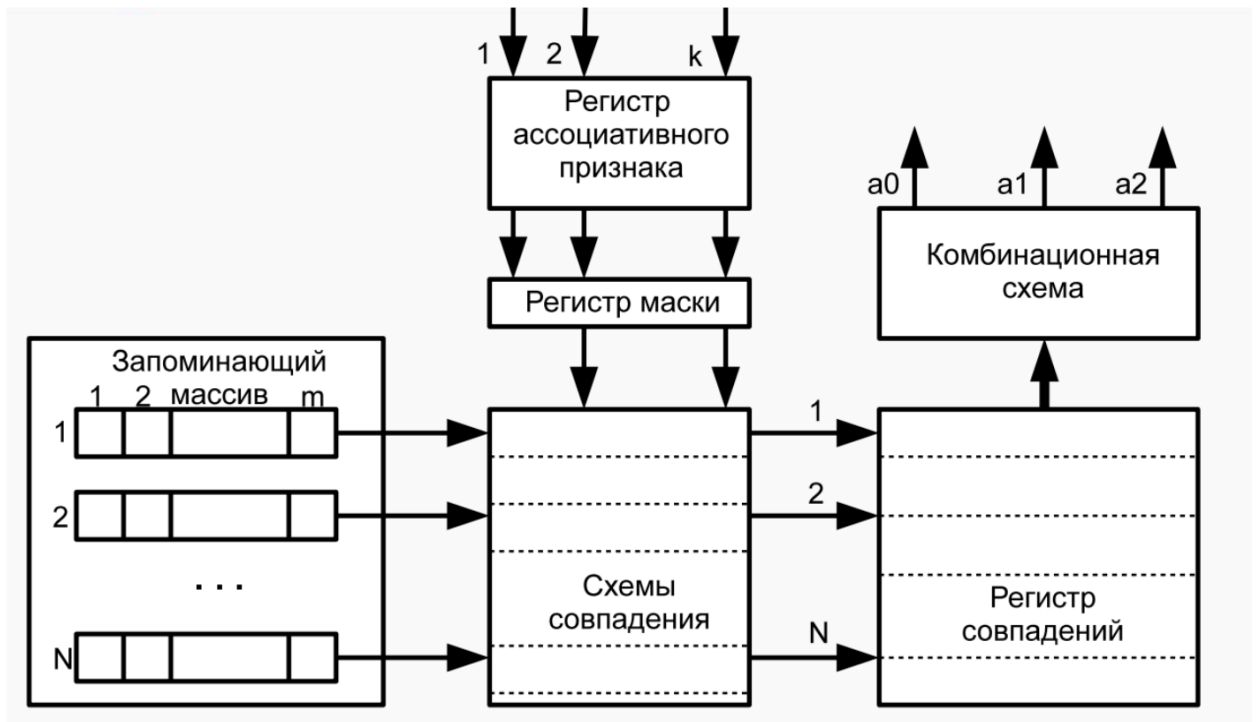


The diagram shows a pyramid representing the memory hierarchy, with levels from top to bottom: CPU, L1 Cache, L2-L3 Cache, Основная память (Main memory), SSD, Жесткие диски (Hard disks), and Магнитные ленты (Magnetic tapes). The pyramid is drawn with a thick black outline.

	Объем	T_d	*	Тип	Управл.
CPU	100-1000 б.	<1нс	1с	Регистр	компилятор
L1 Cache	32-128Kб	1-4нс	2с	Ассоц.	аппаратура
L2-L3 Cache	0.5-32Mб	8-20нс	19с	Ассоц.	аппаратура
Основная память	0.5Гб-4ТБ	60-200нс	50-300с	Адресная	программно
SSD	128Гб-1Тб/drive	25-250мкс	5д	Блочн.	программно
Жесткие диски	0.5Тб-4Тб/drive	5-20мс	4м	Блочн.	программно
Магнитные ленты	1-6Тб/к	1-240с	200л	Последов.	программно

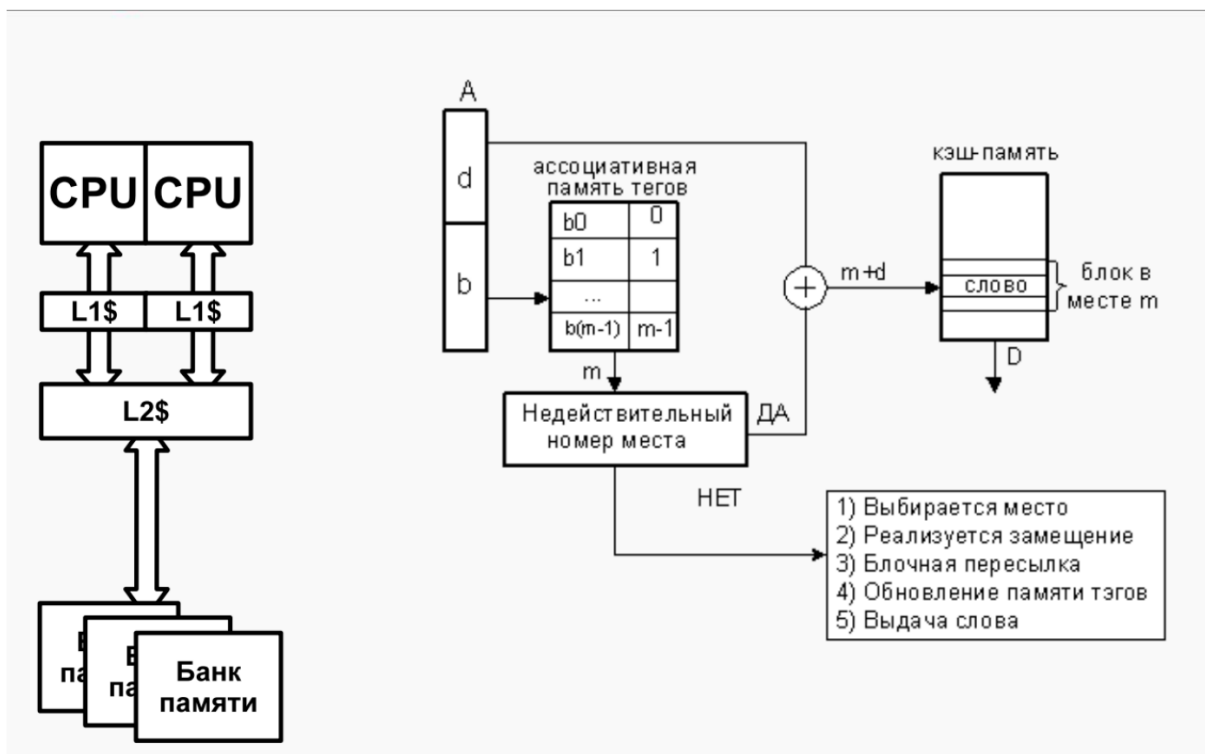
Ассоциативная память, Кэш-память. Влияние промахов кэш-памяти на производительность.

Ассоциативная память



Ассоциативная память построена по иному принципу, чем адресуемая память. Она выбирает информацию не по адресу, а по нужному нам признаку. То есть у нас есть регистр ассоциативного признака, а каждая ячейка памяти содержит в себе схему сравнения (компаратор), то есть всё, что поступает на неё, сравнивается на наличие необходимого содержимого и те ячейки, которые совпадают, передаются при помощи регистров сравнения в дальнейшую комбинационную схему. Подобным образом устроена кэш-память. За счёт того, что у каждой ячейки есть свой компаратор, такая память довольно дорогая.

Кэш-память



Кэш-память используется для уменьшения времени доступа к данным. Она хранит копии часто используемых данных из основной памяти. Например, список инструкций, которые могут храниться на так называемом нулевом уровне кэша. Структура кэш-памяти выглядит следующим образом:

1. Кэш-контроллер – управляет работой кэш-памяти, загружает данные и оперативной памяти и возвращается модифицированные данные обратно.
2. Процессор – запрашивает данные и кэш-контроллера.
3. Оперативная память – хранит все данные, к которым обращается процессор.

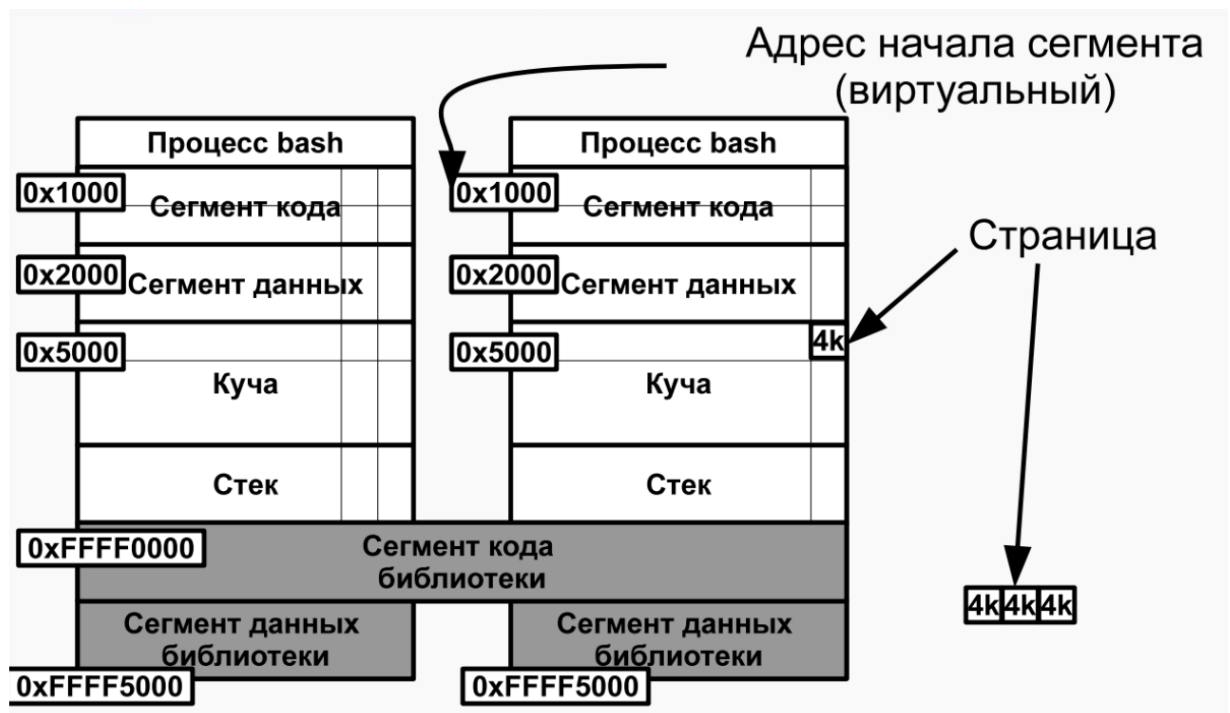
Адрес бьётся на некоторое количество частей, которые делятся на тег и просто адрес. В кэше нужные данные выбираются по тегу, и уже благодаря нему кэш возвращает искомую строку. Структура записи кэша выглядит следующим образом:

Блок данных	Тег	Бит актуальности
-------------	-----	------------------

При доступе процессора к кэшу сначала производится проверка, если ли искомые данные. Для этого производится сравнение адреса запроса со значениями всех тэгов кэша, в которых эти данные могут храниться. Если они есть, то данные извлекаются, то есть происходит кэш-попадание. Иначе происходит кэш-промах, и кэш-контроллер запрашивает данные уже из оперативной памяти компьютера.



Предназначение и организация виртуальной памяти.
 Сегментно-страничная организация. Устройство управления памятью (MMU), буфер трансляции (TLB).



Виртуальная память – это технология управления памятью компьютера, которая создаёт у каждого пользовательского процесса иллюзию того, что ему единолично предоставлен большой, непрерывный и однородный массив адресов (виртуальное адресное пространство), независимый от реального объёма физической памяти. Виртуальная память нужна для:

1. Изоляция и безопасность процессов. Процессы не могут обратиться к памяти друг друга или ядру ОС напрямую. Если один процесс упадёт из-за ошибки адресации, остальные процессы продолжают свою работу

2. Абстракция от физических адресов. Компилятору и программисту не нужно знать, сколько физической памяти установлено в системе и по каким адресам она распаяна по плате. Программа должна рассчитывать на стандартный диапазон адресов.
3. Эффективное использование ОЗУ. Неиспользуемые на текущий момент файлы можно временно выгрузить на жёсткий диск в файл подкачки, освободив физическую ОЗУ для активных задач.
4. Упрощение совместного использования памяти. Разные процессы могут использовать одни и те же разделяемые библиотеки, которые физически выгружены в ОЗУ в единственном экземпляре, но отображаются в адресном пространстве каждого процесса.

Сегментно-страничная организация памяти

Сегментная организация виртуальной памяти (логическое деление) – вид организации, при котором виртуальное пространство делится на части произвольного размера – сегменты.

Такой механизм позволяет разделять память на логические блоки. Как пример: сегмент кода, сегмент данных, сегмент стека. У каждого сегмента есть свои права доступа для пользователя или процесса.

Страничная организация виртуальной памяти (физическое деление) – и физическая, и виртуальная память делится на жёсткие блоки фиксированного размера – страницы. Виртуальная страница отображается на физическую страницу в ОЗУ.

В современных системах общего назначения чаще всего используют как раз гибрид вышеописанных организаций памяти.

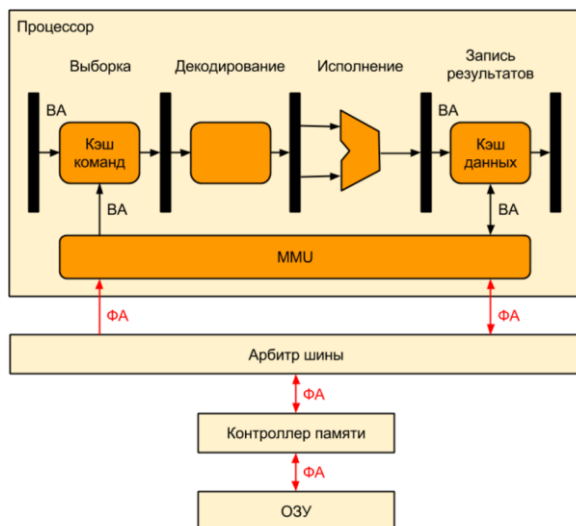
Принцип работы основан на том, что сначала виртуальный адрес, генерируемый процессором, проходит через блок сегментации. Он проверяет права доступа к сегменту и преобразует логический адрес в линейный (промежуточный) адрес.

Полученный линейный адрес передаётся в блок страничной организации, который делит его на три части:

1. Номер каталога страниц (Page Directory Index) – для многоуровневых таблиц.
2. Номер таблицы страниц (Page Table Index)
3. Смещение внутри страниц (Offset)

По вышеописанным индексам находится нужная запись таблицы страниц (PTE), где хранится базовый индекс физической страницы. К базовому индексу смещения прибавляется смещение, в результате чего получается физический адрес в ОЗУ.

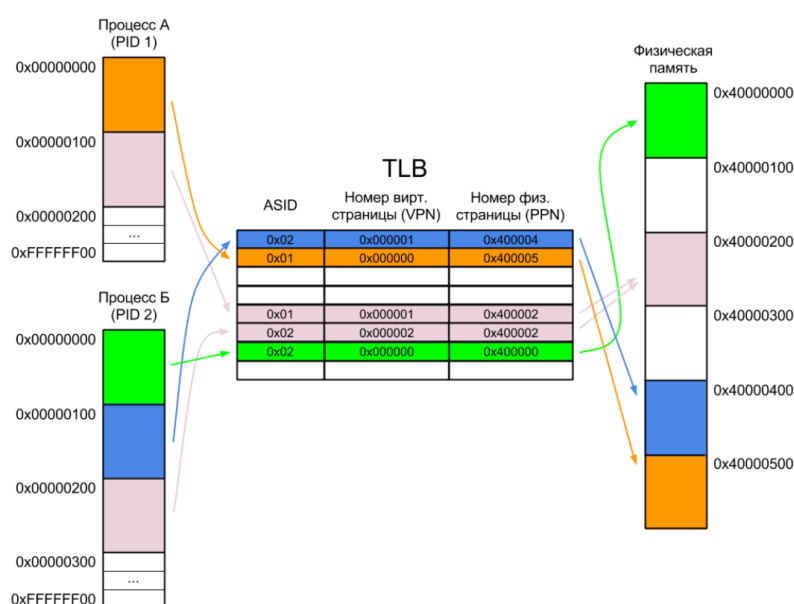
MMU (Memory Manager Unit)



MMU – аппаратный блок, расположенный непосредственно между ядром CPU и системной шиной памяти. Блок используется для:

- Преобразования виртуальных адресов команд и данных, с которыми работает процессор, в реальные физические адреса ОЗУ.
- Проверки наличия прав у процесса на выполнение определённой операции, например, запись в страницу, помеченную как “только для чтения”. Если происходит нарушение, то MMU генерирует аппаратное исключение (Page Fault).
- Управления кэшированием страниц: в записях таблиц страниц хранятся биты, указывающие, можно ли кэшировать данную область или нет. Если страничное управление выключено, то MMU просто приравнивает виртуальный адрес к физическому.

TLB (Translation Lookaside Buffer)



Поскольку таблицы страниц хранятся в ОЗУ, для каждой трансляции адреса процессору пришлось бы делать несколько дополнительных обращений к медленной памяти. Для

решения данной проблемы внутри ММУстроен TLB – ассоциативный буфер трансляции адресов.

TLB – это специализированная, очень быстрая аппаратная кэш-память, которая хранит соответствия между номерами виртуальных страниц и номерами физических страниц для самых последних обращений к памяти.

Когда виртуальный адрес необходимо преобразовать в физический адрес, то вначале просматривается TLB. Если совпадение найдено (TLB hit), то возвращается физический адрес, и доступ к памяти может продолжаться. Если совпадение не найдено (TLB miss), то просматривается вся таблица страниц.

Если сопоставление существует, то оно записывается обратно в TLB, так как аппаратное обеспечение обращается к памяти через TLB в системе виртуальной памяти.

Сетевые технологии, Понятие сети ЭВМ, классификация компьютерных сетей. Сообщение и пакет. Модель взаимодействия открытых систем.

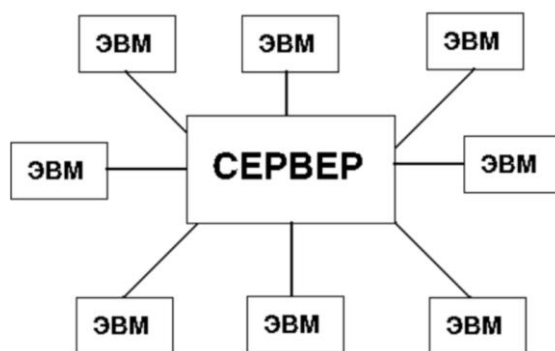
Под сетью ЭВМ подразумевается соединение двух или более ЭВМ с целью совместного использования их ресурсов (процессоров, устройств памяти, устройств ввода-вывода, данных).

По степени охвата территории можно выделить:

1. Локальные (местные) – в пределах одного учреждения, помещения или в радиусе 1 километра.
2. Региональные – внутри населённого пункта, района
3. Национальные – внутри государства
4. Глобальные

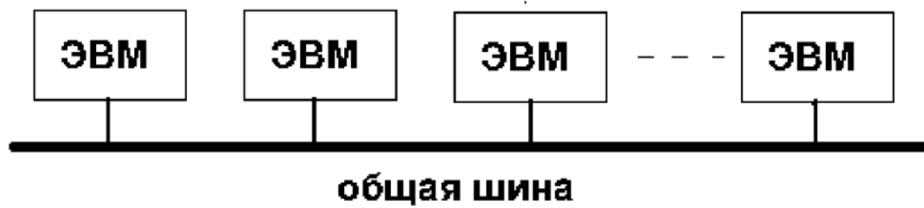
По степени доступности отмечают корпоративные и общедоступные сети.

По топологии:

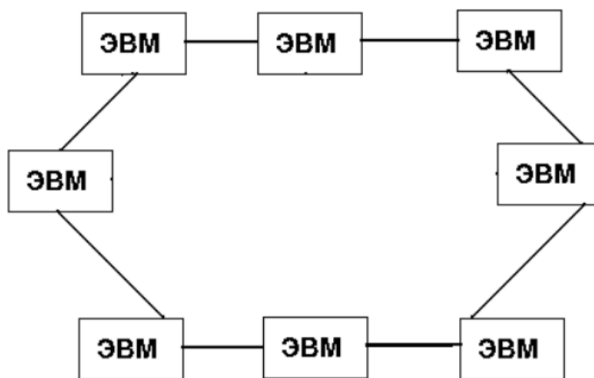


- 1) Звезднообразная топология – при такой топологии обмен данными между ЭВМ происходит через более мощную ЭВМ – сервер. Недостатком такой сети является

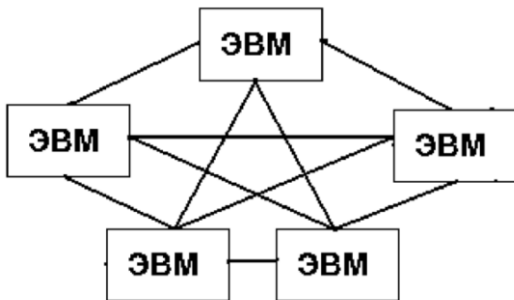
низкая живучесть, ведь если сервер упадёт, то упадёт вся система. Однако из плюсов можно отметить простоту и дешевизну всей схемы.



- 2) Топология с общей шиной – при таком способе обмен данными происходит через общую шину, которая, используя механизм прерывания, может захватить тот или иной компьютер. Характерной особенностью здесь является отсутствие сервера. Часто используется в домашних сетях.

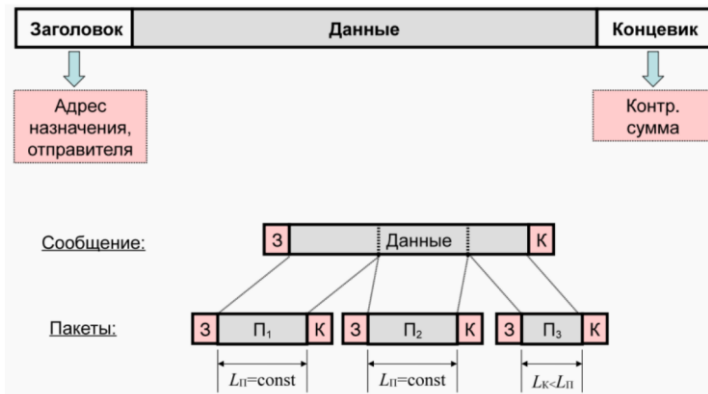


- 3) Кольцевая топология – в этой структуре каждая ЭВМ, используя механизм прерывания, работает в качестве ретранслятора. Важно, что система более живучая, чем с общей шиной, ибо при обрыве связи между соседними ЭВМ, сеть продолжить функционировать



- 4) Полная топология – соединение ЭВМ “каждая с каждой”. Позволяет получить хоть и дорогую сеть, но при этом самую живучую.

- Сообщение – цифровые данные определённого формата, предназначенные для передачи
- Пакет – это определённым образом оформленный блок данных, передаваемый по сети. Часто состоит из заголовка и полезной нагрузки (payload)

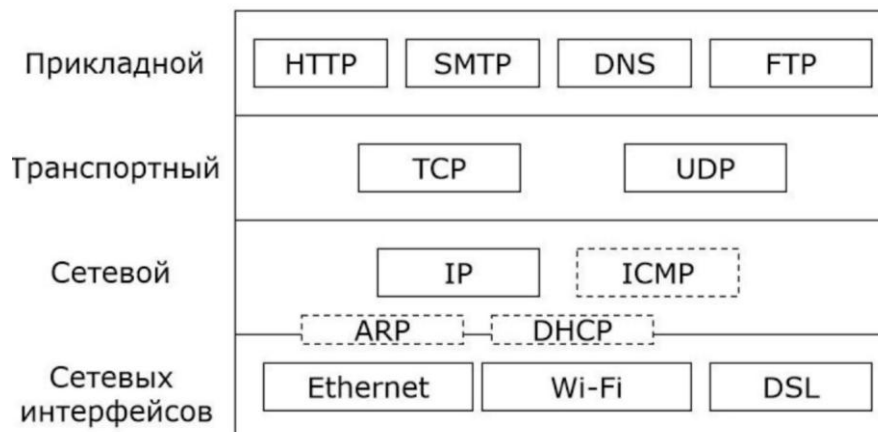


87

Модель открытых систем OSI

Модель открытых систем OSI					
Уровень (layer)		Тип данных	Функции уровня	Особенность адресации	Примеры протоколов
Уровни хоста (узла)	7. Прикладной (application)	Данные (строки из байтов)	Доступ к сетевым службам	URL	HTTP(S), FTP(S), RPC, POP3
	6. Представительский (представления) (presentation)		Представление (кодировка) и шифрование данных		ASCII, EBCDIC
	5. Сеансовый (session)		Управление сеансом связи		PAP
	4. Транспортный (transport)	Сегменты (segment) / Дейтаграммы (datagram)	Прямая связь между конечными пунктами и надёжность	Порт	TCP, UDP, SCTP, PORTS
Уровни связи (сети)	3. Сетевой (network)	Пакеты (packet)	Определение маршрута и логическая адресация	IP-адрес	IPv4, IPv6, IPsec, AppleTalk
	2. Канальный (data link)	Биты (bit) / Кадры (frame)	Физическая адресация	MAC-адрес (физический адрес компьютера)	PPP, IEEE 802.22, Ethernet, DSL, ARP, L2TP, сетевая карта.
	1. Физический (physical)	Электрические сигналы, Биты (bit)	Работа со средой передачи, сигналами и двоичными данными		USB, кабель ("витая пара", коаксиальный), оптоволоконный), радиоканал

Модель TCP/IP: передающая среда, канальный и сетевой уровень. Адресация, передача и маршрутизация пакетов.



Модель TCP/IP – это многоуровневая структура, или же сетевой стек, который стандартизирует обмен информации в интернете. Набор интернет-протоколов описывает следующие правила взаимодействия устройств:

1. В стеке протоколы нижних уровней работают непосредственно с сетевым оборудованием, в то время как протоколы верхних уровней взаимодействуют с пользовательскими приложениями.

2. При отправке данные проводят вниз по уровням стека, при получении они поднимаются вверх в обратном порядке. Если в модели OSI 7 уровней абстракций, то в модели TCP/IP уже 4 уровня:

1. Канальный уровень
2. Межсетевой/сетевой уровень
3. Транспортный уровень
4. Прикладной уровень

Уровень передающей среды (Физический уровень)

Передающая среда определяет физическую основу для работы канального уровня. В зависимости от используемого сетевого оборудования выделяют следующие типы среды:

1. Коаксиальный кабель – является устаревшим вариантом (используется, например, при передаче сигнала с тарелки в телевизор на даче). Делится на тонкий (рассчитан на передачу до 50 метров) и толстый (до 500 метров).
2. Витая пара – состоит из медных проводов, разделяя на категории в зависимости от пропускной способности. Такой проводник может передавать до 100 гбит/с до 15 метров без потерь.
3. Оптика – одно из самых распространённых решений для передачи данных на большие расстояния. Позволяет передавать данные на расстояние до 80 километров в режиме одного луча
4. Беспроводная среда – в эту категорию входят Wi-fi, 3g, 4g и так далее. Обеспечивает пропускную способность до 15 гбит/с.

Канальный уровень отвечает за физические и логические соединения между устройствами в пределах одной локальной сети.

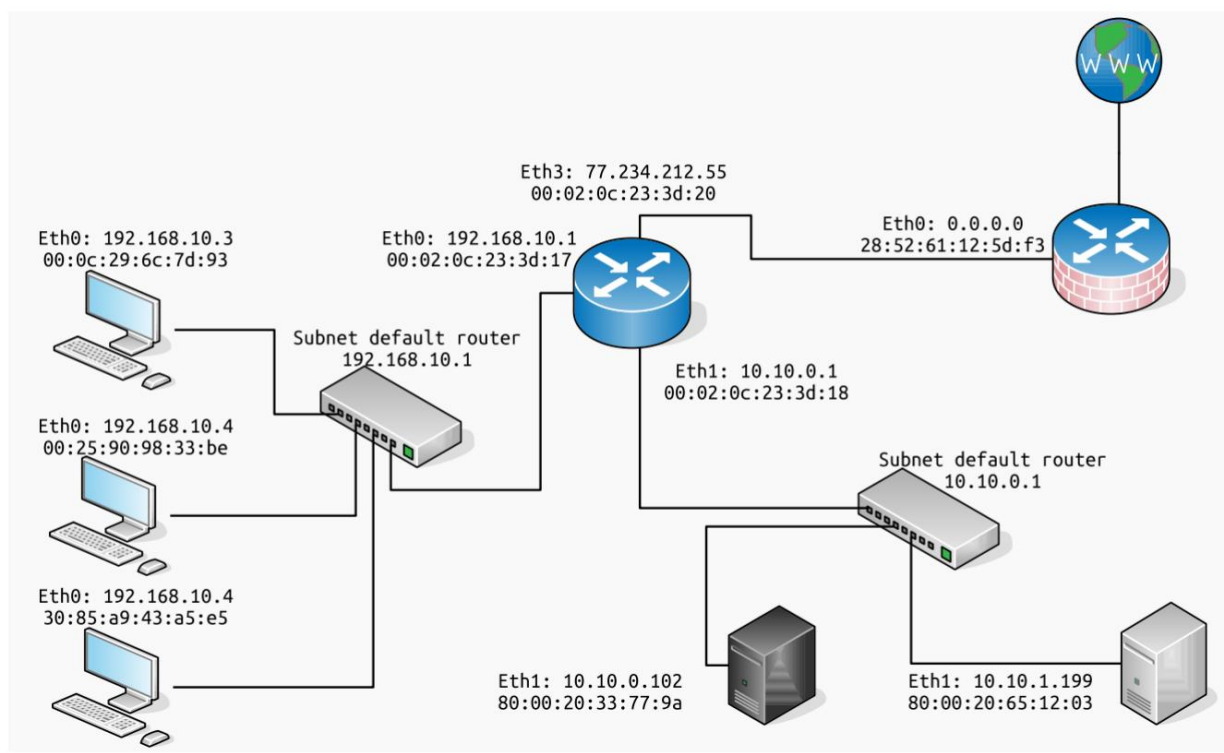
Единица данных здесь является кадр или фрейм. Заголовок кадра содержит MAC-адреса источника и назначения, а также тип передаваемых данных в кадре данных.

Адресация по отношению к узлу в локальной сети производится при помощи MAC-адреса (Media Access Control), состоящий из 6 байт, уникального для каждого узла в рамках одной локальной сети. Для его записи используется 16-ричная форма.

Базовой технологией и протоколом передачи на канальном уровне является Ethernet.

Из ключевых оборудования данного уровня можно отметить сетевой концентратор (или же хаб), который принимает на одном порту фреймы и ретранслирует сигнал на все остальные порты, не анализируя сами данные

Сетевой уровень и IP-адресация



Сетевой уровень обеспечивает взаимодействие между различными подсетями и выход данных за пределы локальной сети. Основным протоколом, использующийся на данном уровне, называется IP (Internet Protocol).

Единицей пересылки данных является пакет, который при передаче инкапсулируется внутри кадра канального уровня.

Адресация на данном уровне осуществляется при помощи IPv4 (IPv6), состоящий из 4 8-ричных чисел, записанных в 10-ной СС через точки.

Например: 198.168.1.1

Структура IP адреса выглядит следующим образом:

1. Сетевой префикс – общая часть для всех устройств в пределах одной подсети.
2. Идентификатор узла (хоста) – уникальный номер устройства в рамках одной сети.

Передача и маршрутизация пакетов

Маршрутизация – это процесс определения пути следования данных между различными сетями через интернет. Алгоритм передачи выглядит следующим образом:

1. Отправитель сравнивает IP-адрес получателя со своей маской подсети
2. Если адрес назначения принадлежит той же подсети, то происходит прямая передача, то бишь данные передадутся на канальном уровне.
3. Если же получатель находится в другой подсети, отправитель передаёт пакет в роутер (маршрутизатор) – устройство, связывающее подсети.

Работы маршрутизатора и таблица маршрутизации

Для выбора пути роутеры используют таблицу маршрутизации. Каждая запись в такой таблице содержит:

- Сеть назначения и маску подсети
- Адрес следующего узла
- Выходной интерфейс
- Метрику (приоритет маршрута)

Пакет сначала поступает в роутер, где извлекается его IP-заголовок. IP-адрес назначения сопоставляется с записями в таблице маршрутизации. Если подходящий путь найден, то пакет пересылается на соответствующий интерфейс следующему узлу. Если маршрут отсутствует, то пакет отправляет по маршруту по умолчанию. Если такого маршрута нет, пакет отбрасывается и отправителю уходит уведомление о недостижимости узла.

На каждом транзитном уровне (роутере) пакет претерпевает изменения на нижних уровнях. Канальный уровень меняется таким образом, что в кадре перезаписывается MAC-адрес текущего роутера на MAC-адрес следующего роутера или же конечного узла.

Сетевой уровень остаётся неизменным: исходные IP-адреса отправителя и получателя в заголовке IP-пакета сохраняются на протяжении всего пути.

Модель TCP/IP: выделение адресов (DHCP), сервисы имен, транспортный и прикладной уровни.

Протокол DHCP (Dynamic Host Configuration Protocol) позволяет пользовательским ЭВМ при подключении к сети в автоматическом режиме получать необходимые настройки от DHCP-сервера: IP-адрес, маску подсети, основной шлюз и стандартный DNS-сервер.

Процесс получения настроек со стороны клиента выглядит следующим образом:

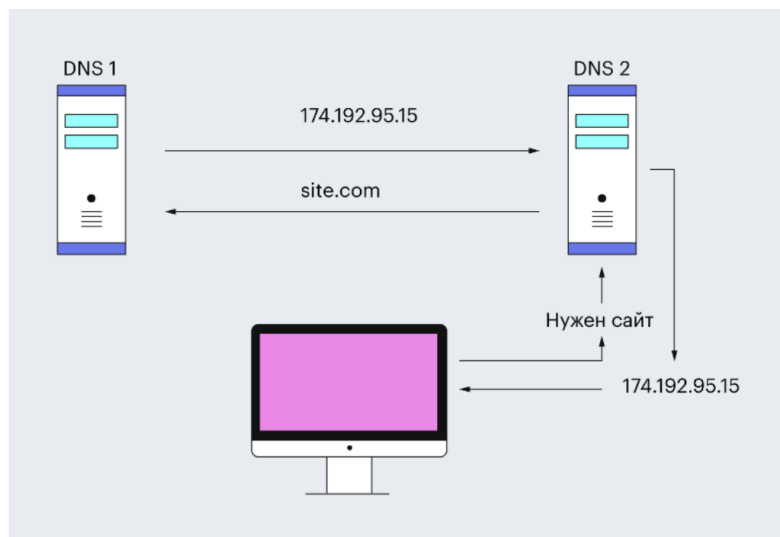
1. DHCPDISCOVER: клиент отправляет в сеть широковещательный запрос (его получают абсолютно все узлы локальной сети), чтобы обнаружить доступные DHCP-серверы.
2. DHCPOFFER (предложение): DHCP-сервер, получив запрос, отвечает клиенту предложением выдать определённый IP-адрес и сопутствующие параметры.
3. DHCPREQUEST (Запрос): Если клиент соглашается на предложение, то он отправляет запрос DHCPREQUEST. Данный пакет отправляется как широковещательный. Это необходимо для того, чтобы уведомить оставшиеся DHCP-серверы в сети, что их предложения отклонены и их услуги не требуются.
4. DHCPACK (Подтверждение): Выбранный сервер резервирует IP-адрес за клиентом и окончательно подтверждает право на его использование, отправляя пакет подтверждения DHCPACK

DNS (Domain Name System)

Для удобства пользователей TCP/IP предоставляет возможность заменить сложные для запоминания числовые IP-адреса на текстовые аналоги – доменными именами. За эту задачу отвечает служба DNS.

Принцип работы DNS основан так:

1. Когда устройству нужно узнать IP-адрес хоста по его доменному имени, оно отправляет запрос локальному DNS-серверу.
2. Если у локального DNS сервера есть запись о данном соответствии, то он отправляет IP-адрес клиенту
3. Если информации на сервере нет, то DNS-сервер обращается к вышестоящему DNS-серверу. На самом верху иерархии стоят корневые DNS-серверы.
4. Полученную от вышестоящего сервера информацию локальный сервер не только передаёт клиенту, но и кэширует у себя. При повторном запросе к этому же доменному имени, локальный сервер сможет сразу выдать адрес.



Транспортный уровень

Транспортный уровень обеспечивает надёжную доставку данных между приложениями на передающем и принимающем устройстве. Он выполняет три важные функции:

1. Отслеживание индивидуальных сеансов общения: (подобный прикол есть только у TCP) перед отправкой данных узел-инициатор устанавливает соединение с получателем, чтобы убедиться, что последний готов к приему. Все данные, передающиеся в рамках данной сессии, которая заканчивается по завершении обмена.
2. Сегментация данных: большие объёмы данных разбиваются на более мелкие блоки – сегменты – для их индивидуальной отправки по сети. На стороне получателя эти сегменты собираются обратно в исходном порядке.
3. Идентификация приложений: для распределения данных между разными программами используются номера портов. Каждое сетевое приложение регистрирует за собой порт в ОС. Получив данные, транспортный уровень считывает номер порта и перенаправляет данные нужному приложению.

Прикладной уровень

Прикладной уровень – это верхний уровень модели, представляющий собой набор разнообразных протоколов. Через эти протоколы пользователи и системные службы получают непосредственный доступ к сетевым ресурсам: файлам, принтерам, сайтам или электронной почте. На данном уровне единица данных считается сообщением.

В качестве примеров протоколов можно привести HTTP(S), которым пользуются веб-серверы для получения и обновления веб-контента.

Интерфейсы ввода-вывода. Контроллеры внешних устройств. Уровни стандартизации, сопряжения с системной шиной, циклы обмена. Регистры контроллера.

Интерфейсы ввода-вывода

Интерфейсы определяют конкретные детали обмена, например, частота, набор каналов передачи, способ кодирования, команды, представления данных, набор данных и их последовательность обмена.

Требуется аппаратная и программная реализация. Нуждаются в точной спецификации и стандартизации. Стороны должны однозначно интерпретировать детали обмена.

Уровни стандартизации

- Логическое подключение (все алгоритмы подключения понимаются системой) – в качестве примера можно привести подключение вилки в розетку (ток идёт и всё хорошо).
- Физический параметр сигнала (уровень напряжения, тока)
- Конструктивные особенности (физическая реализация разъёмов, портов и так далее)

Уровни сопряжения



1. Первый уровень сопряжения: соединение процессора с КВУ. На данном уровне какая-никакая стандартизация присутствует, например, в x86 процессорах от Intel. По большей части используется PCI Express в современных машинах

2. Второй уровень сопряжения: соединение КВУ с ВУ. Здесь уже присутствует большое количество стандартов, например, интерфейс USB с большим видов штекеров и скорость передачи информации.

Управляющие сигналы позволяют организовать 2 режима обмена информацией с КВУ:

1. Программно-управляемый. Сюда входит асинхронный и управляемый по прерыванию ввод-вывод
2. Прямой доступ к памяти

Сопряжение с системной шиной

При использовании команд ввода-вывода для обмена с ВУ адрес ВУ передаётся по шине адреса. По этой же шине передаются адреса ячеек памяти.

Информация на шине адреса имеет смысл только при наличии соответствующих управляющих сигналов. Например, сигнал на осуществление ввода или вывода.

При реализации обмена с ВУ нет необходимости в специальных сигналах, которые указывает на то, что в шине адреса расположен адрес ВУ. Для таких адресов выделено отдельное адресное пространство, а внутри КВУ происходит селекция адресов ВУ. Из управляющих сигналов можно только отметить приказ на “запись” или “чтение”, которые и регулируют обмен информацией между ВУ и процессором.

Циклы обмена

1. Операция вывода: со стороны шины ВУ по адресной шине подаётся адрес искомого ВУ, по шине данных передаётся слово, необходимое для вывода на ВУ, по управляющей шине поступает сигнал “Вывод”. КВУ принимает входные данные и при выполнении операции отправляет сигнал процессору о готовности. Данный сигнал как бы говорит процессору, что нужно снять информацию со всех шин.
2. Операция ввода: со стороны шины ВУ по адресной шине подаётся адрес искомого ВУ, по управляющей шине подаётся сигнал “Ввод”. КВУ принимает сигналы и отправляет данные на ВУ. После КВУ отправляет сигнал о готовности к процессору.

Регистры контроллеров

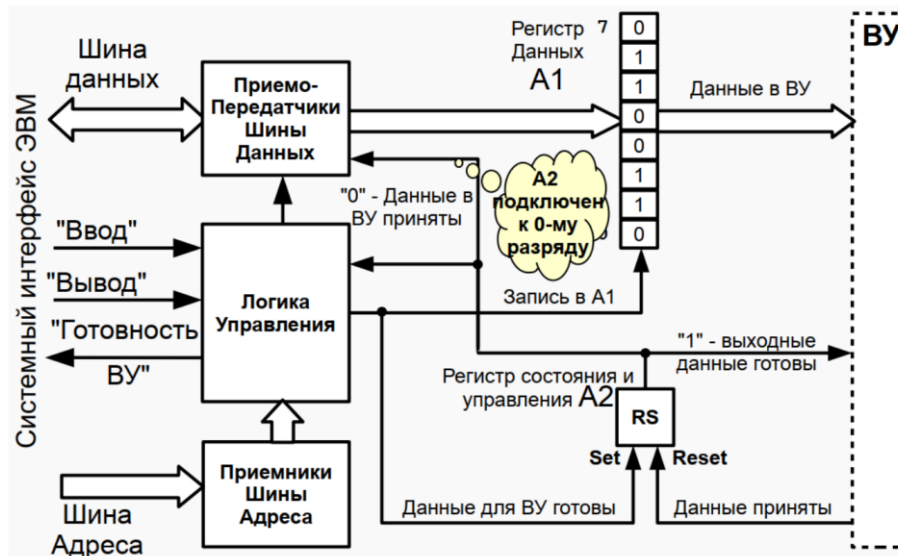
В основе контроллера лежит несколько регистров для временного хранения передаваемой информации. Каждый регистр имеет свой адрес (такие регистры называют портами ввода-вывода).

- Регистр входных данных – работает в режиме чтения
- Регистр выходных данных – работает в режиме записи
- Регистр состояния – содержит информации о готовности ВУ к обмену
- Регистр управления – служит для приёма приказов из микроЭВМ для ВУ.

Параллельная передача данных. Контроллеры параллельной передачи и приема.

Параллельная передача данных в ВУ под управлением асинхронного обмена:

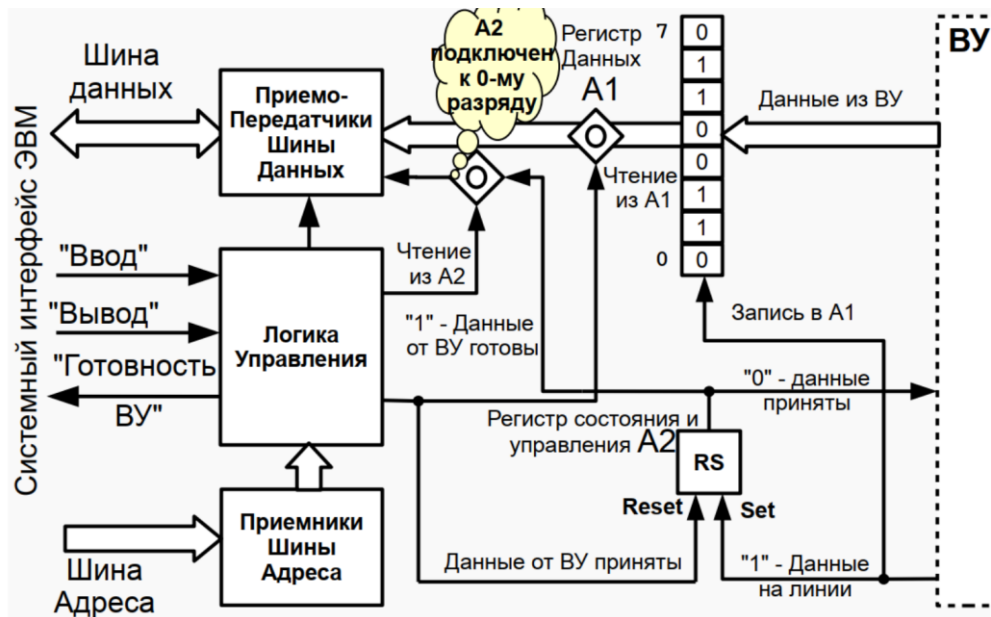
1. Процессор проверяет готовность ВУ к обмену данных.
2. Если ВУ готово к приему данных (логический 0 в регистре A2), то данные передаются из шины данных в регистр данных A1 контроллера. Если ВУ не готово, то идём в 1-ый пункт



При передаче данных в регистр A1, у нас идёт сигнал на установку логической единицы в RS-триггер. Состояние готовности на чтение со стороны процессора сбрасывается и отправляется сигнал к ВУ о готовности данных к выводу. Как только ВУ прочитает данные, отправляется сигнал о сбросе состояния RS-триггера. Логический ноль сигнализирует блоку логики управления, что данные были приняты со стороны ВУ и можно выводить следующую пачку данных

Особенность параллельной передачи данных заключается в том, что слово целиком передаётся на внешнее устройства, а не последовательно бит за бит. В плане реализации КВУ это проще, но с точки зрения количества шин (11 проводов vs 3 провода в последовательном) не очень хорошо.

Контроллер приёма параллельного асинхронного интерфейса.

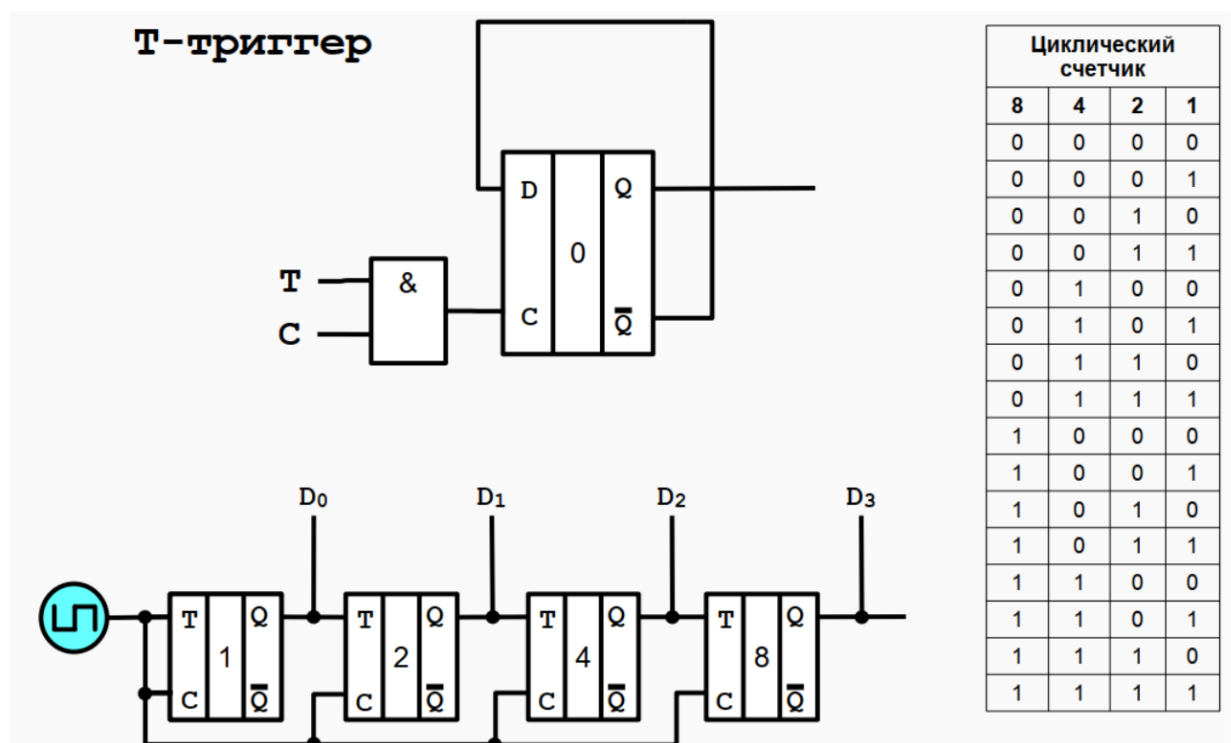


Логика работы такого контроллера противоположна контроллера передачи. Здесь логическая единица в регистре A2 сигнализирует о том, что данные готовы к чтению с КВУ. В блоке логики управления появился отдельный вентиль на чтение регистра состояния, который будет записываться в младший бит в шину данных. Чтобы разобраться в этой штуке, достаточно просто выучить схему

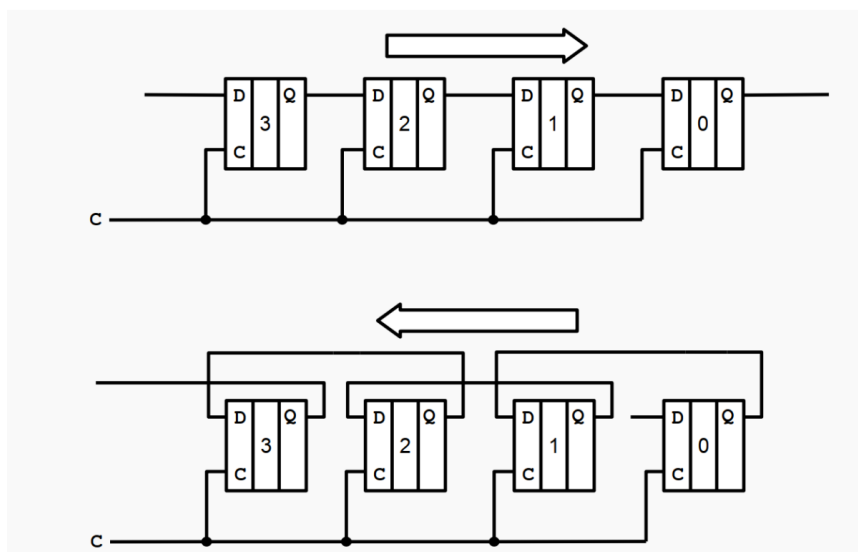
Синхронные последовательные интерфейсы. Контроллеры последовательной передачи и приема

В основе реализации последовательного ввода-вывода лежит Т-триггер, который позволяет считать количество импульсов тактового генератора.

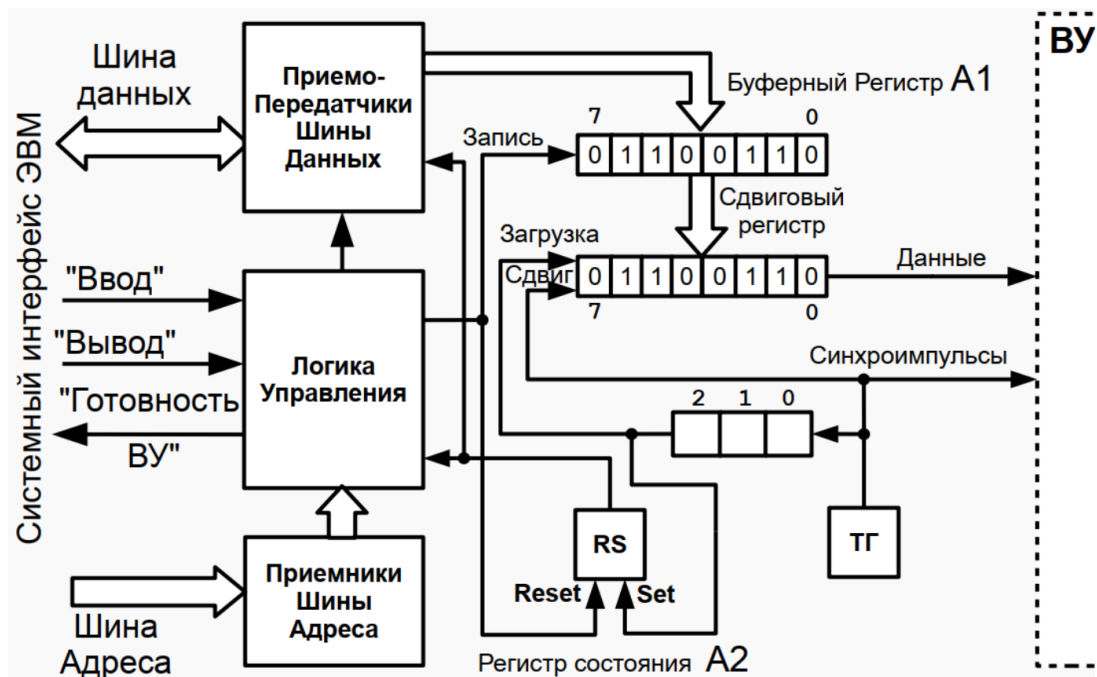
Цепочка из Т-триггеров образует счётчик, нужный в схеме реализации последовательного ввода-вывода.



Чтобы организовать битовый сдвиг, без участия коммутатора, можно воспользоваться схемой, также состоящая из Т-триггеров. В схеме контроллера последовательного интерфейса есть сдвиговый регистр, позволяющий преобразовать параллельный в последовательный код.



Контроллер передачи последовательного синхронного интерфейса



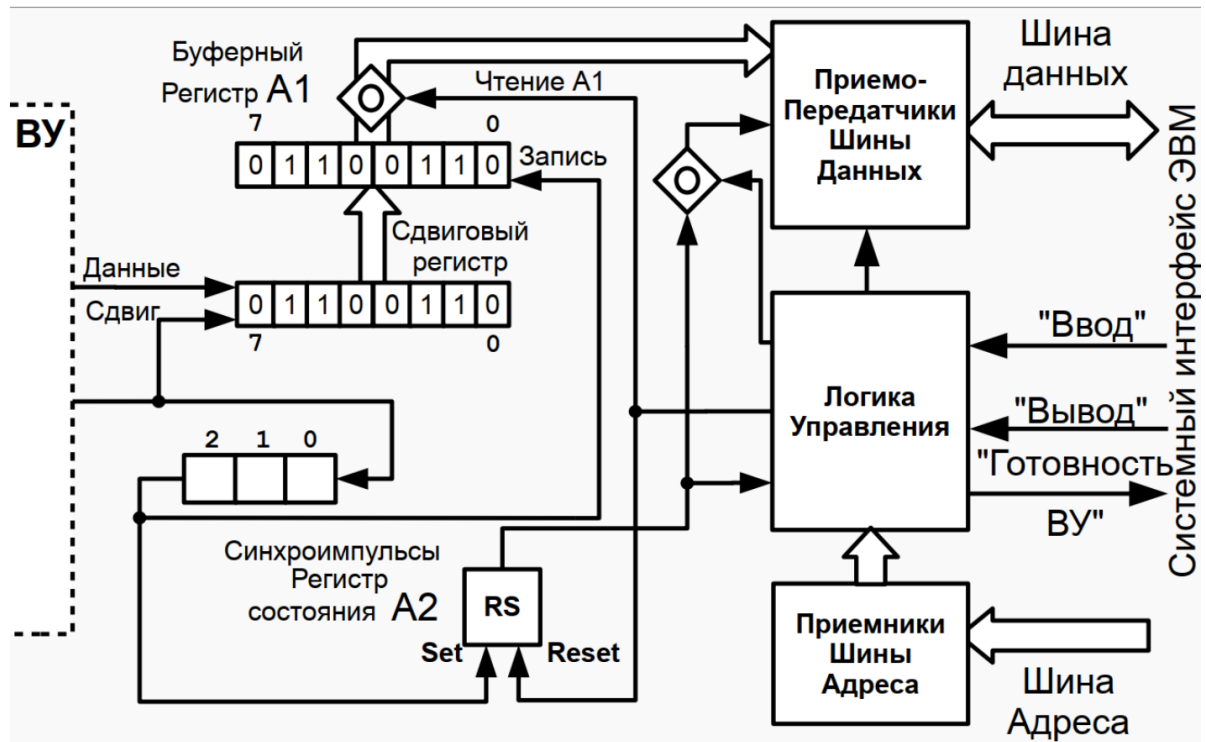
Под последовательным вводом-выводом подразумевается побитовая передача данных по одной шине. Это сокращается количество проводов, нужных для соединения контроллера с внешним устройством. Но при этом аппаратная часть усложнилась.

Когда идёт речь о синхронном интерфейсе, то всегда идёт речь о своём тактовом генераторе внутри КВУ. Он отправляет синхроимпульсы сдвиговому регистру для осуществления сдвига и внешнему устройству как сигнал подтверждения отправки данных.

Когда счётчик импульсов ТГ насчитает 8 импульсов, он отправит управляющий сигнал на загрузку нового значения в сдвиговый регистр из буферного регистра. К тому моменту ЭВМ должна подготовить новое значение для передачи данных, иначе последующая передача не будет осуществлена.

Следующим тактовым импульсом счётчик будет сброшен до 0 и начнётся очередной цикл выдачи 8 бит на ВУ.

Контроллер приёма синхронного последовательного интерфейса



Тактовый генератор располагается только на передающей стороне, то есть, внутри внешнего устройства. После 8 импульсов (сдвиговый регистр имеет все переданные биты), счётчик формирует сигнал записи в буферный регистр и одновременно с этим устанавливает готовность в регистр состояния.

Дальше ЭВМ уже может считывать данные, так как видит эту самую готовность. При чтении данных открывается вентиль чтения буферного регистра и данные идут в приёмо-передатчики шины данных, одновременно с этим снимается готовность регистра состояния.

Последовательный vs. параллельный обмен

1) Параллельный обмен:

- Быстрее при одинаковых условиях
- Дороже, так как требует больше аппаратных ресурсов
- Менее помехозащитный
- Дальность передачи меньше

2) Последовательный обмен – всё наоборот

Асинхронный обмен. Принцип деления частоты, формат кадра

При реализации асинхронного обмена интервал между командами ввода-вывода данных задаётся самим внешним устройством. Контроллеры подобных устройств снабжаются регистрами состояния (Как будет дальше в схемах – RS-триггеры), которые информируют ЭВМ о готовности внешнего устройства к обмену данными.

Преимущества:

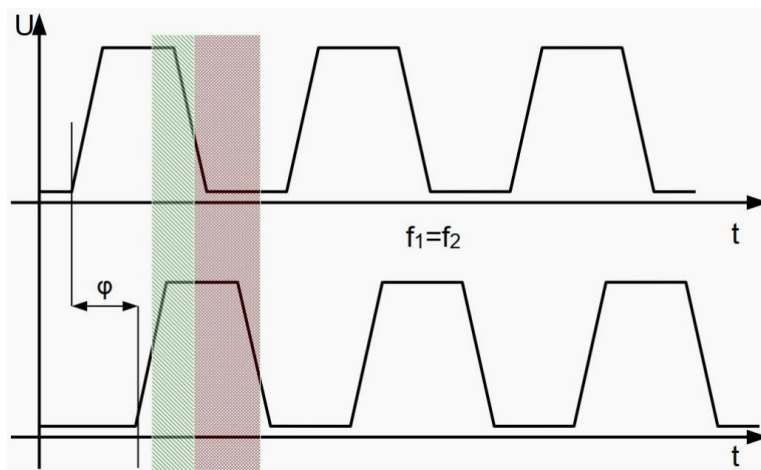
1. Не нужно знать точное время выполнения операции на ВУ.
2. ВУ всегда успеет выполнить обработку данных перед началом следующей операции.

Недостатки:

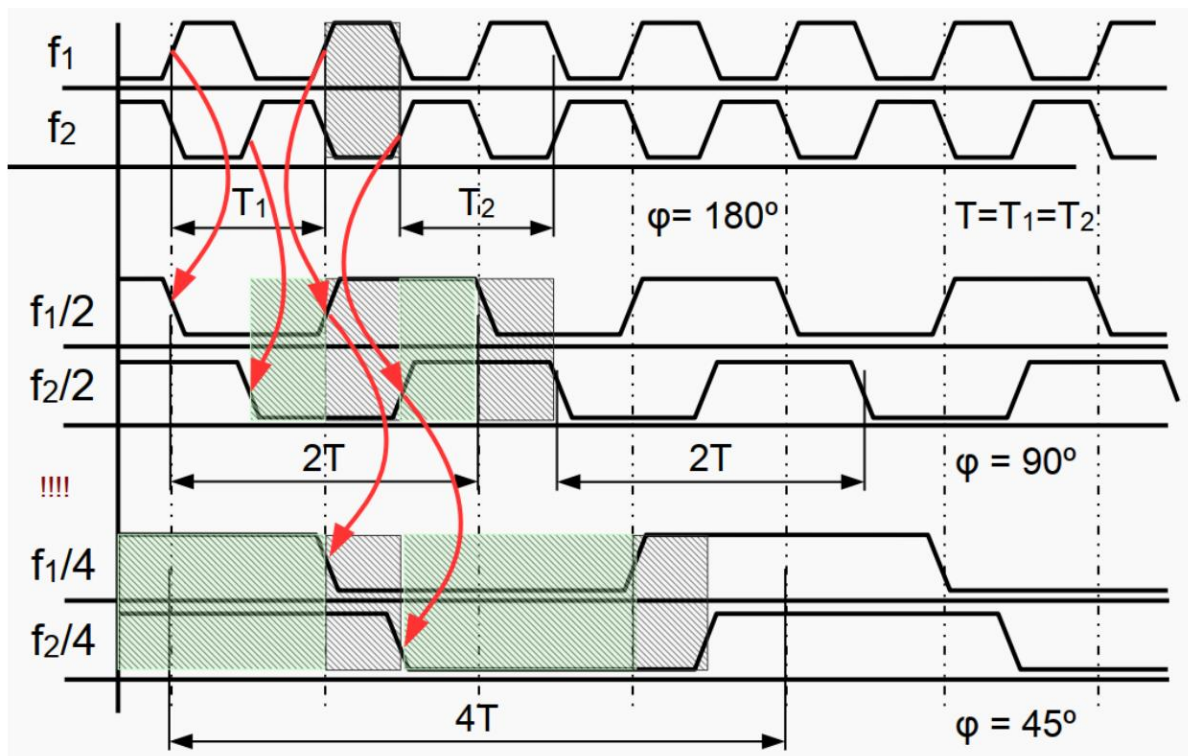
1. ЭВМ простаивает в ожидании готовности ВУ.

Принцип деления частоты

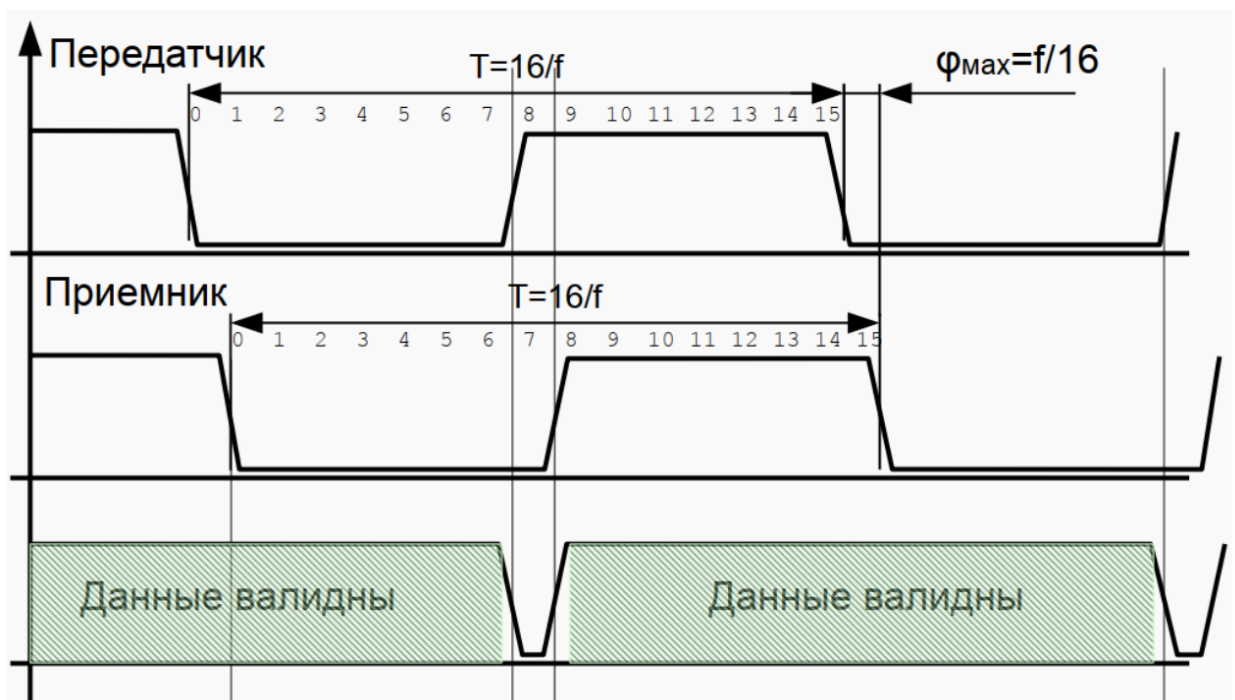
Люди решили, что нужно сократить накладные расходы в проведении кабеля между ВУ и КВУ. Шину данных и землю убрать нельзя, поэтому решили убрать шину с синхроимпульсом. Вследствие чего у КВУ и ВУ появились свои собственные тактовые генераторы. При асинхронной передаче данных рано или поздно может произойти рассинхронизация ТГ у передатчика и приёмника, что ведёт к искажению или потере данных.



Для решения данной проблемы решили делить тактовую частоту при помощи счётчика по модулю. Принцип основан на том, что длительность фазы увеличивается за счёт деления тактовой частоты и мы получаем больше промежутков тактового времени, когда фазы двух ТГ совпадают. Чем больше мы производим деление тактовой частоты, тем меньше будет ошибок, связанный с фазовым сдвигом.

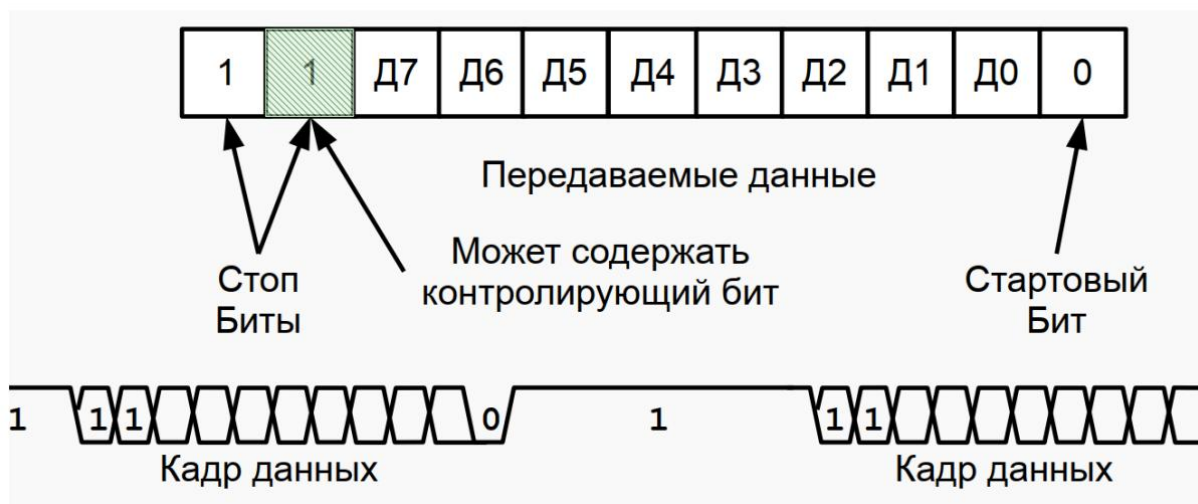


Причём скорость всей системы не будет проседать, так как можно изначально сильно повысить частоту тактового генератора (а её можно будет сделать сильно больше, чем максимальную частоту работы процессора), и потом просто её поделить.



Важно выбрать тот момент такта, когда можно осуществить передачу данных. Обычно выбирают момент времени, когда мы прошли какую-то часть валидной зоны. На картинке выше, например, это 14 или 15 момент времени.

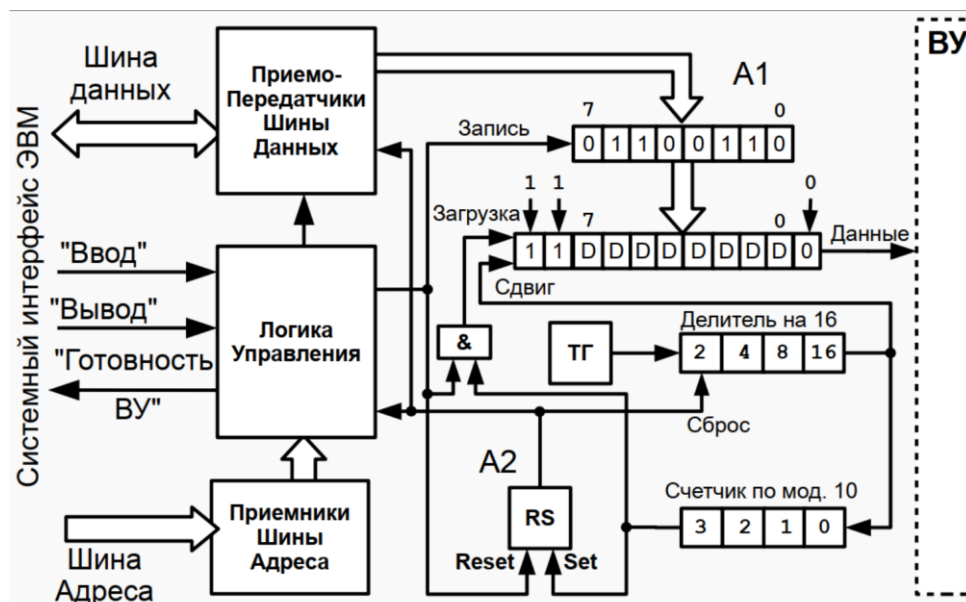
Формат кадра



Кадр представляет из себя последовательность битов, которые мы передаём при асинхронном обмене. Кадр состоит из стартового бита, всегда равный нулю, передаваемые данные, бит контроля чётности и стоп-биты.

В момент времени, когда данные не передаются, сигнал равен 1. Он обрабатывается приёмником таким образом, что его собственный генератор блокируется. При появлении стартового бита, равного 0, ТГ разблокируется и начинается процесс приёма данных. Подробно рассмотрено в следующих 2 вопросах

Контроллер передачи асинхронного последовательного интерфейса.



После передачи предыдущих байтов данных в регистр состояния A2 записывается 1, сигнализирующая о готовности к передаче новых данных.

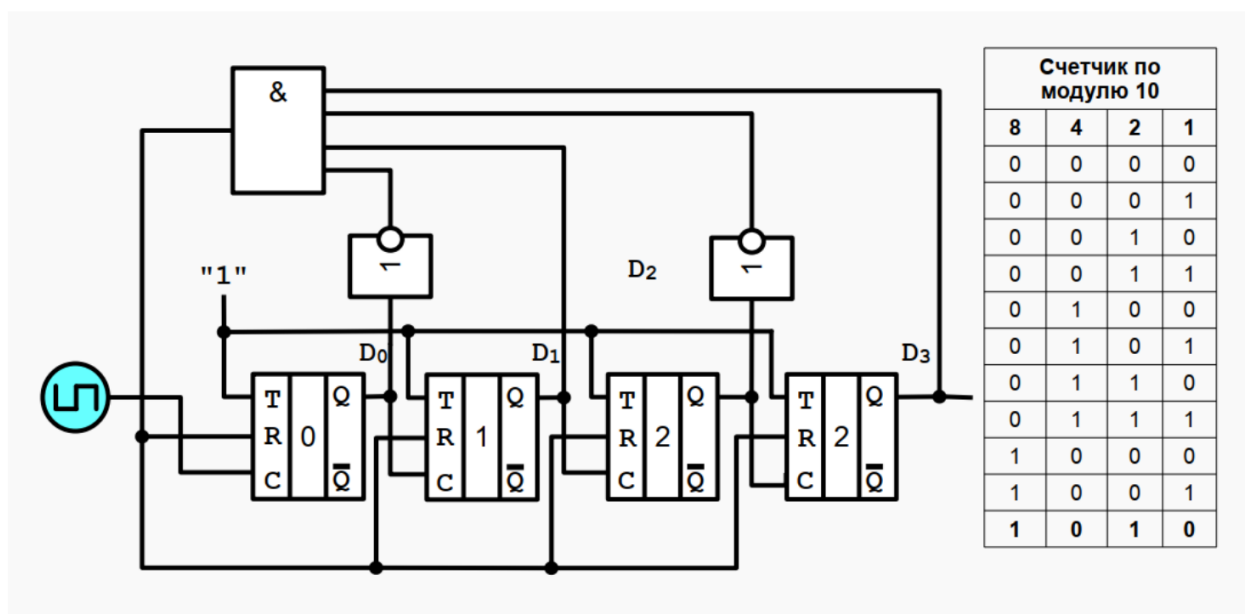
Этот же сигнал через шину “сброс” запрещает формирование импульсов сдвига в делителе частоты ТГ (mod 16). Счётчик по модулю 10 находится в нулевом состоянии, то есть на его выходе сигнал равен логической единице, которая поступает к схему логического “И”.

Система находится в ожидании сигнала “Вывод” из логики управления, который сбросит значение регистра состояния, освободит ТГ и произведёт загрузку данных в буферный регистр А1 (можно это сделать заранее до сигнала готовности со стороны КВУ), и после в регистр сдвига.

Передаваемые биты записываются с 1 по 8 биты сдвигового регистра, в нулевой разряд помещается 0 как стартовый бит, а в 9,10 биты – константные значения стоп-битов.

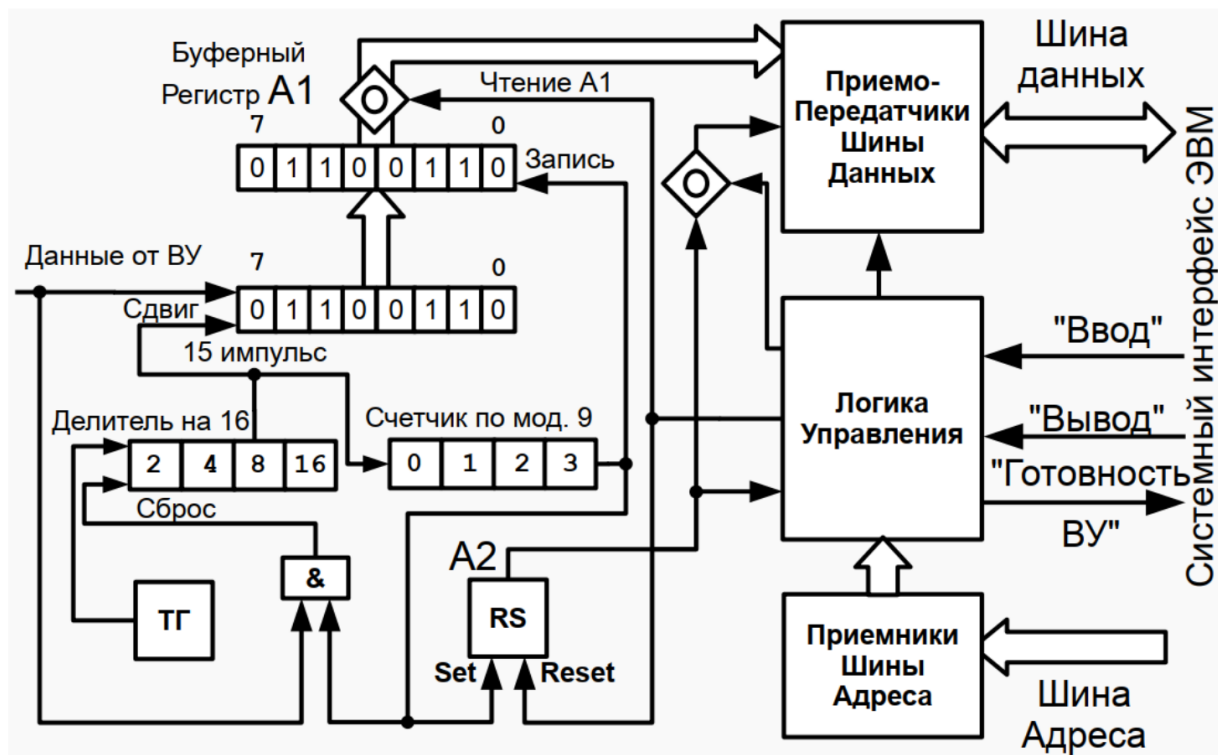
При начале передачи счётчик по модулю обнулится, и прежде пройдёт 10 тактов ТГ, пока не будет сброшено состояние вновь. По ходу этих тактов по шине передадутся все 11 битов. Дальше цикл обмена повторится.

Счётчик по модулю 10.



Счётчик по модулю 10 реализован с помощью обычного счётчика, однако к нему добавления схема логического “И”, проверяющая конкретную комбинацию значений. В нашем случае 1010, что равно как раз 10. При появлении такой комбинации схема срабатывает и отправляется сигнал сброса состояния счётчика. Таким образом, можно реализовать любой счётчик по модулю.

Контроллера приёма асинхронного последовательного интерфейса.



Помним про формат кадра, что обычное состояние шины данных – это единица. С предыдущего шага сигнал с счётчика по модулю 9 равен единице.

Оба вышеописанных сигнала поступают на схему логического “И”, которая уже подаёт управляющий сигнал “Сброс” на схему делителя частоты ТГ (по аналогии с предыдущим с контроллером).

Первый бит, который поступит со стороны ВУ – это стартовый бит, равный 0. Сигнал сброса сбрасывается (тавтология, знаю) и начинается процесс приёма данных.

Пройдёт 9 тактов (для записи 8 информационных битов и выноса стартового бита) и данные из сдвигового регистра попадут в буферный регистр A1. Одновременно с этим подаётся сигнал в регистр состояния, сигнализирующий о готовности принятия данных.

Блок логики управления видит сигнал готовности и на основе этого уже может считать данные с буферного регистра, что повлечёт за собой сброс регистра состояния, либо чтение регистра состояния.

После того, как пачка данных пришла, на шине данных со стороны ВУ будет снова единица и ТГ снова заблокируется.

Организация прямого доступа к памяти. Контроллер ПДП (DMA)

Прямой доступ к памяти (DMA – Direct Memory Access) – режим обмена данными между устройствами вычислительной машины, либо же между устройством и основной памятью, при котором процессор не участвует в обмене. Так как данные постоянно не пересылаются в процессор и обратно, скорость обмена выше, чем разобранные выше методы обмена.

Подобный обмен реализуется полностью аппаратно и реализуется специальным контроллером ПДП.

Организация ПДП

Особенности ПДП:

- Возможность начальной загрузки программ в основную память ЭВМ из устройства ввода.
- Обеспечивает возможность использования в ЭВМ быстродействующих внешних запоминающих устройств (SSD например).

Для экономии ресурсов контроллер ПДП свои ресурсы, а подключается к шинам данных и шине адреса системного интерфейса ЭВМ. Из этого делаем вывод, что процессор и контроллер ПДП не могут одновременно использовать эти шины.

Проблема решается либо захватом цикла, либо блокировкой процесса.

Захват цикла

Простой захват цикла – передача происходит в те машинные циклы, в которых процессор не обменивается данными с памятью. Пометка цикла выполняются либо при помощи специальных управляющих сигналов, указывающие циклы, во время которых процессор не использует память, либо при помощи специальных селектирующих схем внутри ПДП, что усложняет конструкцию процессора.

При таком организации обмена ПДП не снижает производительности машины, но обмен возможен только в случайный момент времени одиночными байтами или словами.

Захват с принудительным отключением процессора от шин системного интерфейса – для реализации такого режима ПДП системный интерфейс дополняется двумя линиями для передачи управляющих сигналов “Требование ПДП” (ТПДП) и “Предоставление ПДП” (ППДП).

Требование ПДП формируется контроллером ПДП. Получив данный сигнал, процессор приостанавливает выполнение очередной команды, не дожидаясь её завершения, выдаёт сигнал предоставления ПДП и отключается от системного интерфейса.

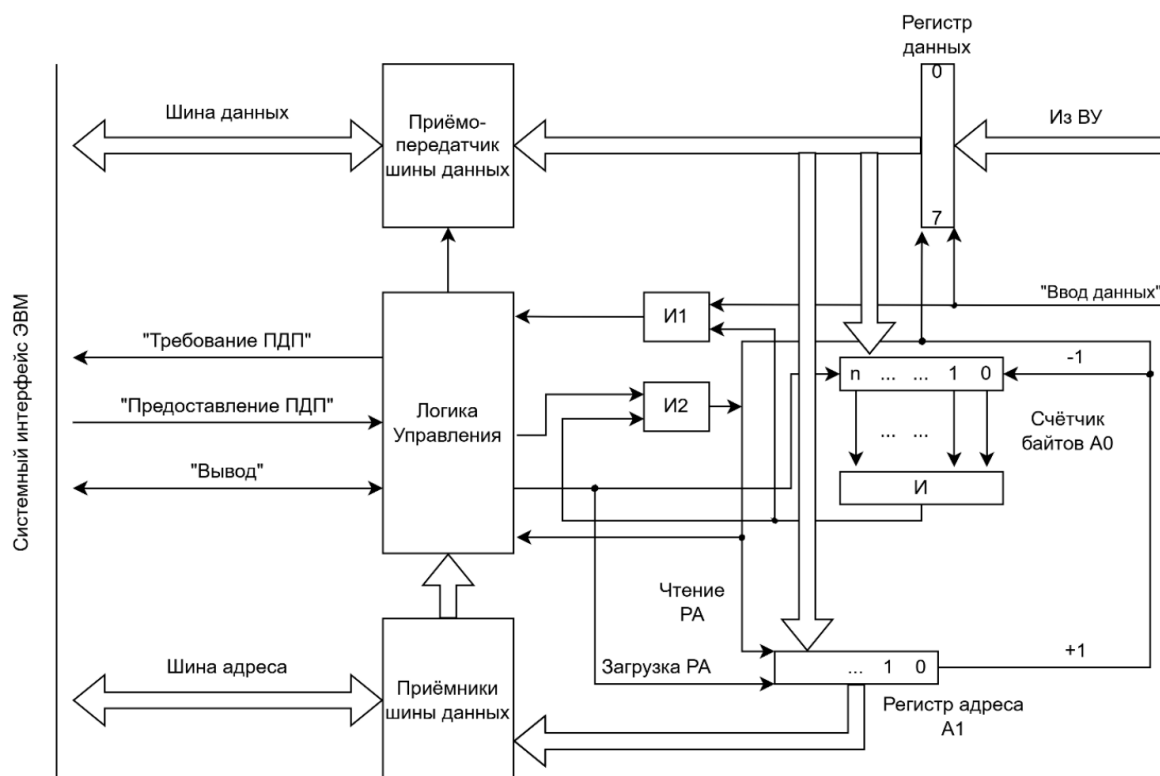
Контроллер ПДП получает доступ к шинам системного интерфейса и осуществляет обмен одним байтом или словом с памятью ЭВМ.

После контроллер ПДП возвращает управление процессору над системным интерфейсом

Блокировка процесса

Отличается от захвата цикла тем, что управление над системным интерфейсом передаётся контроллеру ПДП не на время обмена одним байтом, а на время обмена блоком данных.

Контроллер ПДП ввода данных из ВУ в режиме “Захват Цикла”



1. Процессор загружает в счётчик байтов количество принимаемых байтов, а в регистр адреса помещается начальный адрес области памяти для вводимых данных.
 2. Байты данных из ВУ поступают в регистр данных контроллера, при этом каждый байт сопровождается управляющим сигналом из ВУ “Ввод данных”, который обеспечивает запись байта в регистр данных. По этому же сигналу при ненулевом состоянии счётчика байтов формируется сигнал “Требование ПДП”.
 3. По ответному сигналу процессора “Предоставление ПДП” контроллер выставляет на шину адреса и шину данных содержимое регистра адреса A1 и регистра данных соответственно.
 4. Формируя сигнал приказ “Выход” контроллер ПДП обеспечивает запись байта данных из своего регистра данных в память ЭВМ.
 5. По этому же сигналу “Предоставление ПДП” счётчик байтов декрементируется, а содержимое регистра адреса инкрементируется.
- Как только счётчик байтов станет равным нулю, контроллер прекратит формирование сигналов “Требование ПДП”.